

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH**



BÁO CÁO ĐỒ ÁN TỐT NGHIỆP

**BẢO VỆ TÍNH RIÊNG TƯ TRONG MÔ
HÌNH HỌC LIÊN KẾT PHI TẬP TRUNG**

Nghành: Khoa học máy tính

HĐ: Hội đồng 8 Khoa học Máy tính

CTHD: Nguyễn Thị Ái Thảo

TKHD: Trần Thị Quế Nguyệt

UVHD: Trương Quỳnh Chi

PBHD: Trần Thị Quế Nguyệt

—o0o—

GVHD: Nguyễn Thị Ái Thảo

SVTH 1: Tạ Đình Tiến - 2110583

SVTH 2: Trương Đức Dũng - 2113080

SVTH 3: Phạm Bá Hoàng - 2113414

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 5 / 2025

Lời cam kết

Chúng tôi xin cam đoan rằng nghiên cứu này là công trình của riêng chúng tôi, được thực hiện dưới sự giám sát và hướng dẫn của Tiến sĩ Nguyễn Thị Ái Thảo. Kết quả nghiên cứu của chúng tôi là hợp pháp và chưa từng được công bố dưới bất kỳ hình thức nào trước đây. Tất cả các tài liệu sử dụng trong nghiên cứu này đều do chúng tôi tự thu thập từ nhiều nguồn khác nhau và đã được liệt kê đầy đủ trong phần tài liệu tham khảo. Ngoài ra, trong quá trình nghiên cứu, chúng tôi cũng đã sử dụng kết quả của một số tác giả và tổ chức khác. Tất cả các nguồn tài liệu này đều đã được tham khảo và trích dẫn một cách thích hợp.

Chúng tôi hoàn toàn chịu trách nhiệm về mọi hành vi sao chép hoặc vi phạm bản quyền trong nghiên cứu này. Trường Đại học Bách Khoa Thành phố Hồ Chí Minh không chịu trách nhiệm về bất kỳ vi phạm bản quyền nào mà chúng tôi có thể đã thực hiện trong quá trình nghiên cứu. Chúng tôi cam kết rằng mọi kết quả và dữ liệu trong nghiên cứu này đều là chính xác và trung thực, phản ánh đúng quá trình nghiên cứu của chúng tôi.

Lời cảm ơn

Chúng tôi xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Tiến sĩ Nguyễn Thị Ái Thảo, người đã tận tình hướng dẫn và hỗ trợ chúng tôi trong suốt quá trình thực hiện nghiên cứu này. Sự chỉ dẫn tận tâm và những góp ý quý báu của cô đã giúp chúng tôi hoàn thiện nghiên cứu một cách tốt nhất và đạt được những kết quả như mong đợi.

Chúng tôi cũng xin bày tỏ lòng biết ơn đến các thầy cô trong khoa Khoa học Máy tính, Trường Đại học Bách Khoa Thành phố Hồ Chí Minh, đã tạo điều kiện thuận lợi và cung cấp các nguồn tài liệu cần thiết cho nghiên cứu này. Sự hỗ trợ và động viên của các thầy cô là nguồn động lực to lớn giúp chúng tôi vượt qua mọi khó khăn trong quá trình nghiên cứu.

Đặc biệt, chúng tôi xin gửi lời cảm ơn đến gia đình và bạn bè, những người đã luôn động viên và ủng hộ chúng tôi trong suốt quá trình học tập và nghiên cứu. Sự hỗ trợ và khích lệ của mọi người là nguồn động lực vô cùng quý báu, giúp chúng tôi vượt qua mọi thử thách và hoàn thành nghiên cứu này.

Chúng tôi cũng xin cảm ơn các tác giả và tổ chức đã cung cấp các tài liệu và kết quả nghiên cứu mà chúng tôi đã tham khảo và sử dụng trong nghiên cứu này. Tất cả các nguồn tài liệu này đều đã được trích dẫn một cách thích hợp trong phần tài liệu tham khảo, và chúng tôi xin bày tỏ lòng biết ơn sâu sắc đến những đóng góp của họ.

Cuối cùng, chúng tôi xin cảm ơn tất cả những người đã đóng góp ý kiến và hỗ trợ chúng tôi trong quá trình thực hiện nghiên cứu này. Sự giúp đỡ và đóng góp của mọi người là vô cùng quý báu và đáng trân trọng, giúp chúng tôi hoàn thành nghiên cứu một cách tốt nhất.

Tóm tắt

Học liên kết phi tập trung (Decentralized Federated Learning - DFL) đang thu hút sự quan tâm nhờ khả năng tăng cường bảo mật và chống chịu trước các mối đe dọa trong môi trường học máy phân tán. Báo cáo này tập trung vào việc đề xuất và phân tích các giải pháp bảo vệ mô hình và các node tham gia DFL trước các tấn công đối kháng như Byzantine, model poisoning, cũng như các tấn công trên mạng như nghe lén (eavesdropping), tấn công người trung gian (Man-in-the-Middle - MITM), lập bản đồ mạng (network mapping), và che lấp (obfuscation).

Báo cáo trình bày tổng quan về các thuật toán tổng hợp mô hình (model aggregation) như Federated Averaging, Krum, Trimmed Mean, cùng các phương pháp tiên tiến như FLTrust, UBAR, Self-Centered Clipping, nhằm tăng cường khả năng chống chịu trước các tấn công đối kháng trong DFL.

Đặc biệt, báo cáo đề xuất giải pháp kết hợp hai kỹ thuật BALANCE và Moving Target Defense (MTD) nhằm bảo vệ hiệu quả mô hình và các node tham gia trước các cuộc tấn công đối kháng và tấn công mạng. BALANCE giúp loại bỏ các mô hình độc hại và xử lý tốt dữ liệu IID lẫn non-IID, trong khi MTD tăng cường khả năng phòng thủ động trước các tấn công trên mạng.

Thuật toán kết hợp BALANCE và MTD được trình bày chi tiết, cùng với phân tích lợi ích, hạn chế và các thách thức triển khai thực tế. Báo cáo cũng mở ra các hướng nghiên cứu tiếp theo như tối ưu hóa BALANCE, phát triển các kỹ thuật bảo mật mới cho hệ thống phi tập trung, và đánh giá thực nghiệm toàn diện hệ thống DFL.

Ngoài ra, các vấn đề về bảo vệ quyền riêng tư và các kỹ thuật liên quan như mã hóa đồng nhất (HE), tính toán bảo mật (SMC), và độ nhiễu sai biệt (DP) cũng được đề cập nhằm hoàn thiện bức tranh tổng thể về bảo mật trong DFL.

Nhìn chung, hệ thống đề xuất góp phần nâng cao khả năng phòng thủ chủ động trước các mối đe dọa đối kháng và tấn công mạng, đảm bảo an toàn cho mô hình và các node trong môi trường học máy phi tập trung.

Mục lục

Lời cam kết	i
Lời cảm ơn	ii
Tóm tắt	iii
Mục lục	iv
Danh sách bảng	viii
Danh sách hình vẽ	ix
1 Giới thiệu	1
1.1 Vấn đề nghiên cứu và phạm vi	1
1.2 Mục tiêu của đề tài	2
1.3 Yêu cầu của đề tài	2
1.4 Tổng quan về các nghiên cứu	3
2 Kiến thức nền tảng	5
2.1 Tổng quan về học liên kết (Federated Learning)	5
2.2 Tổng quan về học liên kết phi tập trung (Decentralized Federated Learning) . .	6
2.2.1 Bài toán tối ưu trong Học Liên Kết (FL)	6
2.2.2 Cơ chế hoạt động	7
2.3 Thuật toán tối ưu hóa trong huấn luyện học liên kết	8
2.3.1 Gradient Descent	10
2.3.2 Stochastic Gradient Descent	13
2.3.3 Tại sao lại sử dụng SGD trong học liên kết phi tập trung?	14
2.4 Tổng hợp mô hình trong học liên kết phi tập trung	15
2.4.1 Khái Niệm Về Tổng Hợp Mô Hình (Model Aggregation)	15
2.4.2 Luật tổng hợp	16

2.4.2.1	Federated Averaging	16
2.4.2.2	Krum	16
2.4.2.3	Trimmed Mean	17
2.4.2.4	Median	18
2.4.2.5	FLTrust	18
2.4.2.6	UBAR	19
2.4.2.7	Learn	20
2.4.2.8	Self-Centered Clipping	21
3	Các bài toán trong Học liên kết phi tập trung	23
3.1	Chi phí tính toán và truyền thông	23
3.2	Mạng lưới và Kết nối	24
3.2.1	Thách thức liên quan đến Cấu trúc liên kết Mạng (Network Topology) .	25
3.2.2	Thách thức liên quan đến Giao thức Giao tiếp (Communication Protocols)	26
3.3	Hiệu năng và hội tụ	28
3.4	Vấn đề về tính riêng tư dữ liệu	28
3.4.1	Thách thức	28
3.4.2	Giải pháp tiềm năng	29
3.5	Vấn đề về tấn công đối kháng	31
3.5.1	Thách thức	31
3.5.2	Giải pháp tiềm năng	32
3.6	Kết luận	33
4	Các nghiên cứu liên quan	35
4.1	Tổng quan về phương pháp BALANCE	36
4.1.1	Hạn chế của các phương pháp tổng hợp trước đó	36
4.1.2	Giải thuật BALANCE	36
4.1.3	Thử nghiệm và đánh giá hiệu năng	38
4.1.3.1	Dataset	38
4.1.3.2	Mô hình tấn công	39
4.1.3.3	Thang điểm đánh giá	39
4.1.3.4	Kết quả đánh giá	40
4.1.4	Mở rộng về xử lý dữ liệu IID và non-IID	41
4.2	Kỹ thuật mã hóa và thay đổi mục tiêu phòng thủ (MTD)	41
4.2.1	Vấn đề rủi ro trong hệ thống phi tập trung	42
4.2.2	Kỹ thuật mã hóa và MTD trong DFL	44
4.2.2.1	Mã hóa và các kỹ thuật bảo vệ	44

MỤC LỤC

4.2.2.2	Kỹ thuật MTD	45
4.2.2.3	Thuật toán được đề xuất trong hệ thống DFL	46
4.2.2.4	Đánh giá các kỹ thuật đối với các cuộc tấn công	48
4.3	Phân tích tổng quan giữa BALANCE và MTD	49
5	Đề xuất nghiên cứu	51
5.1	Đề xuất giải pháp kết hợp BALANCE, MTD và Mã hóa	51
5.1.1	Cơ chế hoạt động của framework đề xuất	52
5.1.2	Thuật toán tổng hợp BALANCE với TLS 1.3 và Coin-Flipping	53
5.1.3	Sơ đồ trình tự hệ thống	55
5.2	Phân tích kỹ thuật tích hợp	56
5.2.1	Phân tích cơ chế giao tiếp đảm bảo tính nhất quán	56
5.2.1.1	Locking (Khóa)	56
5.2.1.2	Async/Await Coordination	56
5.2.1.3	Timeout Handling (Xử lý hết thời gian)	57
5.2.2	Áp dụng giải thuật BALANCE	57
5.2.2.1	Khởi tạo lớp BALANCE (Phần <code>__init__</code>)	57
5.2.2.2	Hàm tính toán khoảng cách giữa hai mô hình	58
5.2.2.3	Quá trình tổng hợp mô hình	58
5.2.3	Kỹ thuật mã hóa	60
5.2.3.1	Phân tích hướng tiếp cận ban đầu	60
5.2.3.2	Đề xuất giao thức ban đầu	61
5.2.3.3	Đánh giá sơ bộ và góp ý cho giao thức ban đầu	63
5.2.3.4	Giao thức TLS	64
5.2.3.5	Triển khai TLS 1.3 trong hệ thống	67
5.2.4	Kỹ thuật MTD	70
5.2.4.1	Phân tích hướng tiếp cận ban đầu	70
5.2.4.2	Đề xuất giao thức cho kỹ thuật neighbor selection	72
5.2.4.3	Đánh giá và nhận xét giao thức được đề xuất	74
5.2.4.4	Triển khai giao thức	77
5.3	Phân tích tính bảo mật của hệ thống	80
5.3.1	Hiệu quả bảo mật của giải pháp	80
5.3.2	Phân tích rủi ro và hạn chế trong bảo mật	82
6	Đánh giá thử nghiệm	85
6.1	Giới thiệu nền tảng Nebula (Fedstellar)	86
6.1.1	Triển khai thí nghiệm FL	87

6.2	Thiết lập thử nghiệm	89
6.2.1	Tập dữ liệu và mô hình	89
6.2.2	Cấu trúc mạng	90
6.2.3	Cơ chế tổng hợp	90
6.2.4	Các cuộc tấn công	91
6.2.5	Thang đo đánh giá	95
6.2.6	Triển khai hệ thống mô phỏng trên AWS EC2	95
6.3	Kết quả thử nghiệm	96
6.3.1	Đánh giá hiệu quả giải thuật tổng hợp BALANCE	96
6.3.2	Đánh giá hiệu suất mô hình, chi phí tài nguyên khi kết hợp Mã hoá và MTD	99
6.3.3	Ảnh hưởng của các tham số khác	102
6.3.3.1	Cấu trúc mạng	102
6.3.3.2	Tỉ lệ khách hàng độc hại	102
6.3.3.3	Tỉ lệ chấp nhận hàng xóm trong MTD	103
6.3.3.4	Các tham số trong BALANCE	103
7	Kết luận	105
7.1	Kết luận	105
7.2	Hướng phát triển cho tương lai	106
	Tài liệu tham khảo	107

Danh sách bảng

2.3.1	Ưu và nhược điểm trong SGD	14
3.4.1	Phân tích tính khả thi của các cơ chế bảo vệ về riêng tư dữ liệu trong DFL ^[1]	31
4.2.1	Thông tin có thể truy cập bởi một client "xấu" trong DFL ^[2]	42
4.2.2	Các cuộc tấn công, mục tiêu và thông tin có thể bị khai thác trong DFL ^[2]	43
4.2.3	Các biện pháp giảm thiểu tiềm năng cho các cuộc tấn công trong DFL ^[2]	48
6.3.1	So sánh F1 Score giữa các phương pháp tổng hợp	96
6.3.2	Bảng so sánh thời gian huấn luyện (phút) giữa các phương pháp tổng hợp	97
6.3.3	Bảng so sánh mức sử dụng CPU tối đa (%) giữa các phương pháp tổng hợp . . .	98
6.3.4	Bảng so sánh mức sử dụng RAM tối đa (%) giữa các phương pháp tổng hợp . . .	98
6.3.5	Bảng so sánh số lượng bytes gửi/nhận (MB) giữa các phương pháp tổng hợp . . .	98
6.3.6	Bảng so sánh số lượng gói tin gửi/nhận (MB) giữa các phương pháp tổng hợp . . .	99
6.3.7	Hiệu suất hệ thống dưới các cấu hình bảo mật khác nhau (luật tổng hợp FedAvg và không chịu tấn công)	99
6.3.8	Hiệu suất hệ thống dưới các cấu hình bảo mật khác nhau (luật tổng hợp Balance và không chịu tấn công)	100
6.3.9	Hiệu suất hệ thống dưới các cấu hình bảo mật khác nhau (luật tổng hợp FedAvg và tấn công Gauss)	100
6.3.10	Hiệu suất hệ thống dưới các cấu hình bảo mật khác nhau (luật tổng hợp Balance và tấn công Gauss)	100
6.3.11	Ảnh hưởng của cấu trúc mạng đối với luật tổng hợp BALANCE	102
6.3.12	Ảnh hưởng của tỉ lệ khách hàng độc hại đối với luật tổng hợp BALANCE	102
6.3.13	Ảnh hưởng của tỉ lệ chấp nhận hàng xóm trong MTD	103

Danh sách hình vẽ

2.3.1	Cực trị đơn giản	9
2.3.2	Suy giảm độ dốc	10
2.3.3	Vấn đề suy giảm độ dốc	11
2.3.4	Vấn đề chọn tốc độ học	11
2.3.5	SGD and BGD	14
3.2.1	Các cấu trúc mạng trong hệ thống Học Liên kết Phi tập trung ^[3]	25
4.1.1	Kết quả của các phương pháp tổng hợp khác nhau cho tập dữ liệu tổng hợp ^[4] . .	40
4.1.2	Kết quả của các phương pháp tổng hợp khác nhau cho tập dữ liệu MNIST ^[4] . . .	41
5.1.1	Sơ đồ hệ thống DFL với BALANCE, TLS 1.3 và MTD	52
5.1.2	Sơ đồ trình tự mô phỏng quá trình giao tiếp và huấn luyện giữa các node trong hệ thống DPFL	55
5.2.1	Tấn công man in the middle trong triển khai ban đầu	61
5.2.2	TLS 1.2 với TLS 1.3 ^[5]	65
5.2.3	Thống kê về các web server sử dụng các loại giao thức TLS vào năm 2024 ^[6] . . .	67
5.2.4	Quá trình trao đổi model bị gián đoạn	71
6.1.1	Tổng quan về kiến trúc nền tảng Nebula (Fedstellar) ^[7]	86

Chương 1

Giới thiệu

1.1 Vấn đề nghiên cứu và phạm vi

Decentralized Privacy-Preserving Federated Learning (DPFL) là một lĩnh vực tiên tiến trong học máy, nhằm giải quyết đồng thời nhu cầu bảo vệ quyền riêng tư và những hạn chế của hệ thống liên kết tập trung truyền thống. Thay vì phụ thuộc vào một máy chủ trung tâm để điều phối và tổng hợp các bản cập nhật mô hình, DPFL vận hành theo cách hoàn toàn phi tập trung, sử dụng giao tiếp ngang hàng giữa các nút tham gia. Mô hình này không chỉ giảm thiểu rủi ro về tính tập trung mà còn tăng cường khả năng bảo vệ dữ liệu cá nhân.

Phạm vi nghiên cứu của báo cáo này bao gồm:

1. Tổng quan về các khái niệm và kiến trúc cơ bản của DFL
 - Giới thiệu mô hình học liên kết phi tập trung (FL).
 - Kiến trúc hoạt động của DFL, sự khác biệt so với học liên kết tập trung (FL).
 - Các thành phần chính trong DFL và cơ chế trao đổi thông tin giữa các nút.
2. Quá trình giao tiếp và đồng bộ hóa mô hình giữa các nút
 - Cơ chế tổng hợp mô hình giữa các nút.
 - Phương thức trao đổi thông tin trong môi trường phân tán.
 - Các thuật toán phổ biến được sử dụng trong quá trình đồng bộ hóa mô hình.
3. Các bài toán thường gặp trong Học liên kết phi tập trung
4. Các mô hình tấn công trong DFL
5. Các phương pháp bảo vệ dữ liệu và mô hình trong DFL

1.2. Mục tiêu của đề tài

1.2 Mục tiêu của đề tài

Đề tài **Decentralized Privacy-Preserving Federated Learning - DPFL** nhằm nghiên cứu và phát triển một hệ thống học liên kết phi tập trung đáp ứng các yêu cầu sau:

- Đảm bảo độ chính xác và tin cậy của mô hình tổng hợp trong môi trường phân tán.
- Có khả năng phòng thủ hiệu quả trước các hình thức tấn công đối kháng (adversarial attacks) trong quá trình học tập.
- Đảm bảo hiệu năng tổng thể của hệ thống dưới các kịch bản khác nhau.
- Tăng tính bảo mật, riêng tư của dữ liệu trước các cuộc tấn công như model inversion, membership inference.
- Đảm bảo tính nhất quán và hiệu quả của quá trình giao tiếp và đồng bộ giữa các nút trong mạng phân tán.

1.3 Yêu cầu của đề tài

Để đạt được các mục tiêu đã nêu, đề tài đặt ra các yêu cầu cụ thể như sau:

- Nghiên cứu các khái niệm cơ bản và thuật toán trong học liên kết phi tập trung (DFL), đặc biệt là các phương pháp tổng hợp mô hình và các chiến lược bảo mật liên quan.
- Xây dựng cơ chế đồng thuận đảm bảo quá trình tổng hợp mô hình giữa các nút mạng trong DFL là đáng tin cậy, đồng thời bảo vệ quyền riêng tư và bảo mật của dữ liệu người dùng.
- Khám phá và phân tích các công nghệ bảo vệ quyền riêng tư trong DFL, bao gồm các kỹ thuật mã hóa, tính toán đa bên an toàn (SMC), và các phương pháp phòng thủ như Moving Target Defense (MTD).
- Đánh giá các rủi ro bảo mật liên quan đến DFL, đặc biệt là các cuộc tấn công mô hình (model poisoning) và các phương pháp bảo vệ để giảm thiểu các mối đe dọa này.
- Thử nghiệm, so sánh và đánh giá hiệu quả của giải pháp bảo mật đề xuất dưới các kịch bản mô phỏng khác nhau, nhằm kiểm tra tính khả thi và hiệu suất của hệ thống DFL với các cơ chế bảo vệ.

1.4 Tổng quan về các nghiên cứu

Tổng hợp các hướng nghiên cứu liên quan

Từ quá trình thu thập tài liệu, chúng tôi nhận thấy chúng có thể được chia thành bốn nhóm chính:

- **Tổng quan về học liên kết và các bài toán liên quan:** Nghiên cứu khái quát về mô hình Federated Learning (FL), bao gồm quy trình giao tiếp, tối ưu hóa phân tán và các thách thức chính của FL.
- **Thuật toán tổng hợp mô hình an toàn:** Bao gồm các phương pháp chống tấn công poisoning như BALANCE, Krum, Trimmed Mean, ..., đảm bảo tính toàn vẹn và độ tin cậy của mô hình trong môi trường có tác nhân độc hại.
- **Chiến lược phòng thủ chủ động với Moving Target Defense (MTD):** Áp dụng cơ chế thay đổi liên tục về cấu trúc mạng, vai trò node, hoặc các tham số giao tiếp để làm khó khả năng thực hiện tấn công có chủ đích.
- **Các cuộc tấn công đối kháng và biện pháp phòng thủ:** Tập trung vào các hình thức tấn công đối kháng (adversarial attacks) như Byzantine attack, Model Poisoning,... hay tấn công vào dữ liệu như Membership Inference attack, Deep Leakage from Gradients,... và nghiên cứu các phương pháp phòng thủ như lựa chọn các mô hình tổng hợp tốt hoặc các kỹ thuật như Differential Privacy (DP), Secure Multiparty Computation (SMC) hay Homomorphic Encryption (HM).

Phương pháp tiếp cận và đóng góp của báo cáo

Dựa trên tổng quan các hướng nghiên cứu trong lĩnh vực học liên kết phi tập trung, báo cáo này đề xuất một hướng tiếp cận tích hợp nhằm nâng cao bảo mật và khả năng phòng thủ của hệ thống. Cụ thể, báo cáo tập trung vào việc kết hợp hai thành phần chính: thuật toán tổng hợp mô hình BALANCE và chiến lược Moving Target Defense (MTD).

Sự kết hợp này được kỳ vọng sẽ mang lại các đóng góp sau:

- Tăng cường độ ổn định và độ chính xác của quá trình huấn luyện mô hình trong môi trường phi tập trung thông qua cơ chế tổng hợp BALANCE.
- Giảm thiểu rủi ro từ các cuộc tấn công đối kháng bằng cách làm nhiễu môi trường huấn luyện và phân phối thông tin thông qua kỹ thuật MTD.

1.4. Tổng quan về các nghiên cứu

- Cung cấp một giải pháp bảo mật toàn diện, vừa đảm bảo tính toàn vẹn của mô hình, vừa bảo vệ quyền riêng tư dữ liệu người dùng trong suốt quá trình học tập.
- Đánh giá thực nghiệm hiệu quả của phương pháp đề xuất trên các kịch bản mô phỏng tấn công khác nhau để xác định khả năng áp dụng trong thực tiễn.

Báo cáo không chỉ đề xuất mô hình tích hợp mà còn hướng đến việc phân tích chi tiết các thành phần và vai trò của từng kỹ thuật trong việc nâng cao tính bảo mật cho hệ thống DPFL.

Chương 2

Kiến thức nền tảng

2.1 Tổng quan về học liên kết (Federated Learning)

Federated Learning (FL) là một phương pháp học máy phân tán, trong đó các thiết bị đầu cuối (clients) hợp tác để huấn luyện một mô hình toàn cục chung mà không cần chia sẻ dữ liệu của họ. Thay vì chuyển dữ liệu thô từ các thiết bị về một máy chủ trung tâm, Federated Learning thực hiện huấn luyện mô hình trên dữ liệu cục bộ và chỉ gửi các tham số mô hình đã cập nhật đến một máy chủ trung tâm để tổng hợp. Phương pháp này giúp bảo vệ sự riêng tư của dữ liệu và giảm thiểu băng thông cần thiết cho việc truyền dữ liệu, đặc biệt phù hợp trong bối cảnh dữ liệu lớn và sự gia tăng của các thiết bị IoT (Internet of Things). Các nghiên cứu trước đây như FedAvg, FedProx đã cải tiến khả năng tổng hợp mô hình và xử lý dữ liệu không đồng nhất. Tuy nhiên, FL truyền thống gặp phải một số hạn chế đáng kể:

- **Rủi ro điểm thất bại duy nhất (Single point of failure):** Phụ thuộc rất nhiều vào một máy chủ trung tâm để quản lý và tổng hợp các bản cập nhật mô hình từ tất cả các thiết bị (client). Do đó, nếu máy chủ trung tâm này gặp sự cố hoặc bị tấn công, toàn bộ hệ thống Học liên kết sẽ bị ảnh hưởng.
- **Điểm nghẽn truyền thông:** Máy chủ trung tâm có thể trở thành Nút thắt cổ chai (bottleneck) khi quy mô hệ thống tăng lên. Đồng thời, việc truyền và đồng bộ hóa dữ liệu mô hình từ nhiều client cũng tốn nhiều băng thông, và có thể bị chậm trễ hoặc mất kết nối, dẫn đến giảm chất lượng mô hình.
- **Những thách thức trong tính bảo mật và quyền riêng tư dữ liệu:** Mặc dù FL không chia sẻ dữ liệu thô, việc truyền các gradient hoặc weights vẫn có thể tiết lộ thông tin về dữ liệu cá nhân (dựa trên các tấn công kiểu suy luận hoặc reconstruction).

2.2 Tổng quan về học liên kết phi tập trung (Decentralized Federated Learning)

Để giải quyết các hạn chế trên, Decentralized Federated Learning (DFL) được đề xuất như một hướng tiếp cận thay thế. DFL loại bỏ nhu cầu về máy chủ trung tâm và cho phép các clients giao tiếp và tổng hợp mô hình theo mô hình ngang hàng (peer-to-peer).

Các Thành Phần của Hệ Thống DFL

- **Client ($\mathcal{V}$):** Đây là các thành viên tham gia vào quá trình học. Mỗi client có một tập dữ liệu riêng.
- **Đồ Thị Giao Tiếp ($G = (\mathcal{V}, \mathcal{E})$):** Xác định cách các client kết nối với nhau. Một cạnh giữa hai client có nghĩa là họ có thể giao tiếp trực tiếp.
- **Tham Số Mô Hình (w):** Đây là các trọng số hoặc thiết lập của mô hình học máy mà các client nhằm tối ưu hóa.

Cấu trúc này mang lại nhiều lợi ích như:

- Tăng cường khả năng mở rộng: Hệ thống có thể hoạt động hiệu quả với số lượng lớn clients mà không gặp phải điểm nghẽn truyền thông.
- Giảm thiểu rủi ro tập trung: Loại bỏ nguy cơ điểm thất bại duy nhất nhờ tính chất phân tán.
- Tăng cường bảo mật và quyền riêng tư: Giảm sự phụ thuộc vào một điểm tin cậy duy nhất.

Các nghiên cứu gần đây như LEARN, UBAR, và SCCLIP đã đề xuất các phương pháp tổng hợp mô hình Byzantine-robust để nâng cao khả năng chống chịu của DFL trước các cuộc tấn công độc hại. Tuy nhiên, DFL vẫn tồn tại những thách thức lớn về bảo mật, đặc biệt là đối với các cuộc tấn công model poisoning, trong đó kẻ tấn công gửi các mô hình bị thao túng để làm suy giảm hiệu suất hệ thống.

2.2.1 Bài toán tối ưu trong Học Liên Kết (FL)

Trong **Học Liên Kết (Federated Learning - FL)**, quá trình huấn luyện mô hình có thể được mô tả như một bài toán **tối ưu hóa rủi ro thực nghiệm** (Empirical Risk Minimization - ERM). Mục tiêu là tìm một mô hình w^* sao cho hàm tối ưu đạt giá trị nhỏ nhất, được biểu diễn như sau:

2.2. Tổng quan về học liên kết phi tập trung (Decentralized Federated Learning)

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \Theta} F(\mathbf{w}) = \frac{1}{|D|} \sum_{\zeta \in D} f(\mathbf{w}, \zeta), \quad (1)$$

Trong đó:

- $\Theta \subset \mathbb{R}^d$ là không gian tham số của mô hình, với d là số chiều của tham số.
- $F(\cdot)$ là hàm rủi ro tổng quát (population risk function).
- D là toàn bộ tập dữ liệu huấn luyện.
- $f(\mathbf{w}, \zeta)$ là hàm mất mát thực nghiệm, được tính toán bằng tham số mô hình $\mathbf{w}$ và một mẫu huấn luyện ζ .

Bài toán tối ưu này thể hiện rõ mục tiêu trong FL: tìm tham số mô hình $\mathbf{w}^*$ để giảm thiểu sai số trung bình trên toàn bộ tập dữ liệu D mà không cần tập trung dữ liệu từ các thiết bị.

2.2.2 Cơ chế hoạt động

Trong mô hình **Federated Learning (FL)** truyền thống, quá trình huấn luyện được điều phối bởi một máy chủ trung tâm. Tuy nhiên, **Decentralized Federated Learning (DFL)** loại bỏ sự phụ thuộc này bằng cách cho phép các client trao đổi mô hình theo cách ngang hàng (peer-to-peer) mà không cần sự hỗ trợ của máy chủ.

Cụ thể, hệ thống DFL bao gồm một tập hợp các client $\mathcal{V}$, với $|\mathcal{V}|$ là số lượng client trong hệ thống. Mạng lưới DFL được biểu diễn dưới dạng một đồ thị vô hướng và không có trọng số $G = (\mathcal{V}, \mathcal{E})$, trong đó $\mathcal{E}$ là tập hợp các cạnh kết nối giữa các client. Hai client có thể trao đổi thông tin trực tiếp với nhau nếu có một cạnh kết nối giữa chúng, và các cạnh tự nối (self-loop) không được phép.

Mỗi client $i \in \mathcal{V}$ có một tập dữ liệu huấn luyện riêng biệt $\mathcal{D}_i$. Tập dữ liệu toàn cục được biểu diễn bởi $\mathcal{D} = \bigcup_{i \in \mathcal{V}} \mathcal{D}_i$. Mỗi client i duy trì một mô hình $\mathbf{w}_i$ dựa trên dữ liệu huấn luyện cục bộ và thông tin nhận được từ các client lân cận.

Quá trình huấn luyện trong DFL diễn ra qua hai bước trong mỗi vòng lặp huấn luyện t :

1. Bước 1: Huấn luyện mô hình cục bộ và trao đổi mô hình

- Client i thực hiện huấn luyện cục bộ trên dữ liệu của mình và cập nhật mô hình tạm thời $\mathbf{w}_i^{t+1/2}$.
- Sau đó, client i gửi mô hình tạm thời này đến các client lân cận $j \in \mathcal{N}_i$, với $\mathcal{N}_i$ là tập hợp các client lân cận của i .

2.3. Thuật toán tối ưu hóa trong huấn luyện học liên kết

- Đồng thời, client i cũng nhận mô hình tạm thời $\mathbf{w}_j^{t+1/2}$ từ các client lân cận j .

2. Bước 2: Tổng hợp mô hình cục bộ

- Sau khi nhận được mô hình từ các client lân cận, client i thực hiện tổng hợp mô hình để cập nhật mô hình của mình:

$$\mathbf{w}_i^{t+1} = \alpha \mathbf{w}_i^{t+1/2} + (1 - \alpha) \text{AGG}\{\mathbf{w}_j^{t+1/2}, j \in \mathcal{N}_i\}$$

Trong đó:

- α là tham số điều chỉnh mức độ tin tưởng vào mô hình cục bộ so với mô hình nhận được từ các client lân cận.
- AGG là một quy tắc tổng hợp (aggregation rule).

Trong hệ thống DFL, mỗi client vừa thực hiện việc huấn luyện mô hình cục bộ vừa đóng vai trò như một “máy chủ tham số” để tổng hợp mô hình từ các client khác. Sau khi kết thúc quá trình huấn luyện, mỗi client sẽ giữ một mô hình đã được huấn luyện riêng biệt. Điều này khác biệt với FL truyền thống, nơi chỉ có một mô hình toàn cục được lưu trữ trên máy chủ trung tâm.

Algorithm 1 Huấn luyện cục bộ **LocalTraining**($\mathbf{w}, D, \eta$).

Output: $\mathbf{w}$.

- 1: **for** mỗi vòng lặp cục bộ **do**
 - 2: Lấy một mini-batch từ tập dữ liệu huấn luyện D để tính gradient ngẫu nhiên $g(\mathbf{w})$.
 - 3: $\mathbf{w} \leftarrow \mathbf{w} - \eta g(\mathbf{w})$.
 - 4: **end for**
-

2.3 Thuật toán tối ưu hóa trong huấn luyện học liên kết

Trong học liên kết, quá trình đào tạo mô hình thường dựa vào thuật toán giảm dần độ dốc ngẫu nhiên (SGD) để tối ưu hóa các tham số. Đây là một phương pháp phổ biến, hiệu quả trong việc giảm thiểu hàm mất mát của mô hình bằng cách cập nhật tham số dựa trên gradient cục bộ tính toán từ dữ liệu. SGD không chỉ giúp tiết kiệm tài nguyên tính toán mà còn phù hợp với các hệ thống phi tập trung, nơi dữ liệu được lưu trữ và xử lý cục bộ.

Trước tiên đã đi vào bài toán tìm cực trị:

2.3. Thuật toán tối ưu hóa trong huấn luyện học liên kết

Algorithm 2 Quy trình huấn luyện của Học Liên Kết Phân Tán (DFL).

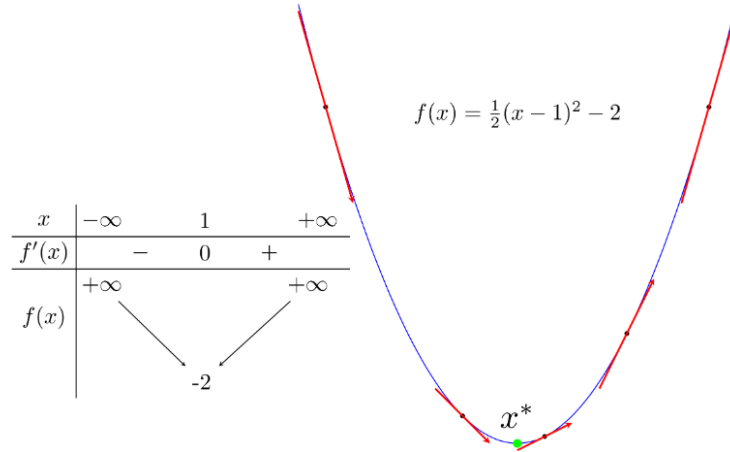
Input: Tập các client $\mathcal{V}$; dữ liệu huấn luyện cục bộ $D_i, i \in \mathcal{V}$; số vòng huấn luyện T ; tốc độ học η ; đồ thị giao tiếp $\mathcal{G}$; tham số α ; quy tắc tổng hợp AGG.

Output: Mô hình cục bộ $\mathbf{w}_i^T, i \in \mathcal{V}$.

```

1: Khởi tạo  $\mathbf{w}_i^0, i \in \mathcal{V}$ .
2: for  $t = 0, 1, \dots, T - 1$  do
3:   for mỗi client  $i \in \mathcal{V}$  (song song) do
4:     // Bước I: Huấn luyện và trao đổi mô hình cục bộ.
5:      $\mathbf{w}_i^{t+\frac{1}{2}} = \text{LocalTraining}(\mathbf{w}_i^t, D_i, \eta)$ .
6:     Gửi  $\mathbf{w}_i^{t+\frac{1}{2}}$  tới tất cả các client láng giềng  $j \in \mathcal{N}_i$ .
7:     Nhận  $\mathbf{w}_j^{t+\frac{1}{2}}$  từ tất cả các client láng giềng  $j \in \mathcal{N}_i$ .
8:     // Bước II: Tổng hợp mô hình cục bộ.
9:      $\mathbf{w}_i^{t+1} = \alpha \mathbf{w}_i^{t+\frac{1}{2}} + (1 - \alpha) \text{AGG} \left\{ \mathbf{w}_j^{t+\frac{1}{2}}, j \in \mathcal{N}_i \right\}$ .
10:   end for
11: end for

```



Hình 2.3.1: Bài toán đơn giản về tìm cực trị của một hàm số^[8]

Ở bài toán trên thì điểm cực tiểu (minimum) màu xanh được xác định thông qua việc tìm nghiệm của phương trình $f'(x) = 0$, các điểm lân cận thì có giá trị $f'(\mathcal{X}) \approx 0$

Trong Học máy (Machine Learning) nói riêng và Toán Tối Ưu (Mathematical Optimization) nói chung, chúng ta thường xuyên phải tìm giá trị các giá trị cực trị của một hàm số nào đó, tuy nhiên việc tìm cực trị của một hàm số là phức tạp hoặc không thể tìm được, do đó người ta sẽ tìm các điểm lân cận thỏa mãn gần với giá trị $f'(\mathcal{X}) \approx 0$ và coi đó như là nghiệm của phương trình qua tiêu chuẩn của mô hình.

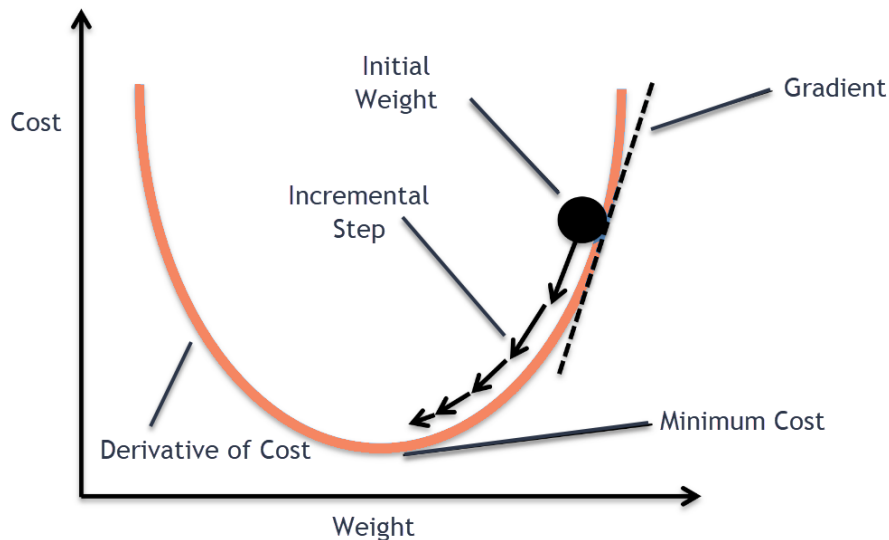
2.3. Thuật toán tối ưu hóa trong huấn luyện học liên kết

2.3.1 Gradient Descent

Gradient Descent (Suy giảm độ dốc): là một thuật toán tối ưu lặp (iterative optimization) thường được sử dụng để đào tạo các mô hình học máy và mạng nơ-ron với mục tiêu là tìm một tập các biến nội tại (internal parameters) cho việc tối ưu các mô hình.

Phương pháp lặp (iterative method) là một phương pháp toán học sử dụng phỏng đoán ban đầu để tạo ra một chuỗi các kết quả gần đúng, trong đó kết quả thứ n sẽ tốt hơn được xác định thông qua kết quả trước đó.

Các biến nội tại (internal parameters) là các giá trị khi đào tạo model được điều chỉnh để giúp model đưa ra các kết quả chính xác hơn.



Hình 2.3.2: Mô tả suy giảm độ dốc^[9]

Ý tưởng: Từ giá trị khởi đầu ban đầu xác định hướng đi và giá trị của độ dốc tức là $f'(x_0)$ sau đó cứ chạy men ngược lại với độ dốc $f'(x_0 - \delta)$ và chậm dần cho đến khi đạt giá trị cực tiểu tại $x_\delta = x_0 - \delta$ mong muốn.

Công thức:

$$x_{t+1} = x_t - \eta f'(x_t) \quad (2.3.1)$$

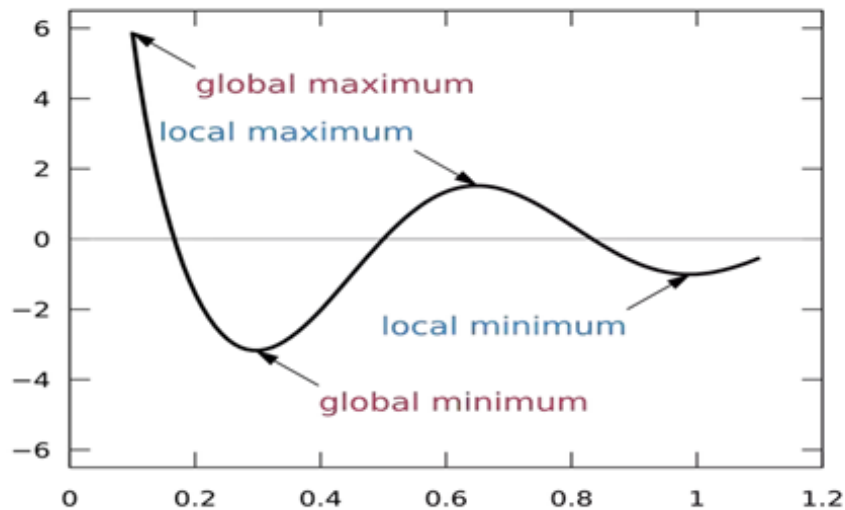
Trong đó:

- η : tốc độ học (learning rate)
- t : Thời điểm học
- $f'(x_t)$: Độ dốc (gradient) tại thời điểm t

2.3. Thuật toán tối ưu hóa trong huấn luyện học liên kết

Vấn đề được đặt ra trong suy giảm độ dốc:

- Việc lựa chọn điểm xuất phát có thể dẫn đến các giá trị cực đại hoặc cực tiểu khác nhau



Hình 2.3.3: Hàm số đồ thị đa cực trị^[9]

- Không thể hoàn toàn tiến tới các giá trị cực tiểu trong thời gian ngắn và tốc độ học không tối ưu. Có thể xảy ra các trường hợp như sau:
 - Quá thấp: độ chính xác cao, mất rất nhiều thời gian để đạt đến giá trị cực tiểu
 - Quá cao: độ chính xác thấp, biên độ dao động quá lớn, sẽ đi qua các giá trị cực tiểu và có khả năng không thể hội tụ về điểm cực tiểu



Hình 2.3.4: So sánh tốc độ học trong các khoảng xác định tương đối^[9]

2.3. Thuật toán tối ưu hóa trong huấn luyện học liên kết

Thuật toán cơ bản của GD - Batch Gradient Descent (Giảm độ dốc hàng loạt). Thuật toán này được áp dụng trên toàn bộ tập dữ liệu huấn luyện trong mỗi bước cập nhật, giúp đảm bảo hội tụ về phía điểm cực tiểu toàn cục.

Giới thiệu về hàm chi phí

Giả sử ta có m cặp dữ liệu huấn luyện $(x_i, y_i), (i = 1, \dots, m)$ được tổ chức tương ứng bằng $(X = [x_1, x_2, \dots, x_m]^T), (y = [y_1, y_2, \dots, y_m]^T \in \mathbb{R}^m)$ và $(\hat{y} \in \mathbb{R}^m)$ là kết quả dự đoán tương ứng. Ta có thể đánh giá mức độ chênh lệch kết quả $(\hat{y})$ và (y) bằng một hàm lỗi (lost function) như sau:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 = \frac{1}{2m} \sum_{i=1}^m (\theta^T \phi(x_i) - y_i)^2 \quad (2.3.2)$$

Trong đó:

- θ là vector tham số của mô hình cần tối ưu hóa.
- $\phi(x_i)$ là vector đặc trưng của dữ liệu đầu vào x_i .
- $\hat{y}_i = \theta^T \phi(x_i)$ là giá trị dự đoán của mô hình cho dữ liệu đầu vào x_i .

Thuật toán BGD hoạt động theo các bước sau:

1. **Khởi tạo:** Chọn một giá trị khởi đầu cho vector tham số θ .
2. **Thiết lập tham số:** Xác định số lần lặp tối đa và tốc độ học η để cập nhật các tham số.
3. **Tính toán gradient:** Tính gradient của hàm chi phí $J(\theta)$ theo các tham số θ dựa trên toàn bộ tập dữ liệu huấn luyện:

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m (\theta^T \phi(x_i) - y_i) \phi(x_i) \quad (2.3.3)$$

Trong đó, m là số lượng mẫu trong tập dữ liệu huấn luyện.

4. **Cập nhật tham số:** Cập nhật giá trị của θ bằng công thức:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta) \quad (2.3.4)$$

Trong đó, η là tốc độ học (learning rate), được dùng để điều chỉnh bước đi của thuật toán.

5. **Lặp lại:** Lặp lại từ bước 3 đến bước 4 cho đến khi hội tụ hoặc đạt số lần lặp tối đa.
6. **Trả về các tham số được tối ưu hóa:** Khi tiêu chí hội tụ được đáp ứng hoặc số lần lặp tối đa đạt được, hãy trả về các tham số mô hình được tối ưu hóa

2.3.2 Stochastic Gradient Descent

Stochastic Gradient Descent (SGD - Giảm dần độ dốc ngẫu nhiên), là một biến thể của thuật toán Gradient Descent được sử dụng để tối ưu hóa các mô hình học máy. Trong SGD, thay vì sử dụng toàn bộ tập dữ liệu cho mỗi lần lặp, chỉ chọn một ví dụ ngẫu nhiên duy nhất (hoặc một tập hợp nhỏ) được chọn để tính toán độ dốc và cập nhật các tham số mô hình^[10].

Thuật toán SGD hoạt động theo các bước sau:

1. **Khởi tạo:** Chọn một giá trị khởi đầu cho vector tham số θ .
2. **Thiết lập tham số:** Xác định số lần lặp và tốc độ học η để cập nhật các tham số.
3. **Lặp lại:** Lặp lại các bước sau cho đến khi mô hình hội tụ hoặc đạt đến số lần lặp tối đa:
 - a) **Trộn tập dữ liệu:** Tiến hành chọn tập dữ liệu cần được đào tạo, sau đó trộn toàn bộ tập dữ liệu được sử dụng để đưa tính ngẫu nhiên vào.
 - b) **Duyệt mẫu và tính toán:** Duyệt lần lượt các mẫu (x_i, y_i) trên tập dữ liệu đã được trộn và thực hiện tính toán:
 - i. **Tính toán gradient:** Tính gradient của hàm chi phí $J(\theta)$ theo các tham số θ dựa trên mẫu đã chọn,

$$\nabla_{\theta} J(\theta; x_i, y_i) = (\theta^T \phi(x_i) - y_i) \phi(x_i) \quad (2.3.5)$$

- ii. **Cập nhật tham số:** Cập nhật giá trị của θ bằng công thức:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta; x_i, y_i) \quad (2.3.6)$$

Trong đó, η là tốc độ học (learning rate), kiểm soát bước đi của thuật toán.

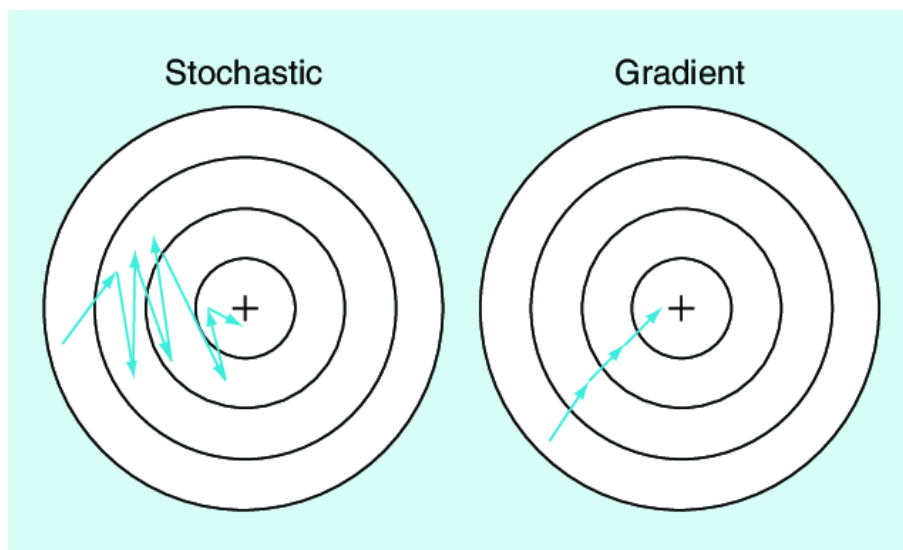
- iii. **Kiểm tra hội tụ:** Đánh giá tiêu chí hội tụ, chẳng hạn như sự thay đổi giữa các lần lặp của hàm chi phí, để xác định khi nào thuật toán dừng lại.

4. **Trả về các tham số được tối ưu hóa:** Khi tiêu chí hội tụ được đáp ứng hoặc số lần lặp tối đa đạt được, hãy trả về các tham số mô hình được tối ưu hóa

2.3. Thuật toán tối ưu hóa trong huấn luyện học liên kết

Loại	Nội dung
Ưu điểm	• Hiệu quả tính toán cao, sử dụng ít dữ liệu, giảm được chi phí tính toán và số lần lặp. • Thuật toán chạy nhanh hơn. • Hàm chi phí không ổn định, dễ tìm được cực trị toàn cục hơn.
Nhược điểm	• Độ nhiễu cao do dữ liệu ngẫu nhiên sau mỗi lần lặp. • Hàm chi phí không ổn định, không tối ưu được kết quả sau khi đào tạo.

Bảng 2.3.1: Ưu và nhược điểm trong SGD



Hình 2.3.5: So sánh tính hội tụ giữa SGD và BGD^[11]

2.3.3 Tại sao lại sử dụng SGD trong học liên kết phi tập trung?

- **Tính chất của học liên kết (FL):** Học liên kết là một phương pháp phân tán (distribution), trong đó nhiều Client cùng hợp tác để huấn luyện một mô hình chung mà không cần chia sẻ dữ liệu gốc. Mỗi Client thực hiện tính toán cục bộ bằng cách huấn luyện mô hình trên dữ liệu của nó và sau đó gửi các gradient hoặc cập nhật mô hình tới máy chủ tổng hợp để

2.4. Tổng hợp mô hình trong học liên kết phi tập trung

cập nhật mô hình toàn cục. Cách tiếp cận này giúp bảo vệ dữ liệu cá nhân và tăng cường bảo mật, bởi vì dữ liệu không cần phải được di chuyển ra ngoài các Client.

- **Hiệu quả và giảm thiểu chi phí tính toán:** SGD, với việc cập nhật mô hình dựa trên một lô nhỏ hoặc một mẫu dữ liệu, giúp giảm thiểu chi phí tính toán^[9]. Khác với BGD phải xử lý toàn bộ tập dữ liệu mỗi lần cập nhật, SGD chỉ cần một phần dữ liệu cho mỗi lần lặp. Điều này khẳng định được rằng:
 - **Giảm chi phí truyền thông:** Việc chỉ gửi các cập nhật gradient thay vì toàn bộ mô hình giúp giảm lượng dữ liệu truyền tải giữa các Client và máy chủ.
 - **Tối ưu hóa tính toán:** Mỗi Client có thể thực hiện cập nhật mô hình một cách độc lập dựa trên dữ liệu của mình, giúp phân tán tải tính toán và tránh tắc nghẽn trong hệ thống.
- **Khả năng thích ứng với dữ liệu không độc lập và phân phối (Non-IID):** Trong học liên kết phi tập trung, dữ liệu giữa các Client thường không đồng nhất (Non-IID). Việc SGD thực hiện cập nhật theo từng bước, sử dụng dữ liệu địa phương (local data) của từng Client, giúp điều chỉnh tốt hơn và cải thiện khả năng tổng quát của mô hình.

Tóm lại, các đặc tính của SGD rất phù hợp với học liên kết phân tán, giúp cập nhật các tham số mô hình theo cách hiệu quả, bảo mật và khả năng mở rộng.

2.4 Tổng hợp mô hình trong học liên kết phi tập trung

Trong bối cảnh Học Liên Kết Phi Tập Trung (Decentralized Federated Learning - DFL) ngày càng phát triển, tổng hợp mô hình đóng vai trò quan trọng trong việc kết hợp các mô hình được huấn luyện riêng biệt trên nhiều máy khách (clients) thành một mô hình chung cho máy khách hiện tại, mà không cần sự hỗ trợ từ một máy chủ trung tâm. Điều này không chỉ bảo vệ quyền riêng tư của dữ liệu mà còn tăng cường tính linh hoạt và độ tin cậy của hệ thống học máy phân tán.

2.4.1 Khái Niệm Về Tổng Hợp Mô Hình (Model Aggregation)

Tổng hợp mô hình là quá trình kết hợp các cập nhật mô hình từ nhiều máy khách để tạo ra một mô hình tổng thể. Trong DFL, không có máy chủ trung tâm để thực hiện việc này, do đó quá trình tổng hợp diễn ra thông qua sự tương tác giữa các máy khách.

2.4. Tổng hợp mô hình trong học liên kết phi tập trung

2.4.2 Luật tổng hợp

2.4.2.1 Federated Averaging

Cách hoạt động của Federated Averaging

Bước 1: Thu Thập Mô Hình Cục Bộ từ Các Khách Hàng Gần

Mỗi khách hàng thu thập các mô hình cục bộ từ các khách hàng lân cận trong mạng lưới phi tập trung. Các mô hình được thu thập là:

$$\{V_k \mid k \in \text{lân cận}\}.$$

Bước 2: Tính Trung Bình Có Trọng Số của Các Mô Hình Thu Thập Được

Sau khi thu thập, khách hàng tính trung bình có trọng số của tất cả các mô hình đã thu thập. Trọng số được tính dựa trên kích thước dữ liệu cục bộ của mỗi khách hàng k , đảm bảo rằng các mô hình từ các khách hàng có dữ liệu lớn hơn được ưu tiên hơn.

Trọng số của mô hình thu nhập từ khách hàng k là:

$$w_k = \frac{n_k}{n},$$

với:

n_k là số lượng dữ liệu của khách hàng k ,

$$n = \sum_{k \in \text{lân cận}} n_k \text{ là tổng số lượng dữ liệu của các khách hàng lân cận.}$$

Mô hình trung bình có trọng số được tính như sau:

$$V_{\text{avg}} = \sum_{k \in \text{lân cận}} w_k V_k = \sum_{k \in \text{lân cận}} \frac{n_k}{n} V_k.$$

Bước 3: Cập Nhật Mô Hình Toàn Cục

Mô hình trung bình V_{avg} được cập nhật sẽ được sử dụng làm mô hình mới cho các lần huấn luyện tiếp theo:

$$V_{\text{global}} \leftarrow V_{\text{avg}}.$$

Điều này đảm bảo rằng mô hình toàn cục được cải thiện liên tục thông qua sự hợp tác giữa các khách hàng.

2.4.2.2 Krum

Cách Hoạt Động của Krum

Bước 1: Tính Khoảng Cách Giữa Các Mô Hình

2.4. Tổng hợp mô hình trong học liên kết phi tập trung

Giả sử có n khách hàng, mỗi khách hàng gửi một mô hình d -chiều:

$$V_1, V_2, \dots, V_n.$$

Đối với mỗi mô hình V_i , tính khoảng cách Euclidean đến tất cả các mô hình khác V_j ($j \neq i$):

$$d(V_i, V_j) = \sqrt{\sum_{k=1}^d (V_i[k] - V_j[k])^2}.$$

Bước 2: Xác Định Các Mô Hình Gần Nhất

Với mỗi V_i , chọn $(n - f - 2)$ mô hình gần nhất, trong đó f là số lượng khách hàng độc hại tối đa mà hệ thống có thể chịu đựng.

Tính tổng các khoảng cách từ V_i đến các mô hình gần nhất này. Tổng này được gọi là điểm số (score) của V_i :

$$\text{score}(V_i) = \sum_{j \in S_i} d(V_i, V_j),$$

với S_i là tập hợp $(n - f - 2)$ mô hình gần nhất của V_i .

Bước 3: Chọn Mô Hình Tốt Nhất

Chọn mô hình V_i^* có điểm số thấp nhất trong số tất cả các mô hình:

$$V_i^* = \arg \min_i \text{score}(V_i).$$

Nếu có nhiều mô hình có điểm số thấp nhất giống nhau, chọn mô hình có định danh (identifier) nhỏ nhất.

2.4.2.3 Trimmed Mean

Mô tả Quy tắc: Khi khách hàng $i \in V$ nhận được $|N_i|$ mô hình từ các khách hàng lân cận, nó thực hiện các bước sau cho mỗi chiều (dimension) $k \in \{1, 2, \dots, d\}$:

1. **Thu thập và sắp xếp giá trị:** Thu thập các giá trị $w_j^{(k)}$ từ tất cả các mô hình lân cận và sắp xếp chúng theo thứ tự tăng dần.
2. **Cắt bỏ các giá trị cực đoan:** Loại bỏ phần trăm nhất định các giá trị lớn nhất và nhỏ nhất. Cụ thể, giả sử có c_i là tỉ lệ máy khách có thể độc hại, loại bỏ $\lfloor c_i |N_i| \rfloor$ giá trị lớn nhất và nhỏ nhất.
3. **Tính trung bình:** Tính trung bình của các giá trị còn lại sau khi đã cắt bỏ.

2.4. Tổng hợp mô hình trong học liên kết phi tập trung

2.4.2.4 Median

Mô tả Quy tắc: Trong quy tắc Trung vị, mỗi khách hàng i tính toán trung vị từng chiều (dimension) từ tất cả các mô hình lân cận.

2.4.2.5 FLTrust

1. Tính Độ Tương Đồng Cosine Độ tương đồng cosine giữa hai mô hình $\mathbf{w}_{t+1/2}^i$ và $\mathbf{w}_{t+1/2}^j$ được tính như sau:

$$\text{Cosine Similarity}(\mathbf{w}_{t+1/2}^i, \mathbf{w}_{t+1/2}^j) = \frac{\mathbf{w}_{t+1/2}^i \cdot \mathbf{w}_{t+1/2}^j}{\|\mathbf{w}_{t+1/2}^i\| \cdot \|\mathbf{w}_{t+1/2}^j\|}$$

Trong đó:

- $\mathbf{w}_{t+1/2}^i \cdot \mathbf{w}_{t+1/2}^j = \sum_k w_{t+1/2}^i[k] \cdot w_{t+1/2}^j[k]$ là tích vô hướng.
- $\|\mathbf{w}_{t+1/2}^i\| = \sqrt{\sum_k (w_{t+1/2}^i[k])^2}$ là độ lớn của vector $\mathbf{w}_{t+1/2}^i$.
- $\|\mathbf{w}_{t+1/2}^j\| = \sqrt{\sum_k (w_{t+1/2}^j[k])^2}$ là độ lớn của vector $\mathbf{w}_{t+1/2}^j$.

2. Chuẩn Hóa Mô Hình Khi Độ Tương Đồng Cosine Dương Nếu:

$$\text{Cosine Similarity}(\mathbf{w}_{t+1/2}^i, \mathbf{w}_{t+1/2}^j) > 0$$

thì chuẩn hóa $\mathbf{w}_{t+1/2}^j$ thành:

$$\tilde{\mathbf{w}}_{t+1/2}^j = \frac{\mathbf{w}_{t+1/2}^j}{\|\mathbf{w}_{t+1/2}^j\|} \cdot \|\mathbf{w}_{t+1/2}^i\|$$

Trong đó:

- $\frac{\mathbf{w}_{t+1/2}^j}{\|\mathbf{w}_{t+1/2}^j\|}$ là vector đơn vị của $\mathbf{w}_{t+1/2}^j$.
- $\|\mathbf{w}_{t+1/2}^i\|$ là độ lớn của vector $\mathbf{w}_{t+1/2}^i$.

3. Tính Trung Bình Các Mô Hình Đã Chuẩn Hóa Sau khi chuẩn hóa, tính trung bình các mô hình $\tilde{\mathbf{w}}_{t+1/2}^j$ có độ tương đồng cosine dương:

$$\mathbf{w}_t^i = \frac{1}{N} \sum_{j=1}^N \tilde{\mathbf{w}}_{t+1/2}^j$$

Trong đó:

2.4. Tổng hợp mô hình trong học liên kết phi tập trung

- N là số lượng các client lá cụ thể j có Cosine Similarity > 0 .
- $\tilde{\mathbf{w}}_{t+1/2}^j$ là mô hình chuẩn hóa từ client j .

2.4.2.6 UBAR

Khởi tạo:

$$x_{0,i} \leftarrow x_0$$

Vòng lặp đào tạo (với $k = 0$ đến $K - 1$):

1. Tính khoảng cách Euclidean:

$$d_{i,j} = \|x_{k,i} - x_{k,j}\|, \quad \forall j \in N_i$$

2. Chọn tập hợp các ứng viên gần nhất:

$$N_{k,i}^s \leftarrow \arg \min_{N^* \subseteq N_i, |N^*| = \rho_i |N_i|} \sum_{j \in N^*} d_{i,j}$$

3. Cập nhật tập hợp ứng viên thỏa mãn:

$$N_{k,i}^r = \{j \in N_{k,i}^s \mid l_{k,i} - l_{k,j} \geq 0\}$$

Nếu $N_{k,i}^r = \emptyset$:

$$j^* \leftarrow \arg \min_{j \in N_{k,i}^s} l_{k,j}$$

$$N_{k,i}^r \leftarrow \{j^*\}$$

4. Tính trung bình từ các nút trong $N_{k,i}^r$:

$$R_{k,i} = \frac{1}{|N_{k,i}^r|} \sum_{j \in N_{k,i}^r} x_{k,j}$$

5. Cập nhật ước lượng cục bộ:

$$x_{k+1,i} \leftarrow \alpha x_{k,i} + (1 - \alpha) R_{k,i} - \lambda \nabla l_{k,i}$$

Kết quả sau K lần lặp:

$$x_{K,i}$$

2.4. Tổng hợp mô hình trong học liên kết phi tập trung

Giải thích công thức

- $x_{k,i}$: ước lượng mô hình nút i tại vòng thứ k .
- $d_{i,j}$: khoảng cách Euclidean giữa nút i và j .
- $N_{k,i}^s$: tập hợp các nút ứng viên gần nhất trong lần lặp k .
- $N_{k,i}^r$: tập hợp các nút láng giềng được chọn để tính trung bình.
- $R_{k,i}$: giá trị trung bình của các ước lượng từ tập hợp $N_{k,i}^r$.
- $\nabla l_{k,i}$: gradient cục bộ tính từ dữ liệu của nút i ở lần lặp k .
- α : hệ số cân bằng giữa giá trị hiện tại và trung bình.

Ưu Điểm của Thuật Toán UBAR

- **Hiệu Quả Tính Toán:**
 - Với độ phức tạp tính toán $O(|N_i| \cdot d)$ cho mỗi nút tại mỗi lần lặp, UBAR duy trì hiệu suất cao ngay cả khi số lượng nút láng giềng tăng lên.
- **Tính Mở Rộng:**
 - Thuật toán thích hợp với các hệ thống phân tán lớn nhờ vào việc giảm số lượng nút cần đánh giá trong mỗi giai đoạn.

2.4.2.7 Learn

Số lần tổng hợp tại vòng t

Mỗi khách hàng thực hiện tổng hợp mô hình $\lceil \log_2 t \rceil$ lần ở vòng t :

$$\text{Số lần tổng hợp} = \lceil \log_2 t \rceil$$

- $\lceil x \rceil$: Hàm làm tròn lên giá trị x .

Công thức cập nhật mô hình

Sau khi tổng hợp, mô hình mới θ_{t+1} được cập nhật bằng cách sử dụng Trimmean từ các mô hình lân cận:

$$\theta_{t+1} = \text{Trimmean}(\{\theta_i^{(j)} : j \in \mathcal{N}_i\})$$

- $\theta_i^{(j)}$: Mô hình tại vòng t của khách hàng j .
- $\mathcal{N}_i$: Tập các khách hàng lân cận của khách hàng i .

2.4. Tổng hợp mô hình trong học liên kết phi tập trung

Trao đổi mô hình địa phương

Sau khi thực hiện $\lceil \log_2 t \rceil$ lần tổng hợp:

- Mỗi khách hàng trao đổi một lần mô hình địa phương với các lân cận của mình.

2.4.2.8 Self-Centered Clipping

Input:

- $m_1, m_2, \dots, m_n$: Các mô hình hoặc gradient từ n nút.
- τ : Ngưỡng cắt (*clipping threshold*).
- v : Giá trị trung tâm ban đầu.
- L : Số lần lặp, mặc định $L = 1$.

Thiết Lập Mặc Định:

- $L = 1$: Thực hiện một vòng lặp duy nhất nếu không chỉ định khác.
- $v = \hat{m}$: Giá trị trung tâm khởi tạo từ kết quả tổng hợp trước.

Thuật Toán: Vòng lặp chính: Với mỗi $l = 1, \dots, L$:

1. Tính toán c_i cho mỗi nút i :

$$c_i \leftarrow (m_i - v) \cdot \min \left(1, \frac{\tau}{\|m_i - v\|} \right)$$

- $m_i - v$: Khoảng cách giữa mô hình m_i và giá trị trung tâm v .
- $\|m_i - v\|$: Độ lớn (*norm*) của khoảng cách.
- $\min \left(1, \frac{\tau}{\|m_i - v\|} \right)$: Hệ số cắt giảm để đảm bảo $\|c_i\| \leq \tau$.

2. Cập nhật giá trị trung tâm v :

$$v \leftarrow v + \frac{1}{n} \sum_{i=1}^n c_i$$

- $\frac{1}{n} \sum_{i=1}^n c_i$: Giá trị trung bình của các c_i .
- Cập nhật v : Giá trị trung tâm mới cho vòng lặp tiếp theo.

Kết Quả (Output):

$$v$$

Giá trị trung tâm sau L vòng lặp. Đây là kết quả tổng hợp cuối cùng đã được cắt giảm.

Chương 3

Các bài toán trong Học liên kết phi tập trung

Học liên kết phi tập trung (Decentralized Federated Learning - DFL) mở ra nhiều cơ hội trong việc huấn luyện mô hình trên dữ liệu phân tán mà không cần một máy chủ trung tâm. Tuy nhiên, do đặc thù phi tập trung, DFL phải đối mặt với nhiều thách thức cả về hiệu năng, bảo mật và tính riêng tư.

Chương này sẽ trình bày các bài toán quan trọng trong DFL, bao gồm:

- Chi phí tính toán và truyền thông
- Mạng lưới và kết nối giữa các nút
- Hiệu năng và hội tụ
- Tính riêng tư dữ liệu
- Bảo vệ mô hình tổng hợp

Mỗi phần sẽ phân tích chi tiết các vấn đề liên quan, thách thức mà DFL phải đối mặt, và những giải pháp tiềm năng đã được nghiên cứu.

3.1 Chi phí tính toán và truyền thông

Trong DFL, các client thực hiện quá trình huấn luyện mô hình cục bộ và trao đổi mô hình với các client khác. Điều này dẫn đến hai loại chi phí chính:

- **Chi phí tính toán**

3.2. Mạng lưới và Kết nối

- **Tải trọng tính toán trên thiết bị biên:** DFL yêu cầu các thiết bị biên (ví dụ: điện thoại thông minh, cảm biến IoT) thực hiện các phép tính phức tạp, điều này có thể làm chậm quá trình huấn luyện hoặc giới hạn độ phức tạp của mô hình có thể được hỗ trợ do hạn chế về năng lực xử lý, bộ nhớ và tuổi thọ pin.
 - **Tính toán phân tán:** Việc phân phối tải trọng tính toán giữa nhiều thiết bị khác nhau có thể dẫn đến sự không đồng đều về hiệu suất và thời gian phản hồi.
- **Chi phí truyền thông**
 - **Truyền thông giữa các thiết bị:** DFL đòi hỏi sự giao tiếp liên tục giữa các thiết bị để cập nhật và tổng hợp mô hình, dẫn đến chi phí truyền thông cao, đặc biệt là khi số lượng thiết bị tham gia lớn.^[12]
 - **Kích thước dữ liệu:** Kích thước của các cập nhật mô hình cần được truyền có thể lớn, làm tăng chi phí truyền thông và ảnh hưởng đến hiệu suất của hệ thống.

Một số kỹ thuật tối ưu hóa đã được đề xuất để giảm chi phí này, chẳng hạn như:

- **Tối ưu tính toán**
 - **Tối ưu hóa thuật toán:** Phát triển các thuật toán học máy phân tán hiệu quả hơn, cho phép giảm tải trọng tính toán trên từng thiết bị.
 - **Sử dụng kiến trúc mô hình nhẹ:** Thiết kế các mô hình học máy nhẹ hơn để giảm yêu cầu về tài nguyên tính toán trên thiết bị biên.
- **Tối ưu truyền thông**
 - **Thuật toán ProFe:** Kết hợp kiến thức từ các mô hình lớn để huấn luyện mô hình nhỏ hơn cho việc tổng hợp, áp dụng lượng tử hóa để giảm dữ liệu truyền trong các vòng giao tiếp.^[13]
 - **Phương pháp chọn lọc tham số:** Sử dụng các phương pháp như top-k sparsification để chọn lọc và truyền chỉ những tham số quan trọng nhất, giúp giảm đáng kể lưu lượng truyền thông.^[14]

3.2 Mạng lưới và Kết nối

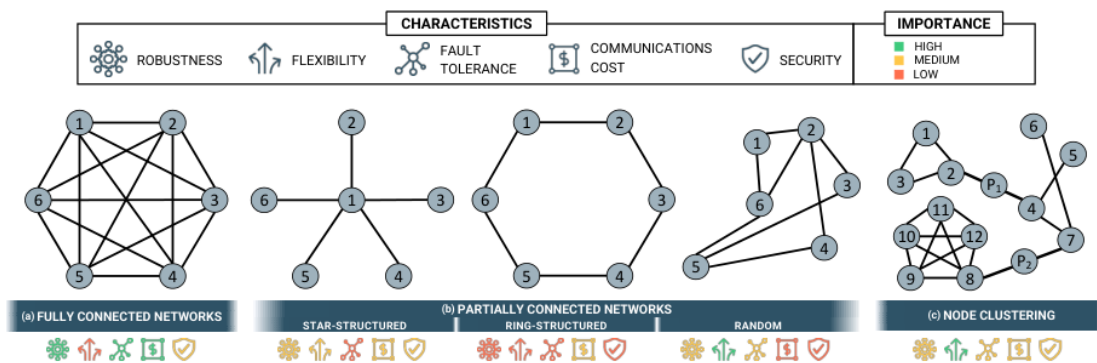
Mặc dù Học Liên kết Phi tập trung (Decentralized Federated Learning - DFL) giải quyết thành công các hạn chế của Học Liên kết Tập trung (Centralized Federated Learning - CFL) như điểm lỗi đơn lẻ và tắc nghẽn tại máy chủ trung tâm, bản chất phi tập trung của nó lại đặt ra

một loạt thách thức riêng biệt. Những thách thức này đặc biệt liên quan đến cấu trúc mạng lưới cơ bản và các phương thức giao tiếp được sử dụng giữa các nút tham gia. Chúng thường liên kết chặt chẽ với nhau và ảnh hưởng đáng kể đến hiệu suất, khả năng mở rộng, độ bền vững và bảo mật của hệ thống DFL.

3.2.1 Thách thức liên quan đến Cấu trúc liên kết Mạng (Network Topology)

Cách thức các nút tham gia được kết nối trong DFL ảnh hưởng cơ bản đến hoạt động và các giới hạn của nó:

- **Khả năng mở rộng và Điểm nghẽn (Bottlenecks):**
 - *Mạng kết nối đầy đủ:* Tuy mạnh mẽ nhưng khả năng mở rộng kém. Chi phí giao tiếp và độ phức tạp quản lý kết nối tăng theo cấp số nhân với số lượng nút, khiến chúng không thực tế cho các liên đoàn lớn (Trang 8)^[3].
 - *Mạng kết nối một phần (ví dụ: Hình sao, Hình vòng):* Cấu trúc hình sao tạo ra điểm nghẽn tiềm ẩn và điểm lỗi đơn lẻ nếu nút proxy trung tâm bị lỗi hoặc quá tải. Cấu trúc hình vòng có thể gặp phải độ trễ tăng khi kích thước mạng tăng lên (Trang 8)^[3]. Cấu trúc ngẫu nhiên đòi hỏi chi phí phụ để duy trì kết nối.
 - *Phân cụm nút:* Mặc dù có thể cải thiện hiệu quả trong các cụm, các nút proxy kết nối các cụm khác nhau có thể trở thành điểm nghẽn và việc quản lý cấu trúc phân cấp tổng thể làm tăng thêm độ phức tạp (Trang 8)^[3].



Hình 3.2.1: Các cấu trúc mạng trong hệ thống Học Liên kết Phi tập trung^[3]

- **Chi phí Giao tiếp Phụ (Overhead) và Chi phí Tổng thể:** Lựa chọn cấu trúc liên kết ảnh hưởng trực tiếp đến chi phí giao tiếp. Mạng kết nối đầy đủ vốn có chi phí cao do số lượng liên kết lớn. Mạng kết nối một phần có thể yêu cầu truyền qua nhiều bước nhảy

3.2. Mạng lưới và Kết nối

(multi-hop), có khả năng làm tăng độ trễ và chi phí phụ tổng thể so với liên kết trực tiếp, ngay cả khi số lượng liên kết trên mỗi nút thấp hơn (Trang 8, Trang 13)^[3]. Cấu trúc động phát sinh chi phí phụ cho việc thiết lập và duy trì liên kết.

- **Độ bền vững và Khả năng chịu lỗi:** Các cấu trúc liên kết khác nhau cung cấp các mức độ phục hồi khác nhau. Cấu trúc hình sao dễ bị tổn thương khi nút trung tâm bị lỗi. Vòng đơn hướng ít chịu lỗi hơn vòng hai chiều. Mặc dù mạng kết nối đầy đủ rất mạnh mẽ, nhưng điều này đi kèm với chi phí cao (Trang 8)^[3]. Cấu trúc ngẫu nhiên hoặc động cung cấp khả năng chịu lỗi vừa phải nhưng có thể không dự đoán được.
- **Tính linh hoạt và Tính động:** Cấu trúc liên kết cố định hạn chế khả năng thích ứng, đặc biệt trong các kịch bản có các nút tham gia di động hoặc kết nối không liên tục (phổ biến trong DFL trên nhiều thiết bị - cross-device). Cấu trúc động mang lại sự linh hoạt nhưng đòi hỏi các cơ chế mạnh mẽ để khám phá hàng xóm và quản lý kết nối, làm tăng chi phí phụ và độ phức tạp (Trang 6, Trang 8)^[3]. Việc xử lý hiệu quả sự thay đổi thành viên (các nút tham gia hoặc rời đi) là một thách thức đáng kể về mặt cấu trúc liên kết.
- **Lỗi hồng Bảo mật:** Cấu trúc liên kết ảnh hưởng đến tính dễ bị tổn thương của hệ thống. Trong mạng hình sao, việc xâm phạm nút trung tâm là rất nghiêm trọng. Trong mạng kết nối đầy đủ hoặc ngẫu nhiên, số lượng lớn các kết nối làm tăng bề mặt tấn công tiềm ẩn cho việc lan truyền phần mềm độc hại hoặc nghe lén, đặc biệt với các liên kết không liên tục hoặc yếu (Trang 9, Trang 10)^[3].

3.2.2 Thách thức liên quan đến Giao thức Giao tiếp (Communication Protocols)

Các quy tắc điều chỉnh cách thức và thời điểm các nút trao đổi thông tin là rất quan trọng và đặt ra một số khó khăn:

- **Đồng bộ hóa và Hội tụ:**
 - *Giao thức đồng bộ:* Bị chậm hội tụ vì toàn bộ hệ thống phải chờ đợi các nút tham gia chậm nhất ("stragglers") trong mỗi vòng, dẫn đến thời gian chết và lãng phí tài nguyên (Trang 9)^[3].
 - *Giao thức không đồng bộ:* Mặc dù có khả năng nhanh hơn do loại bỏ thời gian chờ, chúng lại giới thiệu "tính lỗi thời"(staleness) – các nút tổng hợp các tham số mô hình có thể đã lỗi thời, điều này có thể gây hại cho tốc độ hội tụ, khả năng tổng quát hóa của mô hình và độ chính xác tổng thể. Chúng cũng đòi hỏi các chính sách tổng

hợp phức tạp hơn vì số lượng cập nhật nhận được thay đổi theo từng vòng (Trang 9, Trang 12)^[3].

- *Giao thức bán đồng bộ*: Nhằm mục đích cân bằng nhưng vẫn yêu cầu điều chỉnh cẩn thận và các cơ chế kiểm soát (Trang 9)^[3].
- **Chi phí Giao tiếp Phụ (Overhead)**: DFL thường liên quan đến việc trao đổi thường xuyên các tham số mô hình hoặc gradient có thể có kích thước lớn. Điều này dẫn đến chi phí mạng đáng kể về mặt tiêu thụ băng thông, độ trễ và jitter (Trang 2, Trang 12)^[3]. Các giao thức không đồng bộ có thể làm trầm trọng thêm điều này do các cập nhật thường xuyên hơn, mặc dù nhỏ hơn (Trang 9)^[3]. Các kỹ thuật như lượng tử hóa (quantization) và thưa hóa (sparsification) nhằm giảm thiểu điều này nhưng có thể đánh đổi chi phí giao tiếp lấy độ chính xác của mô hình (Trang 15)^[3].
- **Khám phá và Quản lý Ngang hàng (P2P/Gossip)**: Khi không có điều phối viên trung tâm, các giao thức như P2P và Gossip phải đối mặt với thách thức trong việc khám phá hiệu quả và an toàn các hàng xóm phù hợp để giao tiếp. Lựa chọn ngẫu nhiên có thể không khả thi hoặc không tối ưu. Các cơ chế như đi bộ ngẫu nhiên (random walks) hoặc dựa trên heuristic về khoảng cách/đặc điểm là cần thiết nhưng làm tăng thêm độ phức tạp và chi phí phụ (Trang 9)^[3].
- **Xử lý Tính không đồng nhất (Heterogeneity)**: Các nút tham gia DFL thường có khả năng tính toán, băng thông mạng, khối lượng dữ liệu và tính khả dụng khác nhau (đặc biệt trong cài đặt cross-device). Các giao thức giao tiếp phải mạnh mẽ đối với tính không đồng nhất này, quản lý các nút chậm trong cài đặt đồng bộ hoặc tần số cập nhật và thời gian lỗi thời thay đổi trong cài đặt không đồng bộ (Trang 9, Trang 12)^[3].
- **Bảo mật và Quyền riêng tư**: Trao đổi giao tiếp là vector chính cho các cuộc tấn công. Các nút độc hại có thể gửi các bản cập nhật bị nhiễm độc (tấn công đối nghịch) hoặc phần mềm độc hại. Các giao thức P2P/Gossip có thể dễ bị tấn công như Sybil (đánh tính giả) hoặc Eclipse (thao túng kết nối của một nút) nếu các kênh khám phá hàng xóm và giao tiếp không được bảo mật (Trang 9, Trang 10)^[3]. Đảm bảo quyền riêng tư trong quá trình trao đổi tham số (ví dụ: thông qua mã hóa hoặc quyền riêng tư vi phân - differential privacy) làm tăng thêm chi phí tính toán và giao tiếp (Trang 10, Trang 11)^[3].

Việc giải quyết những thách thức này đòi hỏi sự đồng thiết kế cẩn thận giữa cấu trúc liên kết mạng và chiến lược giao tiếp, thường liên quan đến sự đánh đổi giữa hiệu quả, độ bền vững, khả năng mở rộng và bảo mật, được điều chỉnh cho phù hợp với kịch bản ứng dụng DFL cụ thể.

3.3 Hiệu năng và hội tụ

Do tính phi tập trung, DFL có thể gặp khó khăn trong việc đảm bảo mô hình học có thể hội tụ một cách hiệu quả. Một số thách thức bao gồm:

- **Hiệu suất tính toán:**

- **Tải trọng không đồng đều:** Các thiết bị trong mạng phi tập trung thường có khả năng tính toán khác nhau, dẫn đến sự chênh lệch về thời gian hoàn thành các vòng huấn luyện. Điều này làm giảm hiệu suất tổng thể của hệ thống và gây ra hiện tượng nghẽn cổ chai.
- **Chi phí tính toán cục bộ cao:** Việc thực hiện nhiều bước tối ưu hóa cục bộ trước khi gửi mô hình cập nhật có thể làm tăng chi phí tính toán, đặc biệt đối với các thiết bị có tài nguyên hạn chế.

- **Tốc độ hội tụ:**

- **Dữ liệu không đồng nhất (non-IID):** Sự phân phối dữ liệu không đồng nhất giữa các thiết bị gây khó khăn cho việc hội tụ của mô hình toàn cầu, làm giảm độ chính xác của mô hình.
- **Hội tụ chậm:** Trong môi trường phi tập trung, việc thiếu sự điều phối từ máy chủ trung tâm có thể làm tăng số vòng huấn luyện cần thiết để đạt được hội tụ.

- **Tối ưu hóa mô hình:**

- **Thuật toán tối ưu hóa chưa hiệu quả:** Các thuật toán như FedAvg hoặc FedSGD có thể không hoạt động tốt trong môi trường phi tập trung khi dữ liệu và tài nguyên không đồng đều^[15].
- **Thách thức về tham số hóa:** Việc lựa chọn các siêu tham số như tốc độ học, kích thước batch, và số lượng vòng lặp huấn luyện có ảnh hưởng lớn đến hiệu suất hội tụ của mô hình.

3.4 Vấn đề về tính riêng tư dữ liệu

3.4.1 Thách thức

Mặc dù FL và DFL nói chung giúp bảo vệ dữ liệu cá nhân bằng cách không chia sẻ dữ liệu thô, chúng vẫn đối mặt với các rủi ro riêng tư. Một số hình thức tấn công phổ biến bao gồm^[1]:

- **Tấn công đảo ngược mô hình (Model inversion):** Tấn công này khai thác mô hình học máy để tái tạo dữ liệu riêng tư từ đầu ra mà mô hình cung cấp. Kẻ tấn công sử dụng các thông tin như gradient hoặc các đầu ra trung gian từ mô hình để suy luận về các đặc điểm nhạy cảm của dữ liệu huấn luyện, thậm chí có thể tái tạo một phần dữ liệu ban đầu^[16].

Ví dụ: Trong môi trường DFL, mỗi nút tham gia gửi gradient cục bộ lên mạng để cập nhật mô hình toàn cầu. Kẻ tấn công có thể thu thập gradient từ các nút hoặc trong quá trình truyền tải để phân tích. Ví dụ, nếu một mô hình đang học nhận dạng hình ảnh khuôn mặt, gradient rò rỉ có thể được sử dụng để tái tạo hình ảnh khuôn mặt gần giống với dữ liệu thực tế của người dùng. Điều này đe dọa nghiêm trọng đến tính riêng tư của các cá nhân tham gia.

- **Tấn công suy luận thành viên (Membership inference):** Tấn công suy luận thành viên tập trung vào việc xác định xem một mẫu dữ liệu cụ thể có nằm trong tập huấn luyện hay không. Dựa vào các bản cập nhật mô hình hoặc gradient bị rò rỉ, kẻ tấn công có thể suy ra thông tin về các bản ghi cục bộ của người dùng, từ đó lộ ra thông tin nhạy cảm^[17].

Ví dụ: Trong một mạng DFL, các nút cục bộ gửi bản cập nhật mô hình sau mỗi vòng lặp huấn luyện. Kẻ tấn công có thể quan sát sự khác biệt giữa các bản cập nhật để xác định xem một dữ liệu cụ thể, chẳng hạn như thông tin sức khỏe (như bệnh án), có nằm trong tập huấn luyện của một nút hay không. Điều này có thể xảy ra ngay cả khi dữ liệu không được trực tiếp chia sẻ, dẫn đến vi phạm quyền riêng tư.

3.4.2 Giải pháp tiềm năng

Dưới đây là một số kỹ thuật bảo vệ dữ liệu phổ biến trong DFL^[1]:

- **Homomorphic Encryption (HE)**

Mã hóa đồng hình (HE) là một kỹ thuật mã hóa cho phép thực hiện các phép toán số học trên dữ liệu đã được mã hóa mà không cần giải mã. Điều này đảm bảo rằng các bên tham gia có thể xử lý dữ liệu mà không làm lộ thông tin gốc.

$$E(m_1) \cdot E(m_2) = E(m_1 \cdot m_2) \quad (3.4.1)$$

HE có ba biến thể chính, mỗi loại có độ phức tạp và khả năng áp dụng khác nhau:

- **FHE (Fully Homomorphic Encryption):** Hỗ trợ thực hiện bất kỳ phép toán số học nào (ví dụ: +, -, *, /) trên dữ liệu mã hóa.
- **PHE (Partially Homomorphic Encryption):** Chỉ hỗ trợ một phép toán đơn lẻ trên toàn bộ dữ liệu, ví dụ: phép nhân hoặc phép cộng (ví dụ: RSA chỉ hỗ trợ phép nhân).

3.4. Vấn đề về tính riêng tư dữ liệu

- **SHE (Somewhat Homomorphic Encryption):** Hỗ trợ một số phép toán nhất định trên dữ liệu mã hóa.

Ưu điểm:

- Bảo mật mạnh mẽ: Dữ liệu được bảo vệ ngay cả trong quá trình tính toán, không cần phải giải mã dữ liệu.
- Dễ dàng tích hợp: Có thể áp dụng trong nhiều trường hợp sử dụng khác nhau, đặc biệt là khi cần bảo mật dữ liệu trong quá trình xử lý.

Nhược điểm: Chi phí tính toán đối với các phép toán số học rất cao, dẫn đến yêu cầu tài nguyên đáng kể về thời gian xử lý và bộ nhớ.

• **Secure Multiparty Computation (SMC)**

Tính toán đa bên an toàn (SMC) là kỹ thuật mật mã phân tán cho phép nhiều bên tham gia vào việc tính toán chung mà không cần tiết lộ dữ liệu của họ cho các bên khác. Mỗi bên chỉ chia sẻ kết quả tính toán mà không chia sẻ dữ liệu gốc.

Ví dụ: Giả sử ba bên muốn tính mức lương trung bình của ba nhân viên, nhưng không muốn tiết lộ mức lương của từng người. Giao thức SMC chia nhỏ mức lương của mỗi người thành các phần ngẫu nhiên, sau đó phân phối chúng cho các bên. Khi các bên thực hiện phép toán, họ có thể tính được mức lương trung bình mà không tiết lộ thông tin cá nhân của từng người.

Ưu điểm:

- Bảo mật mạnh mẽ: Các bên không cần phải chia sẻ dữ liệu gốc của họ, chỉ chia sẻ kết quả tính toán.
- Đảm bảo tính riêng tư: Dữ liệu của mỗi bên không bao giờ được tiết lộ trong quá trình tính toán.

Nhược điểm:

- Chi phí giao tiếp và tính toán có thể rất cao, đặc biệt là trong các hệ thống DFL quy mô lớn.
- Việc triển khai các giải pháp SMC yêu cầu kiến thức kỹ thuật phức tạp để đảm bảo tính chính xác và bảo mật của các phép toán.

• **Differential Privacy (DP)**

3.5. Vấn đề về tấn công đối kháng

Differential Privacy là kỹ thuật bảo vệ quyền riêng tư bằng cách thêm nhiễu ngẫu nhiên vào các bản cập nhật dữ liệu, khiến cho việc phân tích dữ liệu trở nên không thể xác định được thông tin cá nhân của bất kỳ cá nhân nào trong tập huấn luyện.

DP thường được sử dụng để bảo vệ dữ liệu trong quá trình huấn luyện DFL, đặc biệt là chống lại các cuộc tấn công suy luận hoặc các cuộc tấn công tìm kiếm thông tin về thành viên của mô hình.

Ưu điểm:

- Chi phí tính toán thấp: DP có chi phí tính toán thấp hơn so với HE và SMC, không làm quá tải hệ thống.
- Cải thiện quyền riêng tư: DP đảm bảo rằng không có cá nhân nào có thể bị lộ thông tin qua việc phân tích kết quả huấn luyện.

Nhược điểm: Việc thêm nhiễu vào dữ liệu có thể làm giảm độ chính xác của mô hình, vì vậy cần phải cân nhắc giữa bảo mật và chất lượng mô hình.

Cơ chế bảo vệ	Chi phí tính toán	Chi phí giao tiếp	Kháng tấn công	Khả năng triển khai trong DFL
HE	Cao	Trung bình	Có	Khó thực hiện trong môi trường phi tập trung vì khả năng quản lý khóa để hiện thực cũng như chi phí tính toán cao
SMC	Cao	Cao	Có	Cần phải cân nhắc cho hệ thống DFL quy mô lớn do chi phí cao, phù hợp với các hệ thống nhỏ hơn
DP	Thấp	Thấp	Có	Dễ triển khai, thích hợp cho hệ thống DFL với chi phí thấp, nhưng cần kiểm soát tốt nhiễu để bảo vệ bảo mật

Bảng 3.4.1: Phân tích tính khả thi của các cơ chế bảo vệ về riêng tư dữ liệu trong DFL^[1]

3.5 Vấn đề về tấn công đối kháng

3.5.1 Thách thức

Một trong những thách thức quan trọng của DFL là đảm bảo tính chính xác và tin cậy của mô hình tổng hợp trong môi trường không có máy chủ trung tâm. Điều này khiến hệ thống dễ

3.5. Vấn đề về tấn công đối kháng

bị tổn thương trước các cuộc tấn công độc hại (tấn công đối kháng), trong đó gồm các loại phổ biến nhất là:

- **Tấn công Byzantine:** Tập trung vào việc làm suy giảm hiệu suất chung của hệ thống. Các client độc hại gửi mô hình ngẫu nhiên hoặc sai lệch nghiêm trọng để làm mất ổn định quá trình học. Mục tiêu là phá hoại mô hình toàn cục, không nhất thiết theo một hướng cụ thể.
- **Tấn công Poisoning:** Được thực hiện một cách có chủ đích hơn. Kẻ tấn công có thể thay đổi dữ liệu huấn luyện hoặc trọng số mô hình theo hướng có lợi cho một kết quả mong muốn, ví dụ như chèn nhãn sai vào mô hình (backdoor attack) hoặc làm mô hình phân biệt đối xử với một nhóm dữ liệu cụ thể.
- **Tấn công mạng:** Nhắm vào việc gây rối loạn hoặc kiểm soát quá trình giao tiếp giữa các nút trong mạng lưới DFL. Kẻ tấn công có thể làm gián đoạn, chặn, hoặc thao túng các luồng thông tin trao đổi giữa các nút, từ đó ảnh hưởng đến quá trình tổng hợp mô hình hoặc làm giảm hiệu suất của hệ thống. Các hình thức tấn công mạng phổ biến bao gồm tấn công từ chối dịch vụ (DoS), kiểm soát kết nối, hoặc chèn thông tin sai lệch vào quá trình truyền thông.

3.5.2 Giải pháp tiềm năng

- **Thuật toán tổng hợp chống Byzantine:** Các thuật toán như FedAvg, Krum, Trim-mean, Median, cũng như các phương pháp tiên tiến như UBAR^[18], LEARN^[19], SCCLIP^[20] được thiết kế để giảm thiểu tác động của các cập nhật mô hình độc hại. Bằng cách sử dụng các phép thống kê mạnh, các thuật toán này có thể phát hiện và loại bỏ các gradient bất thường do các client Byzantine hoặc tấn công poisoning gây ra, từ đó bảo vệ tính ổn định và chính xác của mô hình tổng hợp.
- **Mã hóa và bảo vệ giao tiếp:** Sử dụng các kỹ thuật mã hóa đầu cuối hoặc mã hóa đồng hình (Homomorphic Encryption) để đảm bảo tính toàn vẹn và bảo mật của dữ liệu, mô hình trong quá trình trao đổi giữa các nút. Điều này giúp ngăn chặn các tấn công mạng như nghe lén, chèn sửa hoặc giả mạo thông tin trong quá trình truyền thông.
- **Phát hiện bất thường và phòng thủ chủ động:** Áp dụng các mô hình học sâu hoặc các phương pháp thống kê để nhận diện các client có hành vi bất thường, phát hiện sớm các dấu hiệu của tấn công Byzantine, poisoning hoặc các hoạt động bất thường trên mạng. Việc này giúp chủ động loại bỏ hoặc cô lập các tác nhân độc hại khỏi quá trình huấn luyện.

Bên cạnh các phương pháp chung này, hai cơ chế bảo vệ nổi bật trong DFL là BALANCE và Moving Target Defense (MTD):

- **BALANCE** là một thuật toán tổng hợp mô hình chống Byzantine và poisoning, sử dụng mô hình cục bộ của từng client làm tham chiếu để xác định các cập nhật bị thao túng. Nhờ đó, chỉ các mô hình hợp lệ mới được tổng hợp, giúp duy trì độ chính xác và tin cậy của mô hình chung, đồng thời hạn chế tác động của các tấn công Byzantine và poisoning.
- **MTD (Moving Target Defense)** là chiến lược bảo vệ dựa trên việc kết hợp mã hóa mô hình với thay đổi linh hoạt cấu trúc kết nối mạng qua từng vòng huấn luyện. Cơ chế này giúp ngăn chặn các tấn công mạng như kiểm soát kết nối, nghe lén, hoặc phân tích luồng thông tin, từ đó tăng cường khả năng phòng thủ trước các mối đe dọa từ mạng lưới.

3.6 Kết luận

DFL mang lại nhiều lợi ích nhưng cũng đi kèm với nhiều thách thức lớn. Các bài toán quan trọng bao gồm chi phí tính toán và truyền thông, ổn định kết nối mạng, bảo vệ dữ liệu riêng tư, tính an toàn của mô hình tổng hợp và đảm bảo hiệu suất hội tụ. Những vấn đề này ảnh hưởng trực tiếp đến tính khả thi của DFL trong các ứng dụng thực tế.

Trong số các thách thức trên, bài nghiên cứu này tập trung chủ yếu giải quyết bài toán *tấn công đối kháng*, do đây là yếu tố quyết định tính chính xác và tin cậy của DFL. Ngoài ra, một phần nghiên cứu cũng hướng đến giải quyết bài toán *riêng tư dữ liệu*, đặc biệt trong bối cảnh các cuộc tấn công có thể khai thác thông tin từ quá trình tổng hợp mô hình.

Cụ thể, sự kết hợp giữa BALANCE và MTD hứa hẹn mang lại một giải pháp toàn diện để vừa đảm bảo tính chính xác của mô hình, vừa bảo vệ hệ thống trước các cuộc tấn công tinh vi. Tuy nhiên, việc triển khai hai cơ chế một cách hiệu quả vẫn là một bài toán cần được nghiên cứu thêm. Chương tiếp theo sẽ trình bày các đề xuất nghiên cứu nhằm tối ưu hóa các cơ chế bảo vệ này để nâng cao độ tin cậy và hiệu suất của DFL.

Chương 4

Các nghiên cứu liên quan

Các nghiên cứu trước đây đã tập trung vào việc cải thiện thuật toán tổng hợp mô hình như **LEARN**, **UBAR**, và **Self-Centered Clipping (SCCLIP)** nhằm tăng khả năng chống chịu trước các cuộc tấn công Byzantine. Tuy nhiên, các phương pháp này vẫn còn những hạn chế như chi phí truyền thông cao, thiếu đảm bảo lý thuyết, hoặc yêu cầu các giả định không thực tế. Gần đây, thuật toán **BALANCE** được đề xuất như một giải pháp tổng hợp mô hình mạnh mẽ, giúp tăng cường khả năng chống lại các cuộc tấn công *model poisoning* trong môi trường phi tập trung.

Model poisoning là một loại cuộc tấn công nguy hiểm trong hệ thống học liên kết phi tập trung, nơi mà kẻ tấn công cố gắng làm thay đổi hoặc phá hoại mô hình học thông qua việc gửi dữ liệu bị nhiễm độc từ các client độc hại. Điều này có thể dẫn đến việc mô hình tổng hợp trở nên kém chính xác hoặc bị thao túng theo cách mà kẻ tấn công có lợi. Trong bối cảnh này, thuật toán **BALANCE** đóng vai trò then chốt vì khả năng phát hiện và giảm thiểu ảnh hưởng của các client độc hại. Dựa trên nguyên tắc *local similarity*, **BALANCE** đảm bảo rằng các mô hình hợp lệ được ưu tiên trong quá trình tổng hợp, từ đó bảo vệ hệ thống khỏi sự xâm nhập của các cuộc tấn công model poisoning.

Song song đó, các kỹ thuật phòng thủ như **Moving Target Defense (MTD)** cũng đã chứng minh hiệu quả trong việc bảo vệ hệ thống trước các mối đe dọa mạng. MTD tạo ra một môi trường động, làm gián đoạn và làm phức tạp hóa các hoạt động của kẻ tấn công thông qua việc thay đổi liên tục cấu trúc giao tiếp, tham số hệ thống hoặc địa chỉ mạng. Kỹ thuật này không chỉ bảo vệ các giao tiếp giữa các *client* và *server*, mà còn làm giảm khả năng dự đoán của kẻ tấn công, khiến các phương pháp lập bản đồ mạng (*network mapping*) hoặc nghe lén (*eavesdropping*) trở nên khó khăn hơn. Đặc biệt, MTD còn phù hợp với các môi trường mạng kết nối ngẫu nhiên, nơi mà sự thay đổi linh hoạt trong cấu trúc mạng đóng vai trò quan trọng để giảm thiểu rủi ro.

Bên cạnh đó, mã hóa là một yếu tố quan trọng trong việc bảo vệ dữ liệu và đảm bảo tính riêng tư trong các hệ thống học liên kết phi tập trung. Việc mã hóa dữ liệu và thông tin liên lạc

4.1. Tổng quan về phương pháp BALANCE

giữa các client và server giúp ngăn chặn các cuộc tấn công như nghe lén và gián đoạn thông tin. Mã hóa có thể được thực hiện bằng cách sử dụng các protocol mã hóa khóa công khai hoặc khóa đối xứng, đảm bảo rằng dữ liệu chỉ có thể được giải mã bởi các bên được ủy quyền. Khi kết hợp với MTD, mã hóa cung cấp một lớp bảo mật bổ sung, bảo vệ dữ liệu ngay cả khi cấu trúc mạng thay đổi hoặc địa chỉ liên lạc bị thay đổi liên tục.

4.1 Tổng quan về phương pháp BALANCE

4.1.1 Hạn chế của các phương pháp tổng hợp trước đó

Các phương pháp tổng hợp trước đây trong DFL, như **UBAR**, **LEARN**, và **SCCLIP**, tồn tại một số hạn chế quan trọng:

- **Hiệu quả giao tiếp kém:**

Phương pháp **LEARN** yêu cầu trao đổi thông tin nhiều lần trong mỗi vòng huấn luyện, gây ra chi phí giao tiếp lớn và làm giảm hiệu suất.

- **Thiếu đảm bảo lý thuyết:**

Một số phương pháp như **UBAR** không thể đảm bảo lý thuyết rằng các client hợp lệ sẽ học được mô hình chính xác.

- **Phụ thuộc vào tỉ lệ client độc hại:**

Các phương pháp như **LEARN** và **SCCLIP** yêu cầu mỗi client phải biết được tỉ lệ client độc hại trong mạng lưới của mình, điều này khó thực hiện trong thực tế.

- **Yêu cầu đồ thị giao tiếp đầy đủ:**

LEARN giả định rằng đồ thị giao tiếp là đồ thị đầy đủ (fully connected), một giả định không khả thi trong môi trường thực tế với các kết nối ngẫu nhiên.

4.1.2 Giải thuật BALANCE

Giải thuật **BALANCE**^[4] (Byzantine-robust Averaging through Local Similarity in Decentralization) được đề xuất để khắc phục các hạn chế trên. Giải thuật này hoạt động dựa trên nguyên tắc sau:

- **So sánh độ tương đồng cục bộ:**

Mỗi client sử dụng mô hình trung gian của mình làm tham chiếu để đánh giá độ tin cậy của mô hình nhận được từ các client láng giềng.

- **Loại bỏ mô hình độc hại:**

Nếu mô hình nhận được khác biệt đáng kể về hướng hoặc độ lớn so với mô hình của chính nó, client sẽ coi mô hình đó là độc hại và loại bỏ.

4.1. Tổng quan về phương pháp BALANCE

Công thức để xác định mô hình đáng tin cậy như sau:

$$\|\mathbf{w}_i^{t+\frac{1}{2}} - \mathbf{w}_j^{t+\frac{1}{2}}\| \leq \gamma \cdot \exp(-\kappa \cdot \lambda(t)) \|\mathbf{w}_i^{t+\frac{1}{2}}\| \quad (4.1.1)$$

Trong đó:

- γ : Giới hạn trên để chấp nhận mô hình.
- κ : Hệ số suy giảm của hàm số mũ.
- $\lambda(t)$: Hàm tăng đơn điệu phụ thuộc vào vòng huấn luyện t .

Giá trị của $\gamma \cdot \exp(-\kappa \cdot \lambda(t))$ sẽ giảm khi số vòng huấn luyện t tăng, tức là điều kiện để chấp nhận mô hình ngày càng nghiêm ngặt hơn. Một cách đơn giản để định nghĩa $\lambda(t)$ là tỷ lệ $\frac{t}{T}$, với T là tổng số vòng huấn luyện.

Sau khi lọc các mô hình, client i sẽ xác định được một tập S_i^t chỉ các client láng giềng thỏa điều kiện ở công thức trên (tức là các client tốt) và mô hình trung bình từ tập này. Từ đó, client i thực hiện tổng hợp mô hình cục bộ của mình và mô hình trung bình vừa tính để cho ra được mô hình hoàn thiện theo công thức:

$$\mathbf{w}_i^{t+1} = \alpha \mathbf{w}_i^{t+\frac{1}{2}} + (1 - \alpha) \frac{1}{|S_i^t|} \sum_{j \in S_i^t} \mathbf{w}_j^{t+\frac{1}{2}}$$

Giá trị α điều chỉnh mức độ kết hợp giữa mô hình cục bộ và các mô hình hàng xóm được coi là tương đồng:

- Nếu α gần 1, mô hình cục bộ sẽ chiếm ưu thế.
- Nếu α gần 0, các mô hình hàng xóm sẽ có ảnh hưởng lớn hơn.

4.1. Tổng quan về phương pháp BALANCE

Algorithm 3 BALANCE

[4] **Input:** Tập các client $\mathcal{V}$; dữ liệu huấn luyện cục bộ $D_i, i \in \mathcal{V}$; số vòng huấn luyện T ; tốc độ học η ; đồ thị giao tiếp $\mathcal{G}$; các tham số α, γ, κ và $\lambda(t)$.

Output: Mô hình cục bộ $\mathbf{w}_i^T, i \in \mathcal{V}$.

```
1: Khởi tạo  $\mathbf{w}_i^0, i \in \mathcal{V}$ .
2: for  $t = 0, 1, \dots, T - 1$  do
3:   for mỗi client  $i \in \mathcal{V}$  (song song) do
4:     // Bước I: Huấn luyện mô hình cục bộ và trao đổi.
5:      $\mathbf{w}_i^{t+\frac{1}{2}} = \text{LocalTraining}(\mathbf{w}_i^t, D_i, \eta)$ .
6:     Gửi  $\mathbf{w}_i^{t+\frac{1}{2}}$  tới tất cả các client láng giềng  $j \in \mathcal{N}_i$ .
7:     Nhận  $\mathbf{w}_j^{t+\frac{1}{2}}$  từ tất cả các client láng giềng  $j \in \mathcal{N}_i$ .
8:     // Bước II: Tổng hợp mô hình cục bộ.
9:      $S_i^t = \emptyset$ .
10:    for mỗi client  $j \in \mathcal{N}_i$  do
11:      if Phương trình (4.1.1) thỏa mãn then
12:         $S_i^t = S_i^t \cup \{j\}$ .
13:      end if
14:    end for
15:     $\mathbf{w}_i^{t+1} = \alpha \mathbf{w}_i^{t+\frac{1}{2}} + (1 - \alpha) \frac{1}{|S_i^t|} \sum_{j \in S_i^t} \mathbf{w}_j^{t+\frac{1}{2}}$ .
16:  end for
17: end for
```

4.1.3 Thử nghiệm và đánh giá hiệu năng

4.1.3.1 Dataset

Thí nghiệm được thực hiện trên nhiều tập dataset khác nhau nhằm đánh giá hiệu quả và khả năng chống chịu của mô hình BALANCE. Cụ thể:

- **Tập dataset tổng hợp:** Được sử dụng để kiểm tra các đặc tính lý thuyết và khả năng khái quát hóa của mô hình trong các trường hợp giả lập.
- **Các dataset thực tế:** Bao gồm bốn bộ dữ liệu phổ biến trong nghiên cứu Học Liên Kết Phân Tán (FL):
 - **MNIST:** Dataset hình ảnh gồm các chữ số viết tay từ 0 đến 9.
 - **Fashion-MNIST:** Dataset gồm 10 lớp dữ liệu về các sản phẩm thời trang (áo thun, giày dép, túi xách, v.v.).
 - **Human Activity Recognition (HAR):** Dữ liệu về nhận diện hoạt động của con người, thu thập từ cảm biến điện thoại thông minh với sự tham gia của 30 người

4.1. Tổng quan về phương pháp BALANCE

dùng (mỗi người đại diện cho một client). Đây là một ví dụ điển hình của bộ dữ liệu FL trong thế giới thực.

- **CelebA**: Dataset gồm hơn 200.000 hình ảnh khuôn mặt với nhiều nhãn đặc trưng (giới tính, kính mắt, râu, v.v.), thường được sử dụng để thử nghiệm các bài toán về nhận diện khuôn mặt.

4.1.3.2 Mô hình tấn công

Trong thí nghiệm, 7 mô hình poisoning attack được sử dụng để kiểm tra độ an toàn và khả năng kháng tấn công của BALANCE, bao gồm:

- **Data poisoning:**

1. Label Flipping (LF) attack: Thay đổi nhãn của dữ liệu huấn luyện nhằm gây nhiễu cho mô hình.
2. Feature attack: Tấn công dựa trên việc làm nhiễu các đặc trưng (feature) trong dữ liệu.

- **Model poisoning:** Gaussian (Gauss) attack, Krum attack, Trim attack, Backdoor attack, and Adaptive (Adapt) attack

4.1.3.3 Thang điểm đánh giá

Các thang điểm đánh giá được sử dụng để phân tích hiệu quả và độ bền của giải thuật BALANCE khi đối mặt với các mô hình tấn công trong học liên kết phân tán (DFL). Ba chỉ số cụ thể được sử dụng là:

- **max.MSE (Maximum Mean Squared Error)**

Ý nghĩa: Đo lường độ sai lệch giữa mô hình cục bộ và mô hình toàn cục tối ưu.

Cách tính: Được tính bằng cách lấy trung bình bình phương sai lệch lớn nhất giữa các mô hình tham gia và mô hình tham chiếu.

- **max.TER (Maximum Test Error Rate)**

Ý nghĩa: Phản ánh tỷ lệ lỗi cao nhất trên tập dữ liệu kiểm thử sau khi huấn luyện.

Cách tính: Được tính dựa trên độ chính xác của mô hình trên dữ liệu kiểm thử. Một tỷ lệ lỗi cao cho thấy mô hình bị ảnh hưởng nhiều từ các tấn công độc hại.

- **max.ASR (Maximum Attack Success Rate)**

Ý nghĩa: Đo lường mức độ thành công của các cuộc tấn công vào quá trình học liên kết phân tán.

4.1. Tổng quan về phương pháp BALANCE

Cách tính: Tỷ lệ mà các mô hình tấn công đạt được mục tiêu gây tổn hại hoặc kiểm soát quá trình huấn luyện.

Với cả 3 thang đánh giá trên, giá trị càng nhỏ thể hiện khả năng chống lại tấn công càng tốt

4.1.3.4 Kết quả đánh giá

Các kết quả đánh giá cho thấy hiệu quả của BALANCE trong việc chống lại các cuộc tấn công khác nhau trong quá trình học liên kết phân tán. Các kết quả chính bao gồm:

- **Khả Năng Chống Lại Model Poisoning Attacks** : *Gaussian Attack, Krum Attack, Trim Attack, Backdoor Attack, và Adaptive Attack*: BALANCE cho thấy hiệu quả cao trong việc giảm **max.ASR**. Đặc biệt, đối với các tấn công thích nghi (*Adaptive Attack*), BALANCE vẫn giữ được khả năng bảo vệ tốt hơn so với các phương pháp tổng hợp mô hình khác.
- **So Sánh Với Các Phương Pháp Khác**: BALANCE được so sánh với các phương pháp tổng hợp mô hình phổ biến như **FedAvg** và **Krum**. Kết quả cho thấy BALANCE không chỉ cải thiện độ chính xác tổng thể mà còn giảm thiểu ảnh hưởng của các cuộc tấn công độc hại hơn so với các phương pháp này.
- **Kết Quả Từ Các Bảng Dữ Liệu**: Hình 6.1.1 minh họa kết quả **max.MSEs** đối với tập dữ liệu tổng hợp. Hình 4.1.2 minh họa kết quả **max.TER** đối với tập dữ liệu MNIST (kết quả đối với kịch bản tấn công Backdoor được biểu diễn dưới dạng "**max.TER / max.ASR**").

Method	No	LF	Feature	Gauss	Krum	Trim	Adapt
FedAvg	0.36	0.39	>100	>100	72.18	58.50	90.29
Krum	1.11	1.23	>100	1.19	1.18	1.18	1.19
Trim-mean	0.38	0.39	3.45	0.40	4.17	5.41	5.41
Median	0.39	0.40	2.37	0.42	1.22	3.93	3.93
FLTrust	0.41	0.46	>100	22.87	10.12	8.00	0.42
UBAR	0.40	0.40	>100	0.40	0.40	0.40	0.40
LEARN	0.42	0.42	0.42	0.64	5.36	17.78	1.60
SCCLIP	0.36	0.39	>100	0.42	5.63	5.12	4.83
BALANCE	0.36	0.36	0.36	0.36	0.36	0.36	0.36

Hình 4.1.1: Kết quả của các phương pháp tổng hợp khác nhau cho tập dữ liệu tổng hợp^[4]

4.2. Kỹ thuật mã hóa và thay đổi mục tiêu phòng thủ (MTD)

(a) MNIST dataset.

Method	No	LF	Feature	Gauss	Krum	Trim	Backdoor	Adapt
FedAvg	0.10	0.10	0.90	0.90	0.91	0.91	0.90 / 1.00	0.90
Krum	0.10	0.12	0.90	0.10	0.10	0.15	0.15 / 0.01	0.14
Trim-mean	0.11	0.12	0.49	0.11	0.82	0.81	0.83 / 0.72	0.87
Median	0.14	0.14	0.45	0.15	0.52	0.63	0.66 / 0.01	0.66
FLTrust	0.10	0.11	0.90	0.13	0.10	0.88	0.10 / 0.73	0.10
UBAR	0.14	0.14	0.90	0.14	0.14	0.14	0.15 / 0.01	0.14
LEARN	0.10	0.10	0.30	0.12	0.18	0.57	0.12 / 0.03	0.44
SCCLIP	0.10	0.10	0.10	0.11	0.91	0.91	0.10 / 0.01	0.91
BALANCE	0.10	0.10	0.11	0.10	0.10	0.11	0.11 / 0.01	0.11

Hình 4.1.2: Kết quả của các phương pháp tổng hợp khác nhau cho tập dữ liệu MNIST^[4]

Ngoài ra, BALANCE đạt được khả năng hội tụ lý thuyết trong cả trường hợp hàm mất mát lồi và không lồi. Tốc độ hội tụ tương đương với các phương pháp không có Byzantine trong cùng điều kiện.

4.1.4 Mở rộng về xử lý dữ liệu IID và non-IID

BALANCE hoạt động hiệu quả với cả dữ liệu **IID** (độc lập và đồng phân phối) và **non-IID** (không đồng nhất):

- **Dữ liệu IID:** Các client có dữ liệu đồng đều, giúp mô hình hội tụ nhanh hơn.
- **Dữ liệu non-IID:** Thử nghiệm cho thấy BALANCE vẫn bảo đảm mô hình hội tụ ngay cả khi mỗi client chỉ sở hữu một số lớp nhãn giới hạn.

Kết quả này khẳng định tính linh hoạt và khả năng áp dụng thực tế của BALANCE trong nhiều tình huống khác nhau.

4.2 Kỹ thuật mã hóa và thay đổi mục tiêu phòng thủ (MTD)

Moving Target Defense (MTD)^[2] là một chiến lược an ninh mạng tiên tiến nhằm giảm thiểu nguy cơ từ các mối đe dọa trong mô hình học liên kết phi tập trung (DFL). Trong bối cảnh DFL, các mối nguy hiểm như nghe lén (eavesdropping), tấn công MitM (Man-in-the-Middle), lập bản đồ mạng (network mapping), và tấn công che lấp (eclipse) đặt ra thách thức lớn đối với tính bảo mật, toàn vẹn và sẵn sàng của hệ thống. MTD cung cấp một cách tiếp cận chủ động bằng cách liên tục thay đổi bề mặt tấn công, bao gồm địa chỉ mạng, cấu trúc liên kết giao tiếp, hoặc tham số mô hình, nhằm làm gián đoạn và làm phức tạp các hoạt động của kẻ tấn công. Giải pháp này

4.2. Kỹ thuật mã hóa và thay đổi mục tiêu phòng thủ (MTD)

không chỉ tăng cường khả năng bảo vệ mà còn giảm thiểu tác động của các cuộc tấn công, tạo nền tảng cho một hệ thống học liên bang phi tập trung an toàn và linh hoạt hơn.

4.2.1 Vấn đề rủi ro trong hệ thống phi tập trung

Bảng 4.2.1: Thông tin có thể truy cập bởi một client "xấu" trong DFL^[2]

Thông tin	Mô tả
Tham số mô hình (Model Parameters)	Mỗi layer (lớp) l_i trong mô hình M với n layers có trọng số $w_i \in \mathbb{R}^{d_i \times d_{i-1}}$ và độ lệch $b_i \in \mathbb{R}^{d_i}$, trong đó d_i là số lượng neuron trong tầng i . Các tham số của M là tập hợp $\{w_i, b_i\}_{i=1}^n$.
Cấu trúc mạng (Topology)	Đồ thị của mạng $G(V, E)$, trong đó V là tập hợp các client (thành viên tham gia) và E là tập hợp các cạnh (đường nối giữa các client). Nếu $V = \{v_1, v_2, \dots, v_n\}$ và $E = \{(v_i, v_j) v_i, v_j \in V, i \neq j\}$, thì cấu trúc mạng là hoàn toàn được kết nối với nhau.
Vai trò (Roles)	Mỗi thành viên $p_i \in V$ có một vai trò $r_i \in \{\text{idle, trainer, aggregator, proxy}\}$. Điều này có thể được biểu diễn bằng một hàm $R : V \rightarrow \{\text{idle, trainer, aggregator, proxy}\}$, trong đó $R(p_i) = r_i$.
Chỉ số đo lường (Metrics)	Hiệu suất của mô hình (ví dụ: độ chính xác, độ chính xác thu hồi, F1-score) và tài nguyên sử dụng (CPU, RAM, mạng) của các client. Với tài nguyên R , để U_R là mức sử dụng và C_R là công suất, tỷ lệ sử dụng được tính là $R_{rate} = \frac{U_R}{C_R}$.
Khoảng thời gian hoạt động (Activity Periods)	Nếu $T = \{t_1, t_2, \dots, t_n\}$ là tập hợp tất cả các khoảng thời gian và $A = \{a_1, a_2, \dots, a_k\} \subseteq T$ là các khoảng hoạt động, thì tỷ lệ hoạt động là $A_{ratio} = \frac{\sum_{i=1}^k a_i}{\sum_{i=1}^n t_i}$.
Kiến trúc mô hình (Model Architecture)	Mô hình mạng neural truyền thẳng với n layers có thể được biểu diễn dưới dạng các hàm hợp $f(x) = f_n(f_{n-1}(\dots f_2(f_1(x))\dots))$, trong đó $f_i(x) = \sigma(w_i \cdot x + b_i)$ là phép toán cho lớp i , và σ là hàm kích hoạt (activation function).
Mẫu giao tiếp (Communication Patterns)	Nếu $M = \{m_{ij}\}$ là tập hợp tất cả các tin nhắn được gửi từ thành viên i đến thành viên j , thì tần suất giao tiếp giữa các thành viên có thể được định lượng bởi $F_{ij} = \frac{ m_{ij} }{\sum_{i,j} m_{ij} }$, trong đó $ m_{ij} $ là số lượng tin nhắn trao đổi.

4.2. Kỹ thuật mã hóa và thay đổi mục tiêu phòng thủ (MTD)

Bảng 4.2.2: Các cuộc tấn công, mục tiêu và thông tin có thể bị khai thác trong DFL^[2]

Cuộc tấn công	Mục tiêu	Thông tin có thể bị khai thác
Eavesdropping	Trích xuất thông tin nhạy cảm để làm suy yếu tính toàn vẹn và bảo mật của các thành viên trong mạng [!!].	Tham số mô hình, Cấu trúc mạng, Vai trò
MitM	Thao túng thông tin hoặc chen dữ liệu độc hại để phá hoại các hoạt động học liên kết [!!!].	Mẫu giao tiếp, Vai trò
Network Mapping	Biết cấu trúc mạng để thực hiện các cuộc tấn công nhắm mục tiêu vào mạng liên kết trong tương lai [!].	Cấu trúc mạng, Kiến trúc mô hình
Eclipse Attacks	Cô lập một client hoặc nhóm các client để trích xuất thông tin hoặc làm gián đoạn giao tiếp DFL [!!!].	Khoảng thời gian hoạt động, Cấu trúc mạng, Vai trò, Mẫu giao tiếp

Đánh giá mức độ: ! Ít quan trọng, !! Rất quan trọng, !!! Nguy hiểm.

Các cuộc tấn công ở bảng 4.2.2 đã được xác định^[2]:

- **Eavesdropping (Nghe lén):** Kẻ tấn công có thể bí mật giám sát các liên lạc mạng hoặc xâm nhập vào một client tham gia để truy cập trái phép vào dữ liệu nhạy cảm. Dữ liệu này có thể bao gồm các tham số mô hình, cấu trúc mạng và vai trò của các thành viên tham gia. Kẻ tấn công có thể sau đó lợi dụng thông tin này để làm gián đoạn quá trình liên bang hoặc giả danh một thành viên hợp pháp. Mỗi đe dọa này thường tồn tại mà không bị phát hiện do tính chất ngầm của nó, dẫn đến các khoảng thời gian rò rỉ dữ liệu nhạy cảm kéo dài. (VD: nghe lén thông tin được chia sẻ nội bộ giữa các client, sau đó lấy các thông tin đó phục vụ cho mục đích khác).
- **MitM (Man in the Middle):** Kẻ tấn công chặn và có thể thao túng giao tiếp giữa hai client tham gia. Điều này cho phép kẻ tấn công thay đổi các tham số mô hình trao đổi, đưa vào dữ liệu giả mạo, hoặc nghe lén thông tin trao đổi, gây ra những thách thức lớn đối với tính toàn vẹn của quá trình liên bang.
- **Network Mapping(Bản đồ mạng):** Kẻ tấn công hiểu cấu trúc của mạng liên bang và vai trò của các client tham gia. Bằng cách có được kiến thức này, kẻ tấn công có thể dự đoán và can thiệp vào các hoạt động của mạng, tạo điều kiện cho các cuộc tấn công có mục tiêu và tiềm ẩn nhiều nguy cơ gây hại hơn. (VD: Biết được ai điều phối nhiều dữ liệu hơn,...).

4.2. Kỹ thuật mã hóa và thay đổi mục tiêu phòng thủ (MTD)

- **Eclipse (che lấp):** Cuộc tấn công này trong DFL nhằm cô lập một client cụ thể hoặc một nhóm các client khỏi phần còn lại của mạng. Sự cô lập này làm sai lệch nhận thức của các client bị ảnh hưởng về trạng thái của mạng, khiến chúng hoạt động dựa trên thông tin không chính xác và có thể mở đường cho các vi phạm an ninh bổ sung. (VD: tạo ra các client giả vây quanh nó, khiến nó chỉ kết nối với các client giả. Ví dụ, trong một mạng đào tạo liên bang phân tán cho xe tự hành, một xe có thể bị cô lập với dữ liệu cập nhật từ các xe khác. Điều này khiến nó đưa ra các quyết định lái xe dựa trên thông tin lỗi thời hoặc sai lệch, dẫn đến những rủi ro an toàn đáng kể).

4.2.2 Kỹ thuật mã hóa và MTD trong DFL

Trong bối cảnh bảo vệ hệ thống phân tán, việc áp dụng các kỹ thuật bảo mật để tăng cường tính an toàn là vô cùng cần thiết. Một trong những cách tiếp cận nổi bật là sự kết hợp giữa các kỹ thuật mã hóa mạnh mẽ và các chiến lược MTD (Moving Target Defense), giúp xây dựng một hệ sinh thái an toàn và có khả năng phòng chống các cuộc tấn công hiệu quả.

4.2.2.1 Mã hóa và các kỹ thuật bảo vệ

Việc bảo vệ hệ thống có thể được thực hiện thông qua việc kết hợp các kỹ thuật mã hóa khác nhau, nhằm đảm bảo rằng dữ liệu không bị lộ hoặc xâm phạm khi truyền tải qua mạng. Hai loại mã hóa chính có thể được áp dụng trong hệ thống DFL là mã hóa đối xứng và mã hóa bất đối xứng:

- **Mã hóa đối xứng:** Đây là phương pháp mã hóa sử dụng một khóa chung để mã hóa và giải mã dữ liệu. Mã hóa đối xứng giúp bảo vệ dữ liệu được chia sẻ giữa các client trong mạng, đảm bảo rằng dữ liệu này không bị lộ ra ngoài trong quá trình truyền tải. Nhờ vào tính hiệu quả của thuật toán mã hóa đối xứng, việc xử lý dữ liệu được thực hiện nhanh chóng, tiết kiệm tài nguyên tính toán.
- **Mã hóa bất đối xứng:** Phương pháp này sử dụng cặp khóa (khóa công khai và khóa riêng) để mã hóa và giải mã dữ liệu. Mã hóa bất đối xứng rất quan trọng để bảo vệ quá trình trao đổi khóa đối xứng giữa các client, đảm bảo rằng chỉ những client được ủy quyền mới có thể truy cập vào khóa giải mã và sử dụng các mô hình được chia sẻ. Điều này giúp tăng cường bảo mật hệ thống và hạn chế nguy cơ bị tấn công thông qua việc làm rò rỉ khóa bí mật.

Việc kết hợp mã hóa đối xứng và mã hóa bất đối xứng mang lại một sự cân bằng lý tưởng giữa hiệu quả tính toán và bảo mật. Mã hóa đối xứng đóng vai trò trong việc xử lý nhanh chóng

4.2. Kỹ thuật mã hóa và thay đổi mục tiêu phòng thủ (MTD)

khối lượng dữ liệu lớn, trong khi mã hóa bất đối xứng bảo vệ an toàn việc trao đổi khóa và xác thực các client tham gia.

4.2.2.2 Kỹ thuật MTD

Các kỹ thuật MTD (Moving Target Defense) nhằm tạo ra một hệ thống với các đặc tính thay đổi liên tục, làm cho việc tấn công trở nên khó khăn hơn. Trong hệ thống DFL, hai kỹ thuật MTD đáng chú ý là *Neighbor Selection* và *IP/Port Switching*.

- **Neighbor Selection:** Kỹ thuật này liên quan đến việc chọn ngẫu nhiên hoặc thay đổi động các client hàng xóm để trao đổi thông tin mô hình trong mỗi vòng (round). Việc sử dụng chiến lược này ngăn chặn kẻ tấn công xác định được các client cụ thể nào đang tham gia trao đổi dữ liệu, từ đó làm giảm khả năng bị tấn công. Kỹ thuật này giúp bảo đảm rằng các client trong hệ thống luôn hoạt động trong môi trường mạng không ổn định, khó dự đoán.
- **IP/Port Switching:** Đây là kỹ thuật liên tục thay đổi địa chỉ IP và các cổng giao tiếp trong quá trình truyền tải dữ liệu giữa các client. Bằng cách này, kẻ tấn công gặp khó khăn trong việc theo dõi và xâm nhập vào kết nối, vì các đường truyền luôn thay đổi và không dễ để định vị hoặc tấn công. IP/Port Switching làm tăng tính bảo mật của hệ thống bằng cách giảm thiểu khả năng bị tấn công mạng dựa trên các điểm tĩnh, đảm bảo rằng các client hoạt động trong một môi trường ẩn danh và an toàn hơn.

Sự kết hợp của các kỹ thuật mã hóa và MTD cung cấp một lớp bảo vệ động và có khả năng thích ứng với các cuộc tấn công tiềm tàng. Điều này không chỉ giúp duy trì tính bảo mật và toàn vẹn của dữ liệu mà còn bảo vệ mạng phân tán trước những mối đe dọa ngày càng tinh vi.

4.2. Kỹ thuật mã hóa và thay đổi mục tiêu phòng thủ (MTD)

4.2.2.3 Thuật toán được đề xuất trong hệ thống DFL

Algorithm 4 Các thủ tục cho kỹ thuật IP/Port Switching và Neighbor Selection^[2]

```
1: procedure MTDN( $N_{\text{all}}, n$ ) ▷ Lựa chọn client lân cận
2:   Khởi tạo danh sách rỗng  $N$ 
3:   while  $|N| < n$  do
4:     Chọn một client lân cận  $i$  từ  $N_{\text{all}}$  một cách ngẫu nhiên với xác suất bằng nhau
5:     if  $i \notin N$  then
6:       Thêm  $i$  vào  $N$ 
7:     end if
8:   end while
9:   return  $N$ 
10: end procedure
11: procedure MTDIP(config) ▷ Chuyển đổi IP/Port
12:   Lấy danh sách IP và cổng sẵn có:  $IP_{\text{avail}}, P_{\text{avail}}$ 
13:   Chọn một IP và cổng mới từ  $IP_{\text{avail}}, P_{\text{avail}}$  một cách ngẫu nhiên với xác suất bằng nhau
14:   Cập nhật cấu hình với địa chỉ IP và cổng mới
15:   return config
16: end procedure
```

Trước hết ta định nghĩa 2 thủ tục là MTD_N và MTD_{IP} , để thực hiện các chức năng lựa chọn client lân cận và chuyển đổi địa chỉ IP/port nhằm đảm bảo tính bảo mật và bảo vệ quyền riêng tư trong quá trình học phân tán.

4.2. Kỹ thuật mã hóa và thay đổi mục tiêu phòng thủ (MTD)

Algorithm 5 Hiện thực các client tham gia với Kỹ thuật Mã hóa và MTD trong DFL [2]

Require: R : số vòng lặp cục bộ, η : tốc độ học, λ : tham số điều chỉnh mức độ tiêu chuẩn, S_j : socket đến client lân cận j , D : tập dữ liệu cục bộ, $E_{\text{sym}}, E_{\text{asym}}$: hàm mã hóa đối xứng/phi đối xứng, $D_{\text{sym}}, D_{\text{asym}}$: hàm giải mã đối xứng/phi đối xứng, MTD_{IP} : hàm MTD IP/port, MTD_N : hàm MTD lựa chọn client lân cận

```

1:  $D_{\text{train}}, D_{\text{test}} \leftarrow \text{split}(D)$ 
2: for  $r$  in  $R$  do
3:    $\theta \leftarrow \text{Initialize}()$  ▷ Khởi tạo mô hình cục bộ
4:   for mỗi  $(x, y)$  trong  $D_{\text{train}}$  do
5:      $\theta \leftarrow \theta - \eta \nabla J(\theta, (x, y)) + \lambda \theta$ 
6:   end for
7:    $N \leftarrow MTD_N(N_{\text{all}}, n)$ 
8:   for  $j$  trong  $N$  do
9:      $e_{\text{enc}} \leftarrow E_{\text{sym}}(\theta, K_{\text{sym}})$ 
10:     $K_{\text{sym, enc}} \leftarrow E_{\text{asym}}(K_{\text{sym}}, K_j^{\text{pub}})$ 
11:    Gửi  $e_{\text{enc}}, K_{\text{sym, enc}}$  đến  $j$  qua  $S_j$ 
12:   end for
13:   while not Timeout do
14:     for  $j$  trong  $N$  do
15:        $RP_{\text{enc}}, K_{j, \text{sym, enc}} \leftarrow \text{Nhận từ } j \text{ qua } S_j$  ▷ RP: tham số nhận được
16:        $K_{j, \text{sym}} \leftarrow D_{\text{asym}}(K_{j, \text{sym, enc}}, K_{\text{priv}})$ 
17:        $RP_j \leftarrow D_{\text{sym}}(RP_{\text{enc}}, K_{j, \text{sym}})$ 
18:     end for
19:   end while
20:    $\theta \leftarrow \frac{1}{|N|+1}(\theta + \sum_{j \in N} RP_j)$  ▷ Tổng hợp (FedAvg)
21:   Cập nhật mô hình cục bộ với  $\theta$ 
22: end for
23: for mỗi  $(x, y)$  trong  $D_{\text{test}}$  do
24:    $y_{\text{pred}} \leftarrow \text{Dự đoán}(\theta, x)$  ▷ Test
25: end for
26:  $T_{\text{test}} \leftarrow \frac{1}{|D_{\text{test}}|} \sum_{i=1}^{|D_{\text{test}}|} l(y_i, y_{\text{pred}})$  ▷ Tính toán độ mất mát
27: Gửi các chỉ số tới bộ điều khiển ▷ Báo cáo chỉ số theo dõi
28:  $MTD_{\text{IP}}(\text{config}) \rightarrow \text{config}$ 

```

Sau khi định nghĩa hai thủ tục trên, client tham gia hiện thực thuật toán theo các bước như sau:

1. **Khởi tạo và huấn luyện cục bộ:** Mô hình được khởi tạo và huấn luyện trên tập dữ liệu D_{train} trong số vòng lặp R . Trong mỗi vòng, mô hình sẽ được cập nhật dựa vào các mẫu huấn luyện và hàm lỗi.
2. **Chọn client lân cận:** Thủ tục MTD_N được gọi để chọn các client lân cận từ tập N_{all} , đảm bảo tính ngẫu nhiên và phân tán.

4.2. Kỹ thuật mã hóa và thay đổi mục tiêu phòng thủ (MTD)

3. **Mã hóa mô hình và gửi đi:** Mô hình được mã hóa bằng cách sử dụng mã hóa đối xứng và bất đối xứng trước khi gửi tới các client lân cận qua các kết nối mạng.
4. **Nhận và giải mã phản hồi:** Các client lân cận nhận được khóa trao đổi và tham số được mã hóa, tiến hành giải mã để lấy khóa trao đổi và hiện thực giải mã tham số đã được nhận để thực hiện tổng hợp mô hình.
5. **Tổng hợp và cập nhật mô hình:** Mô hình tổng hợp được tính bằng cách sử dụng phương pháp trung bình liên kết (FedAvg), rồi được cập nhật và lưu trữ tại client.
6. **Kiểm tra và báo cáo:** Mô hình được kiểm tra trên tập dữ liệu D_{test} và kết quả được báo cáo tới bộ điều khiển để theo dõi.
7. **Cập nhật cấu hình IP/Port:** Gọi hàm $\text{MTD}_{\text{IP}}(\text{config})$ để cập nhật cấu hình và chuyển đổi IP/port nếu cần, đảm bảo tiếp tục bảo mật và quyền riêng tư của hệ thống.

4.2.2.4 Đánh giá các kỹ thuật đối với các cuộc tấn công

Bảng 4.2.3: Các biện pháp giảm thiểu tiềm năng cho các cuộc tấn công trong DFL^[2]

Kỹ thuật	Eavesdropping	MitM	Network Mapping	Eclipse Attacks
Mã hóa	✓	✓	✗	✓
MTD	✗	✓	✓	✓

Bảng 4.2.3 cung cấp phân tích về hiệu quả của các biện pháp giảm thiểu khác nhau đối với các cuộc tấn công trong học phân tán phi tập trung (DFL). Nó làm nổi bật cách hai kỹ thuật chính, mã hóa và MTD, hoạt động hiệu quả trước các mối đe dọa khác nhau.

Mã hóa được chứng minh là một biện pháp hiệu quả, bảo vệ tốt các dữ liệu chống lại việc nghe lén, các cuộc tấn công giữa các bên (MitM) và các cuộc tấn công eclipse. Tuy nhiên, mã hóa không bảo vệ được mạng khỏi các cuộc tấn công lập bản đồ mạng, cho thấy rằng mặc dù nó bảo đảm an toàn cho việc truyền tải và truy cập dữ liệu, nhưng không thể ngăn chặn hoàn toàn các hoạt động do thám và lập bản đồ mạng.

Mặt khác, MTD chứng tỏ tính hiệu quả trong việc bảo vệ chống lại các cuộc tấn công MitM, lập bản đồ mạng và các cuộc tấn công eclipse, cho thấy khả năng tạo ra các thay đổi động trong cách hệ thống hoạt động để cản trở nỗ lực tấn công. Tuy nhiên, MTD không hiệu quả đối với việc nghe lén, điều này có nghĩa là mặc dù nó làm khó cho các kẻ tấn công trong việc khai thác và dự đoán cấu trúc mạng, nhưng không bảo vệ trực tiếp được nội dung giao tiếp.

Việc áp dụng hai kỹ thuật này cho thuật toán mang đến một cách tiếp cận bảo mật toàn diện. Mã hóa sẽ bảo vệ tính toàn vẹn và tính riêng tư của dữ liệu trong quá trình truyền tải, trong

khi MTD sẽ tăng cường khả năng chống chịu của hệ thống bằng cách làm khó cho kẻ tấn công trong việc dự đoán hoặc khai thác các lỗ hổng. Sự kết hợp này giúp hệ thống DFL trở nên an toàn hơn, có khả năng chống lại nhiều mối đe dọa tiềm ẩn đối với các cuộc tấn công đã được xác định như trên.

4.3 Phân tích tổng quan giữa BALANCE và MTD

BALANCE và **MTD** đem lại những cải tiến quan trọng trong từng khía cạnh riêng biệt do đó nếu sử dụng riêng cho hệ thống tổng quát sẽ có những hạn chế khác nhau:

- **BALANCE** tập trung vào việc đảm bảo tính chính xác của mô hình và khả năng hội tụ trong môi trường Byzantine, nhưng chưa giải quyết hiệu quả các mối đe dọa liên quan đến giao tiếp mạng.
- **MTD** tăng cường bảo mật mạng bằng cách thay đổi cấu trúc kết nối và sử dụng các kỹ thuật mã hóa, nhưng không trực tiếp giải quyết các vấn đề liên quan đến tính chính xác và hiệu suất của mô hình trong bối cảnh phân tán.

Kết luận: BALANCE và MTD đều đóng vai trò quan trọng trong việc bảo vệ hệ thống phân tán, nhưng mỗi phương pháp lại có những ưu điểm và hạn chế riêng biệt. BALANCE tập trung vào đảm bảo tính chính xác và khả năng hội tụ của mô hình trong môi trường Byzantine, trong khi MTD tăng cường bảo mật mạng trước các tấn công bên ngoài. Do đó, việc kết hợp linh hoạt cả hai phương pháp sẽ giúp nâng cao hiệu quả bảo vệ tổng thể, đồng thời cần cân nhắc đến hiệu suất và khả năng triển khai thực tế của hệ thống.

Chương 5

Đề xuất nghiên cứu

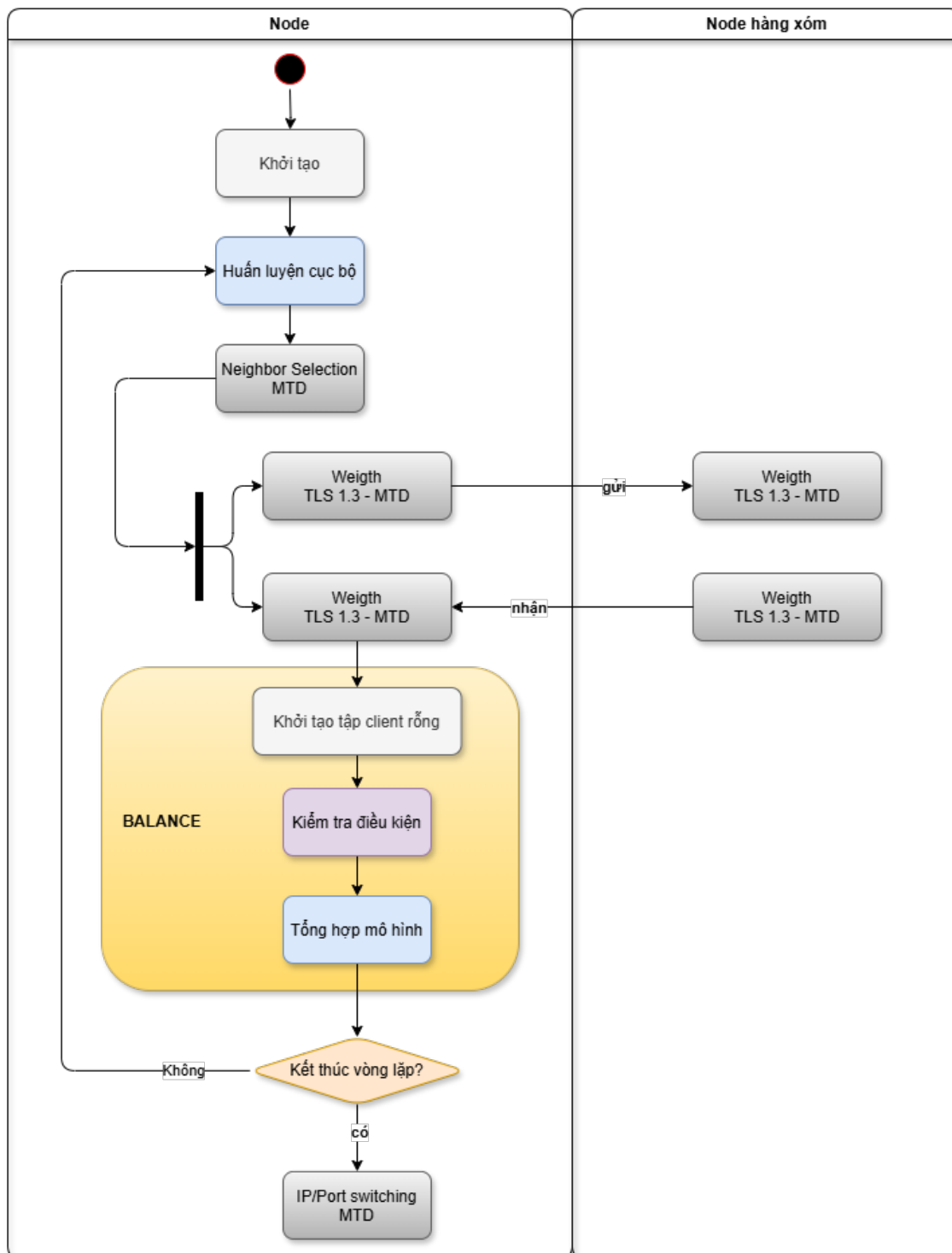
Dựa trên các phân tích trước đó cho thấy việc kết hợp **BALANCE** và **MTD** là một giải pháp hiệu quả nhằm khai thác ưu điểm của từng phương pháp. Cụ thể, **BALANCE** đảm bảo tính chính xác và hiệu suất của mô hình trong môi trường Byzantine, trong khi **MTD** nâng cao khả năng bảo mật mạng, giúp chống lại các mối đe dọa và tăng cường tính riêng tư.

5.1 Đề xuất giải pháp kết hợp **BALANCE**, **MTD** và Mã hóa

Từ những kết luận ở trên chúng tôi đề xuất một giải pháp tích hợp **BALANCE**^[4], **Moving Target Defense (MTD)**^[2] và **mã hóa bằng TLS 1.3** nhằm xây dựng hệ thống an toàn và đáng tin cậy hơn. Cụ thể:

- **BALANCE** được sử dụng làm cơ chế chính để tổng hợp mô hình, đảm bảo tính Byzantine-robust. Dựa trên sự tương đồng cục bộ (*local similarity*), thuật toán này giúp đánh giá và chọn lọc các mô hình hợp lệ, giảm thiểu tác động của các client độc hại và duy trì sự đồng nhất của mô hình.
- **Moving Target Defense (MTD)** được áp dụng thông qua giao thức *coin-flipping* để liên tục thay đổi hàng xóm trong quá trình trao đổi mô hình. Kỹ thuật này ngăn chặn việc lập bản đồ mạng (*network mapping*) và nghe lén (*eavesdropping*), đồng thời tăng cường khả năng chống chịu trước các cuộc tấn công mạng.
- **Mã hóa bằng TLS 1.3** đảm bảo tính bảo mật và toàn vẹn của kênh liên lạc giữa các client. Giao thức này hỗ trợ Perfect Forward Secrecy và giảm thiểu rủi ro tấn công kiểu *Man-in-the-Middle* (MitM), đồng thời duy trì hiệu suất truyền thông cao.

5.1. Đề xuất giải pháp kết hợp BALANCE, MTD và Mã hóa



Hình 5.1.1: Sơ đồ hệ thống DFL với BALANCE, TLS 1.3 và MTD

5.1.1 Cơ chế hoạt động của framework đề xuất

Quá trình thực thi của framework gồm các bước chính sau:

5.1. Đề xuất giải pháp kết hợp BALANCE, MTD và Mã hóa

1. **Thiết lập kết nối TLS 1.3:** Khi một node tham gia mạng, nó thực hiện peer discovery để tìm các node lân cận và tiến hành handshake TLS 1.3 với mutual authentication.
2. **Lựa chọn hàng xóm bằng coin-flipping:** Mỗi node tham gia vào giao thức coin-flipping với các candidate hàng xóm để quyết định kết nối dựa trên kết quả băm cam kết $H(x||\text{nonce})$.
3. **Huấn luyện cục bộ:** Các client thực hiện cập nhật mô hình cục bộ $\mathbf{w}_i^{t+\frac{1}{2}}$ bằng tập dữ liệu địa phương.
4. **Trao đổi mô hình được mã hóa:** Mô hình $\mathbf{w}_i^{t+\frac{1}{2}}$ được mã hóa đến các hàng xóm qua kênh TLS bảo mật.
5. **Lọc mô hình và tổng hợp theo BALANCE:** Client chỉ giữ lại mô hình hợp lệ theo tiêu chí $\|\mathbf{w}_i^{t+\frac{1}{2}} - \mathbf{w}_j^{t+\frac{1}{2}}\| \leq \gamma e^{-\kappa\lambda(t)} \|\mathbf{w}_i^{t+\frac{1}{2}}\|$.

5.1.2 Thuật toán tổng hợp BALANCE với TLS 1.3 và Coin-Flipping

Algorithm 6 Các thủ tục cho kỹ thuật IP/Port Switching và Neighbor Selection (Cập nhật với Coin-Flipping)

```
1: procedure MTDN( $N_{\text{all}}, p$ ) ▷ Lựa chọn client lân cận với coin-flipping
2:   Khởi tạo danh sách rỗng  $N_{\text{sel}}$ 
3:   for mỗi client  $i \in N_{\text{all}}$  do
4:     Chọn một số ngẫu nhiên  $x_i \in [0, 1)$  và một nonce
5:     Tính cam kết  $C_i = H(x_i||\text{nonce}_i)$ 
6:     Gửi  $C_i$  đến  $i$  và nhận lại cam kết  $C_j$  từ  $j$ 
7:   end for
8:   for mỗi client  $j \in N_{\text{all}}$  do
9:     Trao đổi  $(x_i, \text{nonce}_i)$  và nhận  $(x_j, \text{nonce}_j)$ 
10:    Xác minh cam kết:  $H(x_i||\text{nonce}_i) \stackrel{?}{=} C_i$ 
11:    Tính tổng  $S = (x_i + x_j) \bmod 1$ 
12:    if  $S < p$  then
13:      Thêm  $j$  vào danh sách  $N_{\text{sel}}$ 
14:    end if
15:  end for
16:  return  $N_{\text{sel}}$ 
17: end procedure

18: procedure MTDIP(config) ▷ Chuyển đổi IP/Port
19:   Lấy danh sách IP và cổng sẵn có:  $IP_{\text{avail}}, P_{\text{avail}}$ 
20:   Chọn một IP và cổng mới từ  $IP_{\text{avail}}, P_{\text{avail}}$  một cách ngẫu nhiên với xác suất bằng nhau
21:   Cập nhật cấu hình với địa chỉ IP và cổng mới
22:   return config
23: end procedure
```

5.1. Đề xuất giải pháp kết hợp BALANCE, MTD và Mã hóa

Algorithm 7 Hiện thực các client tham gia với Kỹ thuật Mã hóa và MTD trong DFL (Cập nhật với Coin-Flipping và BALANCE)

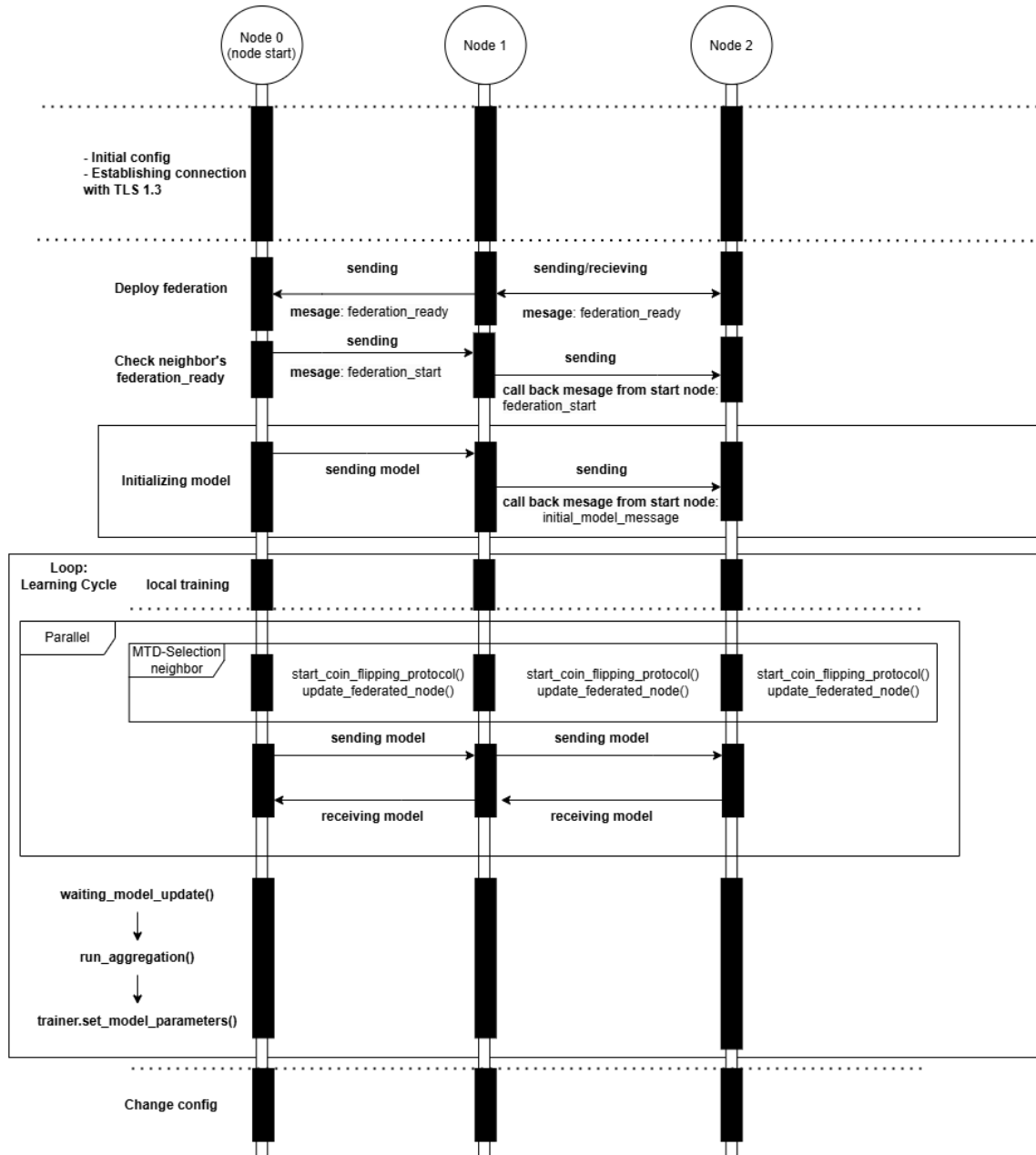
Require: R : số vòng lặp cục bộ, η : tốc độ học, λ : tham số điều chỉnh mức độ tiêu chuẩn, S_j : socket đến client lân cận j , D : tập dữ liệu cục bộ, MTD_{IP} : hàm MTD IP/port, MTD_N : hàm MTD lựa chọn client lân cận, p : ngưỡng coin-flipping, $\alpha, \gamma, \kappa, \lambda(t)$: tham số BALANCE

```

1:  $D_{\text{train}}, D_{\text{test}} \leftarrow \text{split}(D)$ 
2:  $\theta \leftarrow \text{Initialize}()$  ▷ Khởi tạo mô hình cục bộ
3: for  $r$  in  $R$  do
4:   for mỗi  $(x, y)$  trong  $D_{\text{train}}$  do
5:      $\theta \leftarrow \theta - \eta \nabla J(\theta, (x, y)) + \lambda \theta$ 
6:   end for
7:   Bước 1: Chọn hàng xóm bằng Coin-Flipping
8:    $N \leftarrow MTD_N(N_{\text{all}}, p)$ 
9:   Bước 2: Gửi mô hình qua kênh TLS 1.3 bảo mật
10:  for  $j$  in  $N$  do
11:    Gửi  $\mathbf{w}_i^{t+\frac{1}{2}}$  đến  $j$  qua kênh bảo mật  $\text{Channel}_{ij}$ 
12:  end for
13:  Bước 3: Nhận mô hình từ hàng xóm qua TLS 1.3
14:  Khởi tạo tập hàng xóm hợp lệ  $S_i^t \leftarrow \emptyset$ 
15:  while not Timeout do
16:    for  $j$  in  $N$  do
17:      Nhận  $\mathbf{w}_j^{t+\frac{1}{2}}$  từ  $j$  qua  $\text{Channel}_{ij}$ 
18:      if  $\|\mathbf{w}_i^{t+\frac{1}{2}} - \mathbf{w}_j^{t+\frac{1}{2}}\| \leq \gamma e^{-\kappa \lambda(t)} \|\mathbf{w}_i^{t+\frac{1}{2}}\|$  then
19:        Thêm  $j$  vào  $S_i^t$ 
20:      end if
21:    end for
22:  end while
23:  Bước 4: Tổng hợp mô hình theo BALANCE
24:  if  $|S_i^t| > 0$  then
25:     $\mathbf{w}_i^{t+1} \leftarrow \alpha \mathbf{w}_i^{t+\frac{1}{2}} + (1 - \alpha) \frac{1}{|S_i^t|} \sum_{j \in S_i^t} \mathbf{w}_j^{t+\frac{1}{2}}$ 
26:  else
27:     $\mathbf{w}_i^{t+1} \leftarrow \mathbf{w}_i^{t+\frac{1}{2}}$  ▷ Không cập nhật nếu không có hàng xóm hợp lệ
28:  end if
29:  Cập nhật mô hình cục bộ với  $\mathbf{w}_i^{t+1}$ 
30: end for
31: Bước 5: Kiểm tra và báo cáo kết quả
32: for mỗi  $(x, y)$  in  $D_{\text{test}}$  do
33:    $y_{\text{pred}} \leftarrow \text{Dự đoán}(\mathbf{w}_i^T, x)$ 
34: end for
35:  $T_{\text{test}} \leftarrow \frac{1}{|D_{\text{test}}|} \sum_{i=1}^{|D_{\text{test}}|} l(y_i, y_{\text{pred}})$ 
36: Gửi các chỉ số tới bộ điều khiển
37: Bước 6: Thay đổi IP/Port bằng MTD
38:  $MTD_{IP}(\text{config}) \rightarrow \text{config}$ 

```

5.1.3 Sơ đồ trình tự hệ thống



Hình 5.1.2: Sơ đồ trình tự mô phỏng quá trình giao tiếp và huấn luyện giữa các node trong hệ thống DPFL

5.2 Phân tích kỹ thuật tích hợp

5.2.1 Phân tích cơ chế giao tiếp đảm bảo tính nhất quán

Trong hệ thống DFL, các node phối hợp thông qua cơ chế bất đồng bộ (async) để giao tiếp và đồng bộ state, đồng thời sử dụng lock và timeout để đảm bảo không xảy ra race condition và không bị treo.

5.2.1.1 Locking (Khóa)

Việc sử dụng khóa (lock) là bắt buộc để bảo vệ các vùng dữ liệu quan trọng khỏi truy cập đồng thời. Trong hệ thống này, các cơ chế `threading.Lock` (cho môi trường đồng bộ) và `asyncio.Lock` (cho môi trường bất đồng bộ) được sử dụng để đảm bảo rằng chỉ một luồng hoặc coroutine được phép truy cập hoặc cập nhật dữ liệu chia sẻ tại một thời điểm nhất định. Ví dụ, các trường như `self.round`, trạng thái vòng học, và các bước khởi tạo federation đều được bao bọc bởi lock:

```
1         await some_lock.acquire()
2         # critical section
3         some_lock.release()
4
```

Listing 5.1: Sử dụng async lock

Điều này giúp ngăn chặn tình trạng race condition khi có nhiều coroutine hoặc luồng cùng truy cập và thay đổi dữ liệu, đảm bảo tính nhất quán của hệ thống.

5.2.1.2 Async/Await Coordination

Tất cả các hàm có liên quan đến giao tiếp, xử lý message, truyền và tổng hợp dữ liệu đều được định nghĩa bằng `async def`. Việc sử dụng từ khóa `await` giúp tận dụng triệt để khả năng non-blocking của event loop, từ đó nâng cao hiệu suất và khả năng xử lý song song của hệ thống. Cụ thể, một số các thao tác như:

- Kết nối và gửi thông điệp: `connect()`, `send_message()`
- Xử lý sự kiện: `event_manager.publish()`
- Đợi kết quả từ aggregator: `get_aggregation()`

đều được triển khai dưới dạng bất đồng bộ. Việc này không chỉ cho phép nhiều coroutine hoạt động đồng thời, mà còn đảm bảo các bước quan trọng được tuần tự hóa thông qua cơ chế lock.

5.2.1.3 Timeout Handling (Xử lý hết thời gian)

Để tránh việc một node hoặc một bước xử lý bị treo vô thời hạn (deadlock), cơ chế timeout được áp dụng rộng rãi. Hàm `asyncio.wait_for()` cho phép giới hạn thời gian chờ của một coroutine cụ thể. Ví dụ:

```
1     try:
2         result = await asyncio.wait_for(self.aggregator.
3         get_aggregation(), timeout=30)
4     except asyncio.TimeoutError:
5         logging.warning("Timeout.")
```

Listing 5.2: Sử dụng timeout với aggregator

Cơ chế này được áp dụng cho:

- Chờ kết quả aggregation giữa các round
- Chờ synchronization tại các điểm barrier giữa các node
- Các thao tác lock trong trường hợp cần giới hạn thời gian chờ

5.2.2 Áp dụng giải thuật BALANCE

Trong phần này, chúng tôi sẽ minh họa chi tiết cách áp dụng giải thuật BALANCE thông qua phần thực thi cụ thể.

5.2.2.1 Khởi tạo lớp BALANCE (Phần `__init__`)

```
1 class Balance(Aggregator):
2     def __init__(
3         self,
4         config=None,
5         **kwargs,
6     ):
7         super().__init__(config, **kwargs)
8         self.gamma = float(self.config.participant["aggregator_args"]["
9         balance_params"]["gamma"])
10        self.kappa = float(self.config.participant["aggregator_args"]["
11        balance_params"]["kappa"])
12        self.alpha = float(self.config.participant["aggregator_args"]["
13        balance_params"]["alpha"])
14        self.R = int(self.config.participant["scenario_args"]["rounds"])
15        self.lambda_func = lambda r: r / float(self.R)
```

5.2. Phân tích kỹ thuật tích hợp

Giải thích các tham số:

- **gamma:** Hệ số điều chỉnh ngưỡng tương đồng ban đầu.
- **kappa:** Điều chỉnh tốc độ giảm ngưỡng tương đồng qua từng vòng. Giá trị càng lớn thì ngưỡng sẽ giảm càng nhanh.
- **alpha:** Điều chỉnh mức độ kết hợp giữa mô hình cục bộ và các mô hình hàng xóm được coi là tương đồng.

Các tham số trên được lấy từ config khởi tạo của mỗi scenario tương ứng. Điều này cho phép người dùng tùy ý điều chỉnh tham số theo từng trường hợp cụ thể hoặc dựa trên đặc điểm mạng nhằm tối ưu hoá hệ thống.

5.2.2.2 Hàm tính toán khoảng cách giữa hai mô hình

```
1 def model_distance(m1, m2):
2     dist_sq = 0.0
3     for layer in m1:
4         diff = (m1[layer] - m2[layer]).float()
5         dist_sq += torch.sum(diff * diff).item()
6     return math.sqrt(dist_sq)
```

Giải thích: Hàm trên tính khoảng cách **Euclidean (L2)** giữa hai mô hình m_1 và m_2 . Cụ thể:

$$\text{dist}(m_1, m_2) = \sqrt{\sum_{\text{layer}} \|m_1[\text{layer}] - m_2[\text{layer}]\|_2^2}$$

Ví dụ, nếu $m_1[\text{layer}] = [1.0, 2.0]$ và $m_2[\text{layer}] = [1.5, 1.0]$, khoảng cách sẽ là:

$$\text{dist} = \sqrt{(1.0 - 1.5)^2 + (2.0 - 1.0)^2} = \sqrt{0.25 + 1} = 1.118$$

5.2.2.3 Quá trình tổng hợp mô hình

1. Tính ngưỡng tương đồng động:

```
1     decay_factor = math.exp(-self.kappa * lam_t)
2     threshold = self.gamma * decay_factor * model_norm(local_model)
3
```

Ngưỡng này giảm dần qua từng vòng huấn luyện t . Cụ thể:

- **decay_factor** là hệ số giảm ngưỡng.

- `model_norm(local_model)` tính độ lớn của mô hình cục bộ (chuẩn L2 của tensor trọng số).
- Kết quả `threshold` xác định khoảng cách tối đa để coi một mô hình hàng xóm là tương đồng.

2. Khởi tạo bộ đệm để lưu trữ các mô hình tương đồng:

```
1 similar_accum = {layer: torch.zeros_like(local_model[layer])
2 for layer in local_model}
3 total_weight_similar = 0.0
4 similar_models_count = 0
```

- `similar_accum[layer]` có cấu trúc tương ứng với mô hình cục bộ với dữ liệu rỗng (các tensor chứa toàn số 0). Giá trị này được sử dụng để cộng dồn (tích lũy) tham số của các mô hình "tương tự" khi thực hiện quá trình tổng hợp.
- `total_weight_similar` lưu tổng trọng số các mô hình "tương tự". Sau khi cộng dồn xong, tổng trọng số này sẽ được dùng để chuẩn hóa (normalize) các tham số cộng dồn trong `similar_accum`, giúp tính trung bình chính xác.
- `similar_models_count` theo dõi số lượng mô hình tương đồng.

3. Kiểm tra và chọn lọc mô hình tương đồng:

```
1 for addr, (nbr_model, nbr_weight) in models.items():
2     if addr == local_addr:
3         continue
4     dist = model_distance(local_model, nbr_model)
5     if dist <= threshold:
6         for layer in similar_accum:
7             similar_accum[layer] += nbr_model[layer].float() *
8             nbr_weight
9             total_weight_similar += nbr_weight
10            similar_models_count += 1
```

- Lặp qua từng mô hình hàng xóm.
- Nếu khoảng cách Euclidean giữa mô hình cục bộ và hàng xóm nhỏ hơn ngưỡng `threshold`, mô hình đó được coi là tương đồng.
- Các trọng số tương đồng được cộng dồn vào `similar_accum[layer]`.
- Cập nhật các giá trị `total_weight_similar` và `similar_models_count`

5.2. Phân tích kỹ thuật tích hợp

4. Trả về mô hình cục bộ nếu không tìm được mô hình hàng xóm tương đồng

```
1     if similar_models_count == 0:
2         return local_model
3
```

5. Trung bình hóa và trộn:

```
1     for layer in similar_accum:
2         similar_accum[layer] /= total_weight_similar
3     final_model = {}
4     for layer in local_model:
5         final_model[layer] = self.alpha * local_model[layer].float
6         () + (1.0 - self.alpha) * similar_accum[layer]
```

- Sau khi tích lũy các mô hình tương đồng, hệ thống tính trung bình hóa.
- Cuối cùng, `final_model[layer]` là sự kết hợp giữa mô hình cục bộ và trung bình các mô hình hàng xóm tương đồng, với tỉ lệ pha trộn được điều chỉnh bởi `alpha`.

6. Dọn dẹp bộ nhớ và trả về mô hình kết quả:

```
1     gc.collect()
2     return final_model
3
```

Dọn dẹp các tham chiếu không cần thiết để tiết kiệm bộ nhớ.

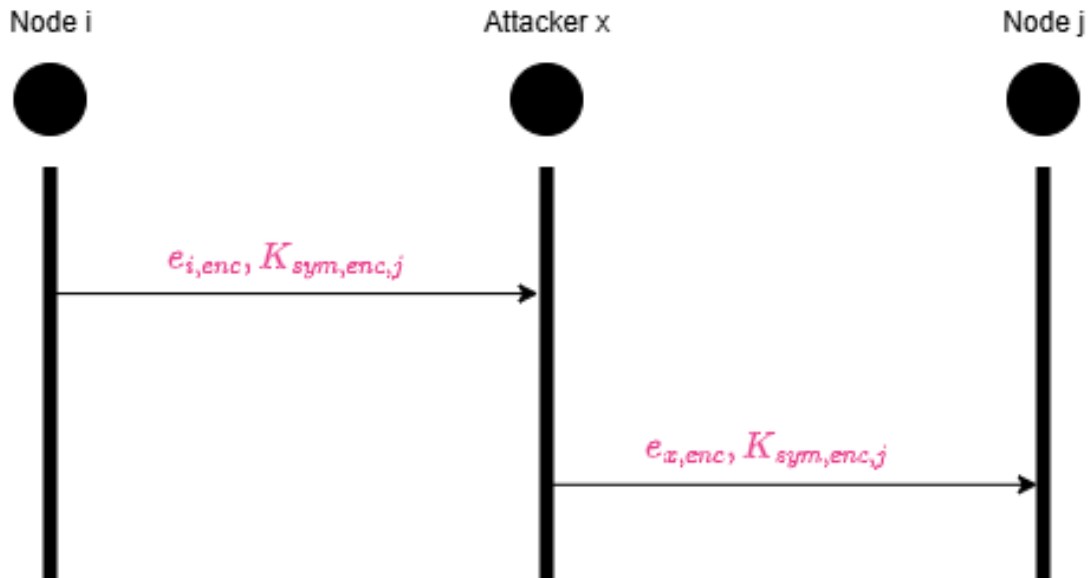
5.2.3 Kỹ thuật mã hóa

5.2.3.1 Phân tích hướng tiếp cận ban đầu

Ở thuật toán 5 đã được nhắc đến ở phần trước, thì sau khi các client đào tạo xong mô hình các client sẽ sử dụng từng khóa đối xứng $K_{sym,j}$ cho từng hàng xóm j để mã hóa trọng số được đào tạo $e_{i,enc} \leftarrow E_{sym}(\mathbf{w}_i^{t+\frac{1}{2}}, K_{sym})$, sau đó sử dụng khóa trao đổi được công khai của hàng xóm K_j^{pub} để mã hóa khóa đối xứng $K_{sym,enc,j} \leftarrow E_{asym}(K_{sym}, K_j^{pub})$, trọng số mã hóa $e_{i,enc}$ và khóa đối xứng mã hóa $K_{sym,enc,j}$ sẽ được gửi cho hàng xóm j tương ứng.

Tuy nhiên với cách tiếp cận đơn giản như này thì kẻ tấn công dễ dàng triển khai cuộc tấn công Man in the middle, kẻ tấn công có thể đưa các dữ liệu của chính chúng thay vì dữ liệu của các client gửi, điều này khiến cho các client nhận mất đi các dữ liệu đúng đắn, giảm chất lượng mô hình đào tạo. Kẻ tấn công có thể triển khai cuộc tấn công này thành một cuộc tấn công cô lập (Eclipse attack) khiến cho dữ liệu nhận vào của 1 client nhận được kẻ tấn công xác định

phụ thuộc hoàn toàn vào kẻ tấn công, không đảm bảo được tính đúng đắn dữ liệu nhận vào của mình.



Hình 5.2.1: Tấn công man in the middle trong triển khai ban đầu

5.2.3.2 Đề xuất giao thức ban đầu

Sau khi phân tích điểm yếu của hướng tiếp cận ban đầu, ta nhận thấy rằng khi các client chỉ sử dụng khóa đối xứng riêng biệt cho việc mã hóa dữ liệu, kẻ tấn công có thể dễ dàng thực hiện cuộc tấn công Man-in-the-Middle (MITM) hoặc tấn công cô lập (Eclipse attack) bằng cách thay thế dữ liệu thật bằng dữ liệu giả mạo. Điều này dẫn đến việc client nhận được thông tin sai lệch, ảnh hưởng tiêu cực đến chất lượng mô hình đào tạo.

Để khắc phục vấn đề này, chúng tôi quyết định áp dụng cơ chế handshake giữa các client trước khi trao đổi dữ liệu. Cụ thể:

- Khi các client kết nối, chúng sẽ thực hiện một quá trình trao đổi khóa (sử dụng thuật toán ECDH^[21]) để tạo ra một khóa đối xứng chung. Việc này giúp đảm bảo rằng khóa dùng để mã hóa không cần phải gửi trực tiếp qua mạng, giảm thiểu nguy cơ bị lộ.
- Đồng thời, các client sử dụng cặp khóa Schnorr với khả năng tạo chữ ký ngắn, hiệu suất cao và khả năng mở rộng tốt, hiện đang được sử dụng phổ biến tại Bitcoin^[22], để ký số các thông điệp trao đổi, giúp xác thực danh tính của nhau và ngăn chặn các cuộc tấn công giả mạo.
- Cuối cùng, dữ liệu được mã hóa bằng thuật toán AES-GCM, không chỉ bảo vệ tính bảo mật mà còn đảm bảo tính toàn vẹn của dữ liệu qua cơ chế kiểm tra MAC tích hợp^[23].

5.2. Phân tích kỹ thuật tích hợp

Dưới đây là scheme chi tiết của giao thức:

Giao thức: A và B thực hiện handshake để tạo khóa đối xứng chung, sau đó dùng AES-GCM để trao đổi dữ liệu.

1. Step 1: Khởi tạo khóa

Mỗi client A và B tạo ra 2 cặp khóa:

- **Schnorr Key Pair** (dùng cho xác thực và chữ ký):
 - A: (SP_a, SPV_a)
 - B: (SP_b, SPV_b)
- **ECDH Key Pair** (dùng để tạo shared key an toàn):
 - A: (P_a, PV_a)
 - B: (P_b, PV_b)

2. Step 2: Handshake để tạo khóa chung

a) Msg1 (từ A):

A gửi khóa công khai ECDH P_a , kèm theo public key xác thực SPV_a và chữ ký số trên P_a :

$$\text{Msg1} = (P_a, SPV_a, \text{Sign}(SPV_a, P_a))$$

b) Xác thực Msg1 tại B:

B sử dụng khóa xác thực của A (đã được chia sẻ trước đó, SP_a) để kiểm tra chữ ký. Nếu không hợp lệ, B từ chối kết nối; nếu hợp lệ, B lưu lại P_a và tiếp tục.

c) Msg2 (từ B):

B gửi khóa công khai ECDH P_b , kèm theo public key xác thực SPV_b và chữ ký số trên P_b :

$$\text{Msg2} = (P_b, SPV_b, \text{Sign}(SPV_b, P_b))$$

d) Xác thực Msg2 tại A:

A sử dụng khóa xác thực của B (đã được chia sẻ trước đó, SP_b) để kiểm tra chữ ký. Nếu không hợp lệ, A hủy kết nối; nếu hợp lệ, A lưu lại P_b và tiếp tục.

e) Thiết lập khóa chung:

Cả A và B tính toán khóa chung qua ECDH:

$$K_{\text{shared}} = \text{KDF}(PV_a \cdot P_b) = \text{KDF}(PV_b \cdot P_a)$$

Sau đó, dùng hàm dẫn xuất khóa (ví dụ HKDF) với salt để tạo ra khóa AES:

$$K_{\text{AES}} = \text{HKDF}(K_{\text{shared}}, \text{salt})$$

3. Step 3: Mã hóa và gửi dữ liệu

a) Mã hóa tại A:

A mã hóa dữ liệu w (ví dụ, trọng số mô hình) bằng AES-GCM với khóa K_{AES} và một IV (Initialization Vector):

$$M_a = \text{AES-GCM}(K_{\text{AES}}, w, IV)$$

b) Gửi dữ liệu:

A gửi cặp (M_a, IV) cho B.

4. Step 4: Giải mã dữ liệu tại B

a) Giải mã:

B sử dụng AES-GCM với khóa K_{AES} và IV nhận được để giải mã:

$$w = \text{AES-GCM-Decrypt}(K_{\text{AES}}, M_a, IV)$$

Lợi ích:

- **Chống MITM:** Chữ ký số đảm bảo rằng các thông điệp trao đổi khóa không bị giả mạo, chỉ cho phép các client hợp lệ tham gia.
- **Bảo mật khóa:** Khóa AES được sinh ra từ ECDH handshake và không bao giờ được gửi qua mạng, giảm nguy cơ rò rỉ thông tin.
- **Xác thực hai chiều:** Cả A và B đều thực hiện xác minh danh tính thông qua chữ ký số, tăng cường tính bảo mật.
- **Toàn vẹn dữ liệu:** AES-GCM đảm bảo rằng dữ liệu không bị chỉnh sửa trong quá trình truyền nhờ cơ chế kiểm tra MAC tích hợp.

5.2.3.3 Đánh giá sơ bộ và góp ý cho giao thức ban đầu

Chúng tôi nhận ra giao thức được đề xuất có một vấn đề lớn ở chỗ xác thực qua Schnorr. Vì quá trình xác thực này hoàn toàn dựa vào mô hình phi tập trung, nên trong quá trình thiết lập handshake, các client không thể đảm bảo chắc chắn rằng keypair Schnorr nhận được thực sự

5.2. Phân tích kỹ thuật tích hợp

thuộc về một client hợp lệ chứ không phải của attacker. Vấn đề này trở nên nghiêm trọng hơn trong hệ thống dynamic giao thức, khi mỗi lần thiết lập kết nối, attacker có thể lợi dụng cơ hội để tiến hành tấn công MITM, giả mạo keypair Schnorr và tạo ra một kết nối giả mạo nhằm thao túng thông tin trao đổi.

Với hệ thống DPFL, chúng tôi đặt mục tiêu cao trong việc đảm bảo các client tham gia phải được xác thực danh tính một cách chính xác và tin cậy. Vì vậy, ngoài việc thực hiện handshake qua ECDH để tạo ra khóa đối xứng chung và mã hóa dữ liệu bằng AES-GCM, cần bổ sung một cơ chế xác thực danh tính mạnh mẽ hơn—có thể dựa trên cơ chế chứng thực phi tập trung hoặc sử dụng hệ thống chứng chỉ (certificate authority) đáng tin cậy—để đảm bảo rằng keypair được sử dụng trong quá trình handshake thực sự là của client thật sự. Việc cải tiến phần xác thực này sẽ góp phần ngăn chặn các cuộc tấn công giả mạo và tăng cường tính bảo mật cho toàn hệ thống.

Chúng tôi nhận thấy rằng scheme của chúng tôi có nhiều điểm tương đồng với giao thức TLS hiện nay:

- **Trao đổi khóa và thiết lập kết nối:** Giống như TLS, scheme của chúng tôi sử dụng handshake dựa trên ECDH để thiết lập khóa đối xứng chung, cho phép cả hai bên cùng tính toán ra một khóa bảo mật mà không cần truyền tải trực tiếp khóa đó qua mạng.
- **Mã hóa và đảm bảo tính toàn vẹn:** Dữ liệu được mã hóa bằng AES-GCM, tương tự như cơ chế trong TLS, đảm bảo rằng dữ liệu không bị nghe lén hay thay đổi thông qua việc tích hợp MAC.
- **Xác thực danh tính:** Trong TLS, chứng chỉ số do CA cấp phát giúp đảm bảo rằng public key được sử dụng thực sự thuộc về các thực thể hợp lệ. Điều này giúp ngăn chặn các cuộc tấn công giả mạo và MITM, tương tự như mục tiêu mà chúng tôi mong muốn đạt được.

Do đó, việc tích hợp CA và sử dụng TLS không chỉ khắc phục hạn chế của xác thực qua Schnorr trong mô hình phi tập trung mà còn đảm bảo rằng hệ thống DPFL đạt được mức độ bảo mật cao, phù hợp với các tiêu chuẩn an ninh hiện đại.

5.2.3.4 Giao thức TLS

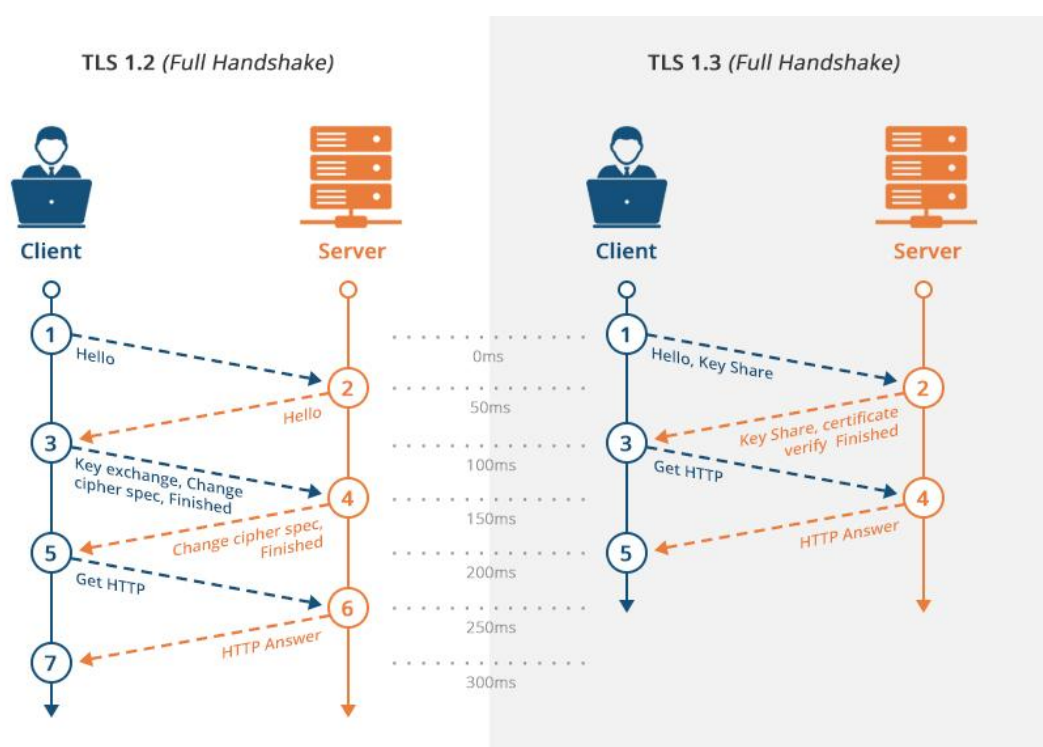
Transport Layer Security (TLS) là giao thức bảo mật chủ yếu dùng để bảo vệ dữ liệu trao đổi giữa hai bên qua mạng. TLS đảm bảo:

- **Mã hóa dữ liệu:** Giúp dữ liệu (ví dụ: thông tin cá nhân, dữ liệu giao tiếp) không bị đọc trộm bởi các bên trái phép.
- **Xác thực danh tính:** Đảm bảo rằng bạn đang giao tiếp với đúng đối tượng mong đợi, từ đó ngăn chặn các cuộc tấn công giả mạo (spoofing).

5.2. Phân tích kỹ thuật tích hợp

- **Bảo đảm tính toàn vẹn:** Giúp phát hiện sớm bất kỳ thay đổi nào đối với dữ liệu trong quá trình truyền tải.

TLS 1.3 là phiên bản cải tiến mới nhất của giao thức TLS với nhiều điểm mạnh vượt trội, được chứng minh qua các kỹ thuật an ninh tiên tiến và được công nhận bởi Viện Tiêu chuẩn và Công nghệ Quốc gia của Hoa Kỳ - NIST^[24] và Lực lượng Chuyên trách về Kỹ thuật Liên mạng - IETF^[25]:



Hình 5.2.2: TLS 1.2 với TLS 1.3^[5]

1. Quá trình bắt tay (handshake) đơn giản và nhanh chóng:

- **Giảm số lượt trao đổi:** So với TLS 1.2, TLS 1.3 rút gọn quy trình handshake xuống còn những bước cần thiết, giúp giảm độ trễ và tăng tốc độ thiết lập kết nối.
- **0-RTT (Zero Round-Trip Time):** Tùy chọn cho phép tái kết nối nhanh cho các phiên giao tiếp đã được xác thực, cải thiện hiệu suất trong các ứng dụng thời gian thực. (Lưu ý: Cần quản lý cẩn thận để tránh các tấn công replay.)

2. Bảo mật được cải thiện:

- **Thuật toán mã hóa hiện đại:** TLS 1.3 chỉ hỗ trợ các ciphers an toàn dựa trên cơ chế AEAD (Authenticated Encryption with Associated Data) như AES-GCM và

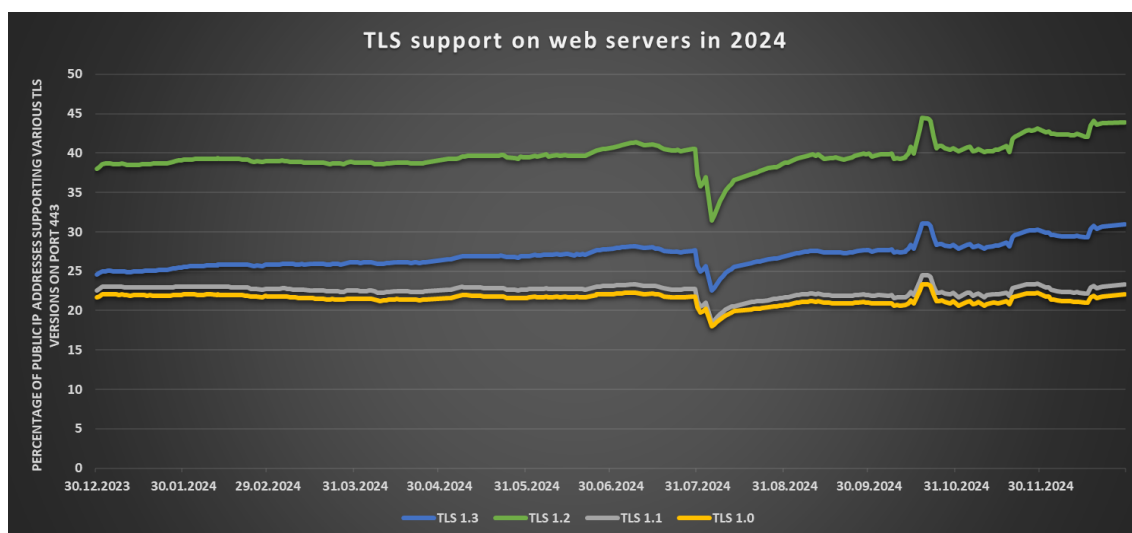
5.2. Phân tích kỹ thuật tích hợp

ChaCha20-Poly1305, loại bỏ các thuật toán lỗi thời. Điều này giúp giảm nguy cơ tấn công như padding oracle và các cuộc tấn công dựa trên thuật toán mã hóa yếu.

- **Khóa tạm thời (Ephemeral Keys) và Forward Secrecy:** Sử dụng các thuật toán trao đổi khóa như Diffie-Hellman tạm thời (DHE) hoặc Elliptic Curve Diffie-Hellman tạm thời (ECDHE) để tạo khóa phiên mới cho mỗi kết nối. Nhờ đó, ngay cả khi khóa dài hạn bị xâm phạm, các phiên giao tiếp trước đó vẫn được bảo vệ an toàn.
- **Loại bỏ các tính năng không cần thiết:** Việc cắt bỏ các tùy chọn và thuật toán lỗi thời giúp giảm bề mặt tấn công và hạn chế rủi ro từ cấu hình sai.

3. Cấu trúc đơn giản, bảo vệ quyền riêng tư và sự tin cậy:

- **Giảm phức tạp:** Cấu trúc được tối ưu hóa giúp hạn chế các tùy chọn cấu hình phức tạp, giảm khả năng mắc lỗi trong triển khai.
- **Bảo vệ quyền riêng tư:** TLS 1.3 giới hạn lượng thông tin trao đổi trong quá trình bắt tay, giảm nguy cơ bị theo dõi và thu thập dữ liệu trái phép.
- **Vai trò của CA (Certificate Authority):**
 - **Xác thực danh tính:** Trong quá trình handshake, máy chủ gửi chứng chỉ số được ký bởi CA đáng tin cậy. Điều này cho phép client xác thực danh tính máy chủ thông qua chuỗi tin cậy (trust chain) từ các chứng chỉ gốc.
 - **Chống giả mạo:** Việc sử dụng chứng chỉ số giúp ngăn chặn các cuộc tấn công trung gian (MITM) và đảm bảo rằng thông tin xác thực không bị thay đổi trong quá trình truyền tải.
 - **Công nghệ Certificate Transparency:** Một số CA hiện nay áp dụng cơ chế giám sát chứng chỉ, giúp phát hiện sớm các chứng chỉ giả mạo hay bị lạm dụng, tăng cường bảo mật toàn diện.



Hình 5.2.3: Thống kê về các web server sử dụng các loại giao thức TLS vào năm 2024^[6]

5.2.3.5 Triển khai TLS 1.3 trong hệ thống

Cấp chứng chỉ từ bên thứ ba đáng tin cậy

Trong hệ thống DPFL, để đảm bảo rằng mỗi client tham gia đều có danh tính xác thực và có thể thiết lập kết nối bảo mật với các client khác, chúng tôi sử dụng một bên thứ ba đáng tin cậy (trusted party) để cấp chứng chỉ số hợp lệ. Trusted party này hoạt động như một Certificate Authority (CA), đóng vai trò xác thực danh tính của các client trước khi cấp chứng chỉ.

Quy trình cấp chứng chỉ

Khi một client muốn tham gia vào hệ thống DPFL, quy trình cấp chứng chỉ sẽ diễn ra như sau:

- Tạo cặp khóa:** Mỗi client tạo một cặp khóa bất đối xứng (public key và private key) theo thuật toán Elliptic Curve Digital Signature Algorithm (ECDSA). Public key này sẽ được sử dụng trong chứng chỉ số.
- Tạo Certificate Signing Request (CSR):** Client tạo một yêu cầu ký chứng chỉ (CSR) chứa các thông tin như:
 - Public key của client
 - Thông tin nhận dạng (ID, địa chỉ, hoặc định danh trong hệ thống)
 - Chữ ký số của client (được tạo bằng private key)

5.2. Phân tích kỹ thuật tích hợp

3. **Gửi CSR đến Trusted Party (CA):** Client gửi yêu cầu CSR đến trusted party, kèm theo các thông tin cần thiết để xác minh danh tính.
4. **Xác minh danh tính:** Trusted party kiểm tra các thông tin của client bằng các cơ chế xác thực, như:
 - Đối chiếu với danh sách các client hợp lệ trong hệ thống
 - Sử dụng giao thức xác thực mạnh như OAuth, OpenID, hoặc xác minh offline theo chính sách của hệ thống
5. **Ký và cấp chứng chỉ:** Nếu quá trình xác minh thành công, trusted party sẽ sử dụng private key của CA để ký chứng chỉ số cho client. Chứng chỉ này sẽ chứa:
 - Public key của client
 - Thông tin nhận dạng đã được xác thực
 - Chữ ký số của CA
 - Thời hạn hiệu lực
6. **Gửi chứng chỉ về client:** Client nhận chứng chỉ số và có thể sử dụng nó để xác thực danh tính trong quá trình thiết lập kết nối TLS với các client khác.

Triển khai mTLS trong hệ thống DPFL

Chúng tôi sử dụng cơ chế Mutual TLS Authentication (mTLS) để đảm bảo rằng cả client và server đều có thể xác thực danh tính của nhau. Khi một client thiết lập kết nối với một client khác, cả hai bên đều phải cung cấp chứng chỉ hợp lệ được ký bởi CA. Cơ chế này giúp ngăn chặn các cuộc tấn công Man-in-the-Middle (MITM) và đảm bảo rằng chỉ các client hợp lệ mới có thể tham gia vào hệ thống.

```
1 import ssl
2
3 def create_ssl_context(self, role: str = "server"):
4     context = None
5     if role == "server":
6         context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER) # verify incoming
           connections
7     else:
8         context = ssl.create_default_context(ssl.Purpose.SERVER_AUTH) #
           verify outgoing connections
9     context.minimum_version = ssl.TLSVersion.TLSv1_3
10    context.load_cert_chain(
```

```
11     certfile=self.config.participant["security_args"]["certfile"],
12     keyfile=self.config.participant["security_args"]["keyfile"],
13 )
14 context.load_verify_locations(self.config.participant["security_args"]["
    cafile"])
15 context.verify_mode = ssl.CERT_REQUIRED
16 return context
17
```

Listing 5.3: Mã nguồn triển khai SSL context

Quá trình thiết lập kết nối mTLS

1. **Cả server và client tải chứng chỉ số của mình từ trusted party:** Mỗi client trong hệ thống (server hoặc client) cần có một chứng chỉ số và khóa riêng do một CA tin cậy cấp. Các chứng chỉ này được lưu trữ trong file cấu hình của từng client, cụ thể là:
 - certfile: Chứng chỉ của client.
 - keyfile: Khóa riêng tư của client.
 - cafile: Chứng chỉ CA để xác thực đối tác kết nối.
2. **Server thiết lập một SSLContext với CA tin cậy:** Khi khởi động, server gọi hàm `create_ssl_context` với tham số `role="server"`. Điều này sẽ:
 - Tạo một đối tượng `SSLContext` với `ssl.PROTOCOL_TLS_SERVER`, đảm bảo server có thể xác thực các kết nối đến.
 - Nạp chứng chỉ và khóa riêng từ file cấu hình của server.
 - Thiết lập danh sách CA tin cậy để xác thực client.
 - Yêu cầu client cung cấp chứng chỉ (`context.verify_mode = ssl.CERT_REQUIRED`).
3. **Client thiết lập một SSLContext tương tự và kiểm tra chứng chỉ của server:** Khi client cần kết nối với server, nó cũng gọi `create_ssl_context` nhưng với tham số mặc định (`role="client"`). Điều này sẽ:
 - Tạo một đối tượng `SSLContext` với `ssl.Purpose.SERVER_AUTH`, đảm bảo client có thể xác thực server.
 - Nạp chứng chỉ và khóa riêng của client.
 - Tải danh sách CA tin cậy để kiểm tra chứng chỉ của server.
 - Yêu cầu server cung cấp chứng chỉ hợp lệ.

5.2. Phân tích kỹ thuật tích hợp

4. **Cả hai bên thực hiện bắt tay TLS 1.3, trao đổi chứng chỉ, và xác minh danh tính trước khi truyền dữ liệu:** Sau khi thiết lập SSL context, server và client tiến hành bắt tay TLS 1.3 với các bước sau:

- Server gửi chứng chỉ của mình để client kiểm tra tính hợp lệ.
- Client gửi chứng chỉ của mình để server xác thực.
- Nếu cả hai chứng chỉ đều hợp lệ (được ký bởi CA tin cậy và không bị thu hồi), quá trình bắt tay thành công.
- Sau khi bắt tay, dữ liệu truyền qua kênh này sẽ được mã hóa end-to-end bằng TLS 1.3.

Việc triển khai mTLS giúp tăng cường tính bảo mật của hệ thống DPFL bằng cách yêu cầu xác thực hai chiều giữa các client tham gia.

5.2.4 Kỹ thuật MTD

5.2.4.1 Phân tích hướng tiếp cận ban đầu

Trong kỹ thuật MTD, có hai thành phần chính là **Neighbor Selection** và **Dynamic IP Switching**. Ở thuật toán 6 và 7, quá trình Neighbor Selection được thực hiện trước, sau đó mới đến bước thay đổi IP. Tuy nhiên, với kiến trúc của hệ thống, việc thay đổi IP không thể diễn ra trong quá trình training và trao đổi model mà chỉ có thể thực hiện **sau khi kết thúc vòng lặp**. Điều này có nghĩa là trong suốt quá trình chọn hàng xóm và trao đổi mô hình, các client sẽ duy trì địa chỉ IP hiện tại của mình mà không có sự thay đổi.

Hơn nữa, hệ thống của chúng tôi sẽ sử dụng **Certificate Authority (CA)** để lưu trữ và xác thực thông tin DNS của từng client. Điều này đảm bảo rằng các client có thể xác minh danh tính của nhau một cách an toàn trước khi thiết lập kết nối, giảm nguy cơ tấn công giả mạo. Trong hệ thống mô phỏng hiện tại, chúng tôi giả định rằng các client có một danh sách các địa chỉ IP có sẵn và có thể lựa chọn ngẫu nhiên từ danh sách này để thực hiện quá trình Dynamic IP. Điều này giúp kiểm soát quá trình thay đổi địa chỉ mà không cần tích hợp các giao thức cập nhật DNS phức tạp.

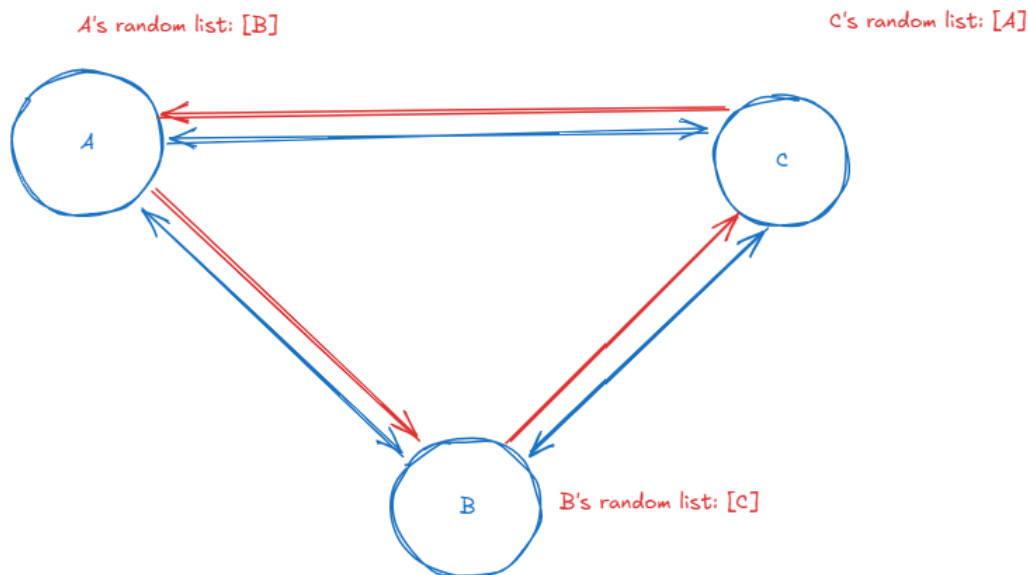
Để tăng cường bảo mật cho hệ thống DNS động, **DNS Security Extensions (DNSSEC)** sẽ được xem xét để sử dụng trong các triển khai thực tế trong tương lai. DNSSEC có thể giúp bảo vệ quá trình phân giải tên miền khỏi các cuộc tấn công như cache poisoning hay man-in-the-middle, đảm bảo rằng các client luôn có thể truy xuất thông tin DNS một cách an toàn.

Khi triển khai thực tế, có thể áp dụng thêm các biện pháp bảo mật nâng cao như xác thực đa yếu tố hoặc cơ chế chống tấn công DNS spoofing. Để xử lý vấn đề IP động, có thể cân nhắc các kỹ thuật sau:

- **Multihop routing:** Các client không kết nối trực tiếp mà thông qua nhiều bước trung gian, giúp giảm rủi ro mất kết nối khi IP thay đổi.
- **Proxy cố định:** Mỗi client có thể sử dụng một proxy với địa chỉ cố định, giúp duy trì danh tính mạng ngay cả khi IP thực thay đổi.
- **Cơ chế cập nhật địa chỉ động:** Sau mỗi vòng lặp, các client có thể gửi thông tin cập nhật IP mới cho hàng xóm của mình để duy trì kết nối mà không cần IP tĩnh.

Tuy nhiên, trong hệ thống của chúng tôi, IP của client chỉ thay đổi sau mỗi vòng lặp, và hệ thống sử dụng nền tảng **Nebula**^[7] không tiếp tục chạy vòng lặp mới. Do đó, chúng tôi sẽ tập trung vào phân tích kỹ thuật **Neighbor Selection**.

Thuật toán MTD_N trong kỹ thuật **Neighbor Selection** chọn ngẫu nhiên n client từ tập hợp các client hàng xóm N_{all} mà không lặp lại, nghĩa là mỗi client tự tạo danh sách hàng xóm riêng dựa trên cơ chế chọn ngẫu nhiên. Tuy nhiên điều này dẫn đến vấn đề: một client có thể gửi model đến một số hàng xóm mà không nhận lại được model từ phía chúng, gây gián đoạn quá trình trao đổi model và làm giảm hiệu suất aggregate.



Hình 5.2.4: Quá trình trao đổi model bị gián đoạn

Như trong hình 5.2.4, giả sử có 3 client A, B, C đã thiết lập kết nối đầy đủ với nhau (kết nối màu xanh). Với $N_{all} = 3$ và $n = 1$, mỗi client sẽ chọn ngẫu nhiên một hàng xóm từ tập hợp các client. Giả sử danh sách kết nối ngẫu nhiên (kết nối màu đỏ) được tạo ra như sau:

5.2. Phân tích kỹ thuật tích hợp

- Danh sách của A: [B]
- Danh sách của B: [C]
- Danh sách của C: [A]

Trong trường hợp này, A chỉ gửi model cho B, B chỉ gửi model cho C và C chỉ gửi model cho A. Điều này dẫn đến việc A chờ nhận model từ B, B chờ từ C, và C chờ từ A, tạo thành một vòng deadlock khi mà không có ai nhận được model mà họ mong đợi. Tình huống này minh họa rõ ràng rằng việc tạo danh sách kết nối ngẫu nhiên có thể dẫn đến sự mất cân bằng trong trao đổi model, gây gián đoạn và ảnh hưởng tiêu cực đến hiệu suất aggregate.

Để giải quyết vấn đề này, có hai giải pháp được đề xuất:

- **Giải pháp gửi model đến tất cả các client:** Mỗi client gửi model của mình đến tất cả các client hàng xóm, nhưng chỉ aggregate những model từ danh sách ngẫu nhiên đã chọn. Mặc dù giải pháp này đảm bảo rằng không bỏ sót thông tin, nhưng nó lại tăng tải truyền thông và mở ra các lỗ hổng bảo mật khi attacker có thể khai thác thông tin từ toàn bộ mạng lưới.
- **Giải pháp Mutual Handshake:** Mỗi client chỉ trao đổi model với những hàng xóm mà cả hai bên đều đồng thuận (đồng thời chọn nhau). Cách này giúp đảm bảo quá trình trao đổi model diễn ra hiệu quả, duy trì tính bất định cần thiết để gây khó khăn cho attacker, đồng thời giảm thiểu rủi ro bảo mật và chi phí truyền thông.

Chúng tôi quyết định chọn giải pháp **Mutual Handshake** và sử dụng **giao thức coin-flipping** để đảm bảo các client có thể hoàn toàn ngẫu nhiên kết nối. Cụ thể, giao thức coin-flipping là một giao thức mật mã cho phép hai bên cùng tạo ra một giá trị ngẫu nhiên một cách công bằng, đảm bảo rằng không bên nào có thể thao túng kết quả. Nhờ đó, việc xác định các kết nối giữa các client trở nên hoàn toàn ngẫu nhiên, từ đó làm khó attacker trong việc dự đoán hoặc can thiệp vào quá trình trao đổi model.

5.2.4.2 Đề xuất giao thức cho kỹ thuật neighbor selection

Trong quá trình **Neighbor Selection**, chúng tôi áp dụng kỹ thuật **giao thức coin-flipping** để đảm bảo rằng việc chọn hàng xóm diễn ra công bằng, không thể bị thao túng. Protocol này không yêu cầu một bên thứ ba đáng tin cậy nhưng vẫn đảm bảo rằng không client nào có thể tự ý điều chỉnh kết quả theo hướng có lợi cho mình^[26].

Để hiện thực hóa giao thức này một cách hiệu quả, chúng tôi sử dụng cơ chế **hash-based commitment** với SHA-256. Cách tiếp cận này không chỉ đảm bảo tính bảo mật mà còn giúp đơn giản hóa việc triển khai so với các phương pháp dựa trên nhóm số nguyên lớn như Pedersen

Commitment^[27] - một phương pháp cam kết mật mã (*cryptographic commitment scheme*) dựa trên bài toán logarit rời rạc, cung cấp tính chất giấu kín (*hiding*) và ràng buộc (*binding*). Cơ chế này cần mang lại hai thuộc tính quan trọng:

- **Computational Hiding:** Một client không thể đoán trước giá trị bit của đối phương trước khi cả hai tiết lộ cam kết của mình.
- **Computational Binding:** Một client không thể thay đổi giá trị của mình sau khi đã gửi cam kết ban đầu, ngăn chặn việc giả mạo kết quả.

Khi đó, chúng tôi có thể ngăn chặn các cuộc tấn công như *bias attack* (một bên thao túng kết quả có lợi cho mình) hoặc *early revelation attack* (một bên tiết lộ sớm để ép bên còn lại làm theo).

Việc lựa chọn **SHA-256** làm hàm băm cam kết mang lại nhiều lợi ích thực tiễn:

- **Hiệu quả cao:** SHA-256 không yêu cầu các phép toán số học phức tạp trên nhóm số nguyên lớn, giúp giao thức nhẹ và dễ triển khai hơn.
- **Bảo mật tốt:** SHA-256 đảm bảo tính chất **Computational Hiding** nhờ *Partial Preimage Resistance*, tức là không thể tìm ra đầu vào nếu chỉ biết giá trị băm^[28].
- **Tính toàn vẹn:** SHA-256 đảm bảo **Computational Binding**, tức là một client không thể tạo ra hai giá trị khác nhau có cùng giá trị băm, ngăn chặn gian lận^[29].

Việc kết hợp **giao thức coin-flipping** với cơ chế hash-based commitment giúp đảm bảo rằng quá trình chọn hàng xóm trong hệ thống diễn ra một cách an toàn, minh bạch và không thể bị thao túng.

Protocol được chia thành bốn giai đoạn chính:

- **Ready Phase (Giai đoạn sẵn sàng)**
 - Mỗi client gửi tín hiệu sẵn sàng đến các client hàng xóm.
 - Chờ tín hiệu sẵn sàng trong vòng **60 giây**. Nếu không nhận được tín hiệu, client đó không tham gia quá trình lựa chọn.
- **Commit Phase (Giai đoạn cam kết)**
 - Mỗi client (A và B) tạo một số ngẫu nhiên x_A, x_B trong khoảng $[0, 1)$.
 - Tạo một **nonce** 128-bit ngẫu nhiên để chống brute-force.
 - Tính toán cam kết:

$$C_A = H(x_A || \text{nonce}_A), \quad C_B = H(x_B || \text{nonce}_B)$$

5.2. Phân tích kỹ thuật tích hợp

- Trao đổi giá trị cam kết với nhau trong vòng **5 giây**.
- **Reveal Phase (Giai đoạn tiết lộ)**
 - Mỗi client gửi (x, nonce) của mình trong vòng **5 giây**.
 - Kiểm tra cam kết đối phương bằng cách so sánh với giá trị băm ban đầu nhằm tránh replay attack.
- **Verification (Xác minh và quyết định kết nối)**
 - Tính tổng:
$$S = (x_A + x_B) \mod 1$$
 - Nếu $S < p$, hai client đồng ý trao đổi và thêm client vào danh sách lựa chọn ngẫu nhiên.
 - Nếu $S \geq p$, hai client không trao đổi với nhau.

Protocol này đảm bảo rằng việc lựa chọn hàng xóm là **công bằng, bảo mật và ngẫu nhiên**, đồng thời tránh được các deadlock như trong mô hình Neighbor Selection ban đầu. Giá trị p có thể được điều chỉnh để kiểm soát xác suất kết nối giữa các client, giúp hệ thống đạt được sự cân bằng giữa **mở rộng mạng lưới** (với p lớn) và **giới hạn kết nối** (với p nhỏ).

5.2.4.3 Đánh giá và nhận xét giao thức được đề xuất

Trong giao thức này, tất cả các client trong mạng sẽ thống nhất sử dụng cùng một tham số ngưỡng p để đảm bảo tính nhất quán trong quá trình lựa chọn hàng xóm. Việc này giúp các client dễ dàng chọn nhau với xác suất p mong muốn, tránh việc nếu p quá khác nhau có thể khiến cho kỳ vọng số hàng xóm cùng tham gia trao đổi khác nhau dẫn đến ảnh hưởng tới khả năng hội tụ của mô hình.

Quá trình lựa chọn hàng xóm diễn ra như sau:

- Mỗi client A và B tạo một giá trị ngẫu nhiên x_A và x_B trong khoảng $(0, 1]$
- Tổng giá trị ngẫu nhiên của hai client được tính theo công thức:

$$S = (x_A + x_B) \mod 1$$

- Hai client sẽ chỉ đồng ý kết nối nếu:

$$S < p, \quad 0 \leq p < 1$$

Vì tất cả các client đều sử dụng chung một giá trị p , mỗi client có thể dễ dàng kiểm tra xem hàng xóm có chấp nhận kết nối hay không mà không cần phải trao đổi thêm thông tin ngoài giá trị cam kết ban đầu.

Dưới cơ chế này, xác suất để một client đưa một hàng xóm vào danh sách ngẫu nhiên của mình là:

$$p$$

Dẫn đến kỳ vọng số lượng hàng xóm được chọn trong mỗi vòng giao tiếp là:

$$E(X) = n \cdot p$$

với n là tổng số client hàng xóm.

Theo nghiên cứu của Shiqiang Wang và Mingyue Ji^[30], khi mỗi client hàng xóm tham gia vào quá trình trao đổi mô hình với **xác suất độc lập** p trong mỗi vòng huấn luyện, thì **lỗi hội tụ giới hạn trên** (sai số giữa kết quả thu được và giá trị tối ưu thực sự) của mô hình được xác định bởi:

$$\mathcal{O}\left(\frac{1 + \sigma^2}{\sqrt{SIT}}\right)$$

Trong đó:

- σ^2 : Phương sai của nhiễu gradient, biểu thị mức độ ngẫu nhiên trong quá trình cập nhật mô hình.
- S : Số lượng client tham gia trong mỗi vòng huấn luyện.
- I : Số lượng bước cập nhật nội bộ mà mỗi client thực hiện trước khi gửi mô hình lên.
- T : Tổng số vòng huấn luyện của hệ thống.

Dễ thấy rằng khi client chỉ tham gia với xác suất p , số lượng client trung bình trong mỗi vòng huấn luyện là $S \approx pN$ (với N là tổng số client). So với trường hợp tất cả client luôn tham gia ($p = 1$), số lượng client trung bình giảm một hệ số p , dẫn đến lỗi hội tụ tăng lên một hệ số $\frac{1}{\sqrt{p}}$.

Khi p tăng, số lượng client trung bình tham gia vào mỗi vòng huấn luyện cũng tăng, tức là S lớn hơn. Theo công thức trên, điều này làm tăng $\sqrt{SIT}$ ở mẫu số, dẫn đến giảm lỗi hội tụ. Do đó, nếu chọn p quá thấp sẽ khiến cho mô hình hội tụ chậm và không đạt hiệu quả tốt.

Protocol này đảm bảo rằng các client quyết định thêm nhau vào danh sách một cách công bằng, không có client nào có thể gian lận, vì giá trị ngẫu nhiên được cam kết trước khi tiết lộ.

5.2. Phân tích kỹ thuật tích hợp

Cụ thể, mỗi client sử dụng một giá trị nonce ngẫu nhiên và cam kết giá trị số ngẫu nhiên của mình dưới dạng:

$$C_A = H(x_A || \text{nonce}_A), \quad C_B = H(x_B || \text{nonce}_B)$$

Sau khi cả hai bên đã trao đổi giá trị cam kết, họ mới tiết lộ giá trị thực tế x_A, x_B . Nhờ đó, không client nào có thể thay đổi giá trị của mình sau khi đã biết giá trị của đối phương, đảm bảo tính trung thực của quá trình lựa chọn.

Trong hệ thống **Decentralized Federated Learning (DFL)** của chúng tôi, các client tham gia giao tiếp sẽ được xác thực thông qua Certificate Authority (CA), đảm bảo rằng chỉ các client hợp lệ mới có thể tham gia vào mạng. Cơ chế *neighbor selection* ngẫu nhiên trong mỗi vòng đào tạo giúp giảm nguy cơ bị tấn công phân tích cấu trúc mạng và hạn chế các cuộc tấn công trong quá trình giao tiếp giữa các client.

Một điểm quan trọng là ngay cả khi một client cố tình thao túng giá trị p (bằng cách chọn một giá trị khác với toàn bộ mạng), nó cũng sẽ không có lợi ích thực tế. Cụ thể:

- Nếu client chọn một p quá cao:
 - Nó sẽ chấp nhận kết nối với nhiều hàng xóm hơn.
 - Tuy nhiên, các client khác với giá trị p thấp hơn vẫn có thể từ chối kết nối, dẫn đến việc client này không thiết lập được số lượng liên kết như mong muốn.
 - Hơn nữa, việc có quá nhiều kết nối làm tăng khả năng bị tấn công, do attacker có thể lợi dụng để khai thác nhiều thông tin hơn.
- Nếu client chọn một p quá thấp:
 - Nó sẽ có rất ít hàng xóm để trao đổi mô hình.
 - Điều này gây bất lợi trực tiếp cho chính client đó, vì nó không thể nhận đủ dữ liệu từ các mô hình khác, làm giảm chất lượng học tập do không tận dụng được lợi ích từ quá trình tổng hợp mô hình.

Như vậy, cơ chế *neighbor selection* không chỉ đảm bảo tính bảo mật của hệ thống mà còn duy trì sự công bằng trong quá trình học liên kết. Đồng thời, do việc tham gia vào hệ thống đã được kiểm soát thông qua CA, nên việc thao túng p không mang lại lợi ích đáng kể mà thậm chí có thể gây bất lợi cho chính client thực hiện hành vi đó.

Ngoài ra, quá trình tổng hợp mô hình từ nhiều client khác nhau giúp tăng tính tổng quát và khả năng khái quát hoá của mô hình học được. Vì việc trao đổi mô hình diễn ra trong kênh được mã hóa với **TLS 1.3**, dữ liệu mô hình được bảo vệ khỏi các cuộc tấn công trung gian hoặc nghe lén. Đồng thời, sử dụng giao thức trao đổi công bằng giúp đảm bảo rằng không client nào có

thể lợi dụng hệ thống để thu thập mô hình mà không đóng góp lại. Ngược lại, nếu một client cố tình không tham gia vào quá trình trao đổi, nó sẽ không nhận được lợi ích từ việc tổng hợp mô hình như đã phân tích trước đó.

Với cơ chế này, hệ thống DFL có thể đạt được sự cân bằng giữa tính bảo mật, tính công bằng trong lựa chọn hàng xóm và chất lượng mô hình tổng hợp, giúp cải thiện hiệu suất học tập mà vẫn đảm bảo tính phân tán và phi tập trung.

5.2.4.4 Triển khai giao thức

Ready Phase Ở giai đoạn đầu tiên, tất cả các client cần đạt trạng thái sẵn sàng trước khi tiến hành trao đổi dữ liệu. Quá trình này được đồng bộ hóa bằng cách gửi thông điệp NEIGHBOR_SELECTION_READY đến tất cả các hàng xóm đang kết nối. Mỗi client sẽ kiểm tra phản hồi từ các client hàng xóm để đảm bảo rằng tất cả đều đã sẵn sàng trước khi chuyển sang giai đoạn tiếp theo. Nếu một hoặc nhiều client không phản hồi trong thời gian giới hạn, chúng sẽ bị loại khỏi vòng lặp liên kết.

```
1 async def wait_for_all_ready(self, peers, timeout=60):
2     tasks = [self.handle_neighbor_selection_ready(peer, timeout) for peer in
3               peers]
4     results = await asyncio.gather(*tasks, return_exceptions=True)
5     self.log_task_errors(results, "Ready Phase")
6
7     start_time = asyncio.get_event_loop().time()
8     end_time = start_time + timeout
9
10    while asyncio.get_event_loop().time() < end_time:
11        if all(self.cm.neighbor_selection_data.get(peer, {}).get("
12              ready_phase", {}).get("status") for peer in peers):
13            return True
14            await asyncio.sleep(0.1)
15    raise asyncio.TimeoutError("Timeout: Some nodes did not reach READY
16                               phase.")
```

Commit Phase Sau khi tất cả các client đã sẵn sàng, mỗi client sẽ thực hiện cam kết giá trị bit ngẫu nhiên của mình bằng cách tạo một giá trị băm $H(\text{bit}||\text{nonce})$ và gửi thông điệp NEIGHBOR_SELECTION_COMMIT chứa giá trị cam kết đến các hàng xóm. Trong giai đoạn này, hệ thống đảm bảo rằng không có client nào tiết lộ giá trị bit của mình trước khi hoàn thành giai đoạn commit. Nếu một client gửi commit trước khi sẵn sàng, nó sẽ bị coi là độc hại.

5.2. Phân tích kỹ thuật tích hợp

Quá trình này sử dụng thư viện `secrets`^[31] của Python để tạo số ngẫu nhiên bảo mật, đảm bảo rằng các giá trị được tạo ra có mức độ bảo mật cao, tránh được các cuộc tấn công dự đoán hoặc giả mạo.

```
1 @staticmethod
2 def hash_commitment(bit, nonce):
3     if not isinstance(bit, float):
4         raise ValueError("bit must be a floating-point number")
5     if isinstance(nonce, str):
6         nonce = nonce.encode() # Convert string nonce to bytes
7
8     bit_bytes = struct.pack("!d", bit) # Convert float to 8-byte IEEE 754
        representation
9     return hashlib.sha256(bit_bytes + nonce).digest()
10
11 async def security_commit_phase(self, peer):
12     bit = secrets.randbelow(2**52) / 2**52
13     nonce = secrets.token_bytes(16)
14     commitment = self.hash_commitment(bit, nonce)
15     self.neighbor_selection_data[peer]["self_commitment"]["bit"] = bit
16     self.neighbor_selection_data[peer]["self_commitment"]["nonce"] = nonce
17     self.neighbor_selection_data[peer]["self_commitment"]["commitment"] =
        commitment
18     return commitment
19
20 async def handle_neighbor_selection_commit(self, peer, timeout=5):
21     async with self.cm.get_neighbor_selection_lock(peer):
22         commitment = await self.cm.security_commit_phase(peer)
23         message = self.cm.create_message("security", "
            neighbor_selection_commit", 0, b'', commitment)
24         await self.cm.send_message(peer, message)
25
```

Reveal Phase Khi tất cả các client đã hoàn thành giai đoạn commit, mỗi client sẽ tiết lộ giá trị bit thực tế và nonce của mình bằng cách gửi thông điệp `NEIGHBOR_SELECTION_REVEAL` cho các hàng xóm. Các client sẽ kiểm tra xem trạng thái của các hàng xóm trước đó. Nếu một client gửi thông tin reveal trước khi thực hiện commit hoặc ready, nó sẽ bị đánh dấu là độc hại.

```
1 async def handle_neighbor_selection_reveal(self, peer, timeout=5):
2     bit = self.cm.neighbor_selection_data.get(peer, {}).get("self_commitment
        ", {}).get("bit")
3     nonce = self.cm.neighbor_selection_data.get(peer, {}).get("
        self_commitment", {}).get("nonce")
```

```

4 message = self.cm.create_message("security", "neighbor_selection_reveal"
    , bit, nonce)
5 await self.cm.send_message(peer, message)
6 ...
7 ready_status = self.cm.neighbor_selection_data.get(peer, {}).get("
    ready_phase", {}).get("status")
8 commit_status = self.cm.neighbor_selection_data.get(peer, {}).get("
    commit_phase", {}).get("status")
9 reveal_status = self.cm.neighbor_selection_data.get(peer, {}).get("
    reveal_phase", {}).get("status")
10 if not ready_status:
11     raise ValueError(f"Malicious: {peer} send
    NEIGHBOR_SELECTION_COMMIT before NEIGHBOR_SELECTION_READY.")
12 if not commit_status:
13     raise ValueError(f"Malicious: {peer} send
    NEIGHBOR_SELECTION_REVEAL before NEIGHBOR_SELECTION_COMMIT.")
14
15     raise asyncio.TimeoutError(f"Timeout: {peer} did not send
    NEIGHBOR_SELECTION_REVEAL in {timeout} seconds.")
16

```

Verify Phase Trong giai đoạn cuối cùng, mỗi client sẽ xác minh rằng hàng xóm của mình đã tuân thủ đúng quy trình cam kết và tiết lộ. Nếu giá trị băm của một hàng xóm không khớp với cam kết trước đó, client đó sẽ bị đánh dấu là độc hại và bị loại khỏi danh sách liên kết. Khi một client xác minh thành công, nó gửi thông điệp NEIGHBOR_SELECTION_VERIFY để xác nhận kết quả kiểm tra và cập nhật danh sách hàng xóm đáng tin cậy.

```

1 async def handle_neighbor_selection_verify(self, peer, timeout=5):
2     async with self.cm.get_neighbor_selection_lock(peer):
3         verified = await self.cm.security_verify_phase(peer)
4         self.cm.neighbor_selection_data[peer]["connect"] = verified
5         message = self.cm.create_message("security", "
    neighbor_selection_verify", 0.0, b'', b'', verified)
6         await self.cm.send_message(peer, message)
7
8     async def security_verify_phase(self, peer):
9         peer_bit = self.neighbor_selection_data.get(peer, {}).get("reveal_phase"
    , {}).get("bit")
10        peer_nonce = self.neighbor_selection_data.get(peer, {}).get("
    reveal_phase", {}).get("nonce")
11        peer_commitment = self.neighbor_selection_data.get(peer, {}).get("
    commit_phase", {}).get("commitment")
12

```

5.3. Phân tích tính bảo mật của hệ thống

```
13 mine_bit = self.neighbor_selection_data.get(peer, {}).get("
    self_commitment", {}).get("bit")
14 mine_nonce = self.neighbor_selection_data.get(peer, {}).get("
    self_commitment", {}).get("nonce")
15 mine_commitment = self.neighbor_selection_data.get(peer, {}).get("
    self_commitment", {}).get("commitment")
16
17 if peer_bit == mine_bit and peer_nonce == mine_nonce and peer_commitment
    == mine_commitment:
18     raise Exception("[neighbor-selection] Malicious: Peer is trying to
    do replay attack")
19 if 0 > peer_bit and peer_bit >= 1:
20     raise Exception("[neighbor-selection] Malicious: Peer is trying to
    make invalid bit")
21
22 logging.info(f"[neighbor-selection] Verify status with {peer}: peer_bit =
    {peer_bit}, peer_nonce = {peer_nonce}, peer_commitment = {
    peer_commitment}")
23 return CommunicationsManager.hash_commitment(peer_bit, peer_nonce) ==
    peer_commitment
24
```

5.3 Phân tích tính bảo mật của hệ thống

5.3.1 Hiệu quả bảo mật của giải pháp

Khi được sử dụng đồng thời, BALANCE, MTD và mã hóa TLS 1.3 không chỉ tăng cường khả năng chống chịu của hệ thống học liên kết phân tán phi tập trung (DFL) trước các cuộc tấn công phổ biến, mà còn góp phần bảo vệ dữ liệu riêng tư của người dùng. Mỗi thành phần đóng vai trò quan trọng trong việc phòng chống một nhóm các mối đe dọa cụ thể, đồng thời tạo ra một môi trường học an toàn, linh hoạt và khó bị khai thác.

- **BALANCE - chống lại các cuộc tấn công vào mô hình (Model Poisoning):**
 - **Tấn công mô hình độc hại (Malicious Model Update):** Các node độc hại gửi bản cập nhật sai lệch nhằm làm giảm độ chính xác toàn cục.
 - **Tấn công chèn nhiễu (Noise Injection):** Thêm nhiễu vào tham số mô hình để phá vỡ hội tụ.
 - **Tấn công backdoor:** Cấy mã độc vào mô hình nhằm gây hành vi sai lệch dưới điều kiện kích hoạt cụ thể.

BALANCE sử dụng cơ chế tổng hợp có trọng số thích ứng và lọc các bản cập nhật nghi ngờ, giúp duy trì độ tin cậy và ổn định hội tụ trong môi trường có sự hiện diện của các node Byzantine. Ngoài ra, BALANCE còn giảm nguy cơ rò rỉ thông tin riêng tư thông qua các bản cập nhật bất thường, hỗ trợ cơ chế bảo vệ tại chỗ trước các hành vi thao túng gradient.

- **MTD - Chống tấn công mạng và bảo vệ dữ liệu riêng tư:**

- **Tấn công cô lập (Eclipse) và lập bản đồ mạng (Network Mapping):** Trong một cuộc tấn công Eclipse, một node trung thực có thể bị bao vây bởi các node độc hại, khiến nó chỉ nhận được thông tin sai lệch và bị tách khỏi các tương tác hợp lệ trong mạng. Song song đó, kẻ tấn công có thể quan sát và phân tích lưu lượng trong thời gian dài để lập bản đồ mạng, từ đó xác định các điểm yếu và triển khai chiến thuật tấn công hiệu quả.

Giải pháp từ MTD và mã hóa:

- * Hệ thống sử dụng hạ tầng khóa công khai với **Certificate Authority (CA)** để xác thực danh tính các node, đảm bảo rằng chỉ các node hợp lệ mới được tham gia mạng. Điều này giúp ngăn chặn kẻ tấn công dễ dàng chen các node giả mạo nhằm xây dựng mạng lưới bao vây.
 - * Trong trường hợp hiếm hoi mà kẻ tấn công có thể “lách luật” và được cấp chứng chỉ hợp lệ (do lỗi vận hành hoặc rò rỉ khóa), cơ chế **MTD** vẫn đóng vai trò then chốt. Việc lựa chọn hàng xóm ngẫu nhiên ở mỗi vòng và thay đổi IP định kỳ khiến cho ngay cả khi node độc hại được tham gia hệ thống, nó cũng rất khó duy trì kết nối ổn định và lâu dài với node mục tiêu để thực hiện tấn công Eclipse hiệu quả.
 - * Ngoài ra, mỗi node duy trì một tập hàng xóm ban đầu, nhưng tại mỗi vòng chỉ chọn ngẫu nhiên n node để trao đổi thông tin (dựa trên giao thức *coin-flipping*). Điều này càng làm tăng tính khó đoán và hạn chế khả năng kiểm soát toàn bộ kết nối của một node trung thực.
- **Giảm thiểu nguy cơ tấn công khôi phục dữ liệu (DLG)^[16]:** Dù chỉ chia sẻ gradient thay vì dữ liệu gốc, việc thu thập gradient liên tục từ cùng một node vẫn có thể bị lợi dụng để khôi phục lại dữ liệu gốc thông qua các phương pháp như DLG.

Giải pháp từ MTD và mã hóa:

- * Việc thay đổi hàng xóm ngẫu nhiên tại mỗi vòng khiến gradient bị phân tán, không thể thu thập liên tục từ một nguồn duy nhất. Điều này làm giảm đáng kể tính chính xác của các kỹ thuật tái cấu trúc dữ liệu như DLG.

5.3. Phân tích tính bảo mật của hệ thống

* Thuật toán BALANCE với cơ chế lọc và gán trọng số giúp làm suy yếu tác động của từng gradient riêng lẻ trong quá trình tổng hợp, từ đó bảo vệ thông tin nhạy cảm hiệu quả hơn. Tuy nhiên, các cơ chế này chủ yếu nhằm giảm thiểu rủi ro, chưa thể loại bỏ hoàn toàn khả năng bị tấn công trong môi trường mở.

- **TLS 1.3 - bảo vệ kênh truyền và xác thực danh tính:**

- **Nghe lén (Eavesdropping):** Kẻ tấn công lắng nghe thông tin trao đổi giữa các node.
- **Tấn công người trung gian (Man-in-the-Middle - MitM):** Chèn mình vào giữa phiên giao tiếp để làm giả dữ liệu.
- **Giả mạo danh tính (Spoofing):** Mạo danh node hợp lệ để gửi mô hình sai.

TLS 1.3 cung cấp mã hóa đầu-cuối, bắt tay bảo mật và xác thực hai chiều. Điều này giúp đảm bảo tính riêng tư, toàn vẹn và xác thực cho mọi giao tiếp, đồng thời ngăn chặn việc gradient hoặc mô hình bị thu thập trong quá trình truyền dẫn. Nhờ vậy, TLS 1.3 đóng vai trò quan trọng trong việc ngăn chặn các cuộc tấn công vào dữ liệu trong quá trình truyền tải (data in transit).

Ngoài ra, hệ thống còn sử dụng **Certificate Authority (CA)** để cấp và xác minh chứng chỉ số cho các node. Mỗi node khi tham gia hệ thống đều cần có chứng chỉ được CA ký, chứa thông tin định danh và khóa công khai của node đó. Khi hai node thiết lập kết nối TLS, chúng sẽ kiểm tra chứng chỉ của nhau. Nếu chứng chỉ không hợp lệ hoặc không do CA tin cậy phát hành, kết nối sẽ bị từ chối. Cơ chế này giúp ngăn chặn các node giả mạo hoặc không đáng tin, đảm bảo chỉ các thực thể hợp lệ mới được phép giao tiếp trong hệ thống.

Tổng thể, giải pháp kết hợp BALANCE, MTD và TLS 1.3 giúp xây dựng một môi trường học liên kết có khả năng chủ động trong việc bảo vệ mô hình tổng hợp - yếu tố then chốt trong các hệ thống phi tập trung thực tế.

5.3.2 Phân tích rủi ro và hạn chế trong bảo mật

Mặc dù hệ thống đề xuất kết hợp BALANCE, MTD và TLS 1.3 mang lại khả năng bảo vệ mạnh mẽ trước nhiều hình thức tấn công, một số rủi ro và giới hạn bảo mật vẫn tồn tại trong các điều kiện triển khai thực tế và trước các đối thủ có năng lực cao. Dưới đây là các nguy cơ đáng lưu ý:

- **Tấn công khôi phục dữ liệu (Gradient Leakage - DLG)^[16]:**

5.3. Phân tích tính bảo mật của hệ thống

- Trong quá trình huấn luyện liên kết, các node trung thực gửi bản cập nhật gradient để cải thiện mô hình chung. Tuy nhiên, các gradient này có thể vô tình tiết lộ thông tin về dữ liệu gốc, đặc biệt khi tập dữ liệu huấn luyện nhỏ hoặc có tính đặc trưng cao.
- Kẻ tấn công (một node độc hại) có thể thu thập liên tục các gradient được gửi từ một node trung thực, sau đó áp dụng kỹ thuật DLG để tái tạo lại dữ liệu đầu vào ban đầu thông qua quá trình tối ưu ngược.
- Nguy cơ này tăng cao khi cấu trúc mạng thiếu linh hoạt — ví dụ, các kết nối cố định hoặc IP không thay đổi thường xuyên khiến cùng một cặp node liên tục giao tiếp qua nhiều vòng huấn luyện.
- Do đó, để giảm thiểu rủi ro, cần kết hợp các kỹ thuật như Differential Privacy cũng như thay đổi topology động một cách chiến lược nhằm hạn chế số lần một node cụ thể bị quan sát lặp lại.

- **Tấn công suy luận thành viên (Membership Inference Attack)^[17]:**

- Mặc dù dữ liệu huấn luyện không được chia sẻ trực tiếp trong hệ thống học liên kết phi tập trung (DFL), các mô hình được cập nhật vẫn có thể tiết lộ thông tin về dữ liệu huấn luyện thông qua hành vi phản hồi đầu ra.
- Kẻ tấn công (có thể là node độc hại trong mạng) có thể sử dụng kỹ thuật *shadow training* để huấn luyện các mô hình tương tự, từ đó xây dựng mô hình tấn công có khả năng phân biệt giữa mẫu dữ liệu đã từng xuất hiện trong quá trình huấn luyện và mẫu chưa từng xuất hiện.
- Đặc biệt trong hệ thống DFL với topology kết nối đầy đủ và chia sẻ tham số nhiều vòng, kẻ tấn công có thể thu thập các bản cập nhật mô hình trong nhiều thời điểm, gia tăng độ chính xác của suy luận thành viên.
- Khi dữ liệu huấn luyện phân tán tại các node với quy mô nhỏ, nguy cơ quá khớp (overfitting) tăng cao, khiến mô hình trở nên nhạy cảm hơn với các cuộc tấn công suy luận.
- Nếu không áp dụng các biện pháp giảm thiểu như làm mờ đầu ra mô hình, sử dụng differential privacy hoặc giới hạn thông tin phản hồi, hệ thống rất dễ bị lộ thông tin dữ liệu người dùng.

- **Giả mạo danh tính hoặc chiếm quyền điều khiển node:**

- Nếu chứng chỉ TLS bị đánh cắp hoặc phân phối khóa không an toàn, kẻ tấn công có thể mạo danh node hợp lệ để gửi mô hình sai lệch hoặc đánh cắp thông tin.

5.3. Phân tích tính bảo mật của hệ thống

- Trường hợp node không có phần cứng bảo mật (như TPM), khóa riêng có thể bị chiếm đoạt thông qua tấn công vào thiết bị đầu cuối hoặc phần mềm độc hại.
- **Tấn công DoS hoặc làm chậm tiến trình huấn luyện:**
 - Các node độc hại có thể không gửi mô hình, gửi trễ, hoặc gửi các bản cập nhật trống, làm tiêu tốn tài nguyên của node trung thực và giảm tốc độ hội tụ.
 - Khi số lượng node độc hại lớn hoặc chiếm tỷ lệ đáng kể, hệ thống dễ rơi vào tình trạng đình trệ hoặc phân kỳ.

Tổng kết: Dù hệ thống đã có cơ chế bảo vệ chủ động, các hình thức tấn công tinh vi – đặc biệt là các cuộc tấn công phối hợp, dài hạn hoặc khai thác vào cấu trúc mạng – vẫn có thể gây ảnh hưởng tiêu cực đến cả hiệu quả mô hình và tính riêng tư. Việc kết hợp các chiến lược bảo mật đa lớp, giám sát hành vi mạng, kiểm định thống kê liên tục, cùng khả năng phản ứng linh hoạt là cần thiết để nâng cao khả năng phòng thủ trong bối cảnh môi trường ngày càng phức tạp.

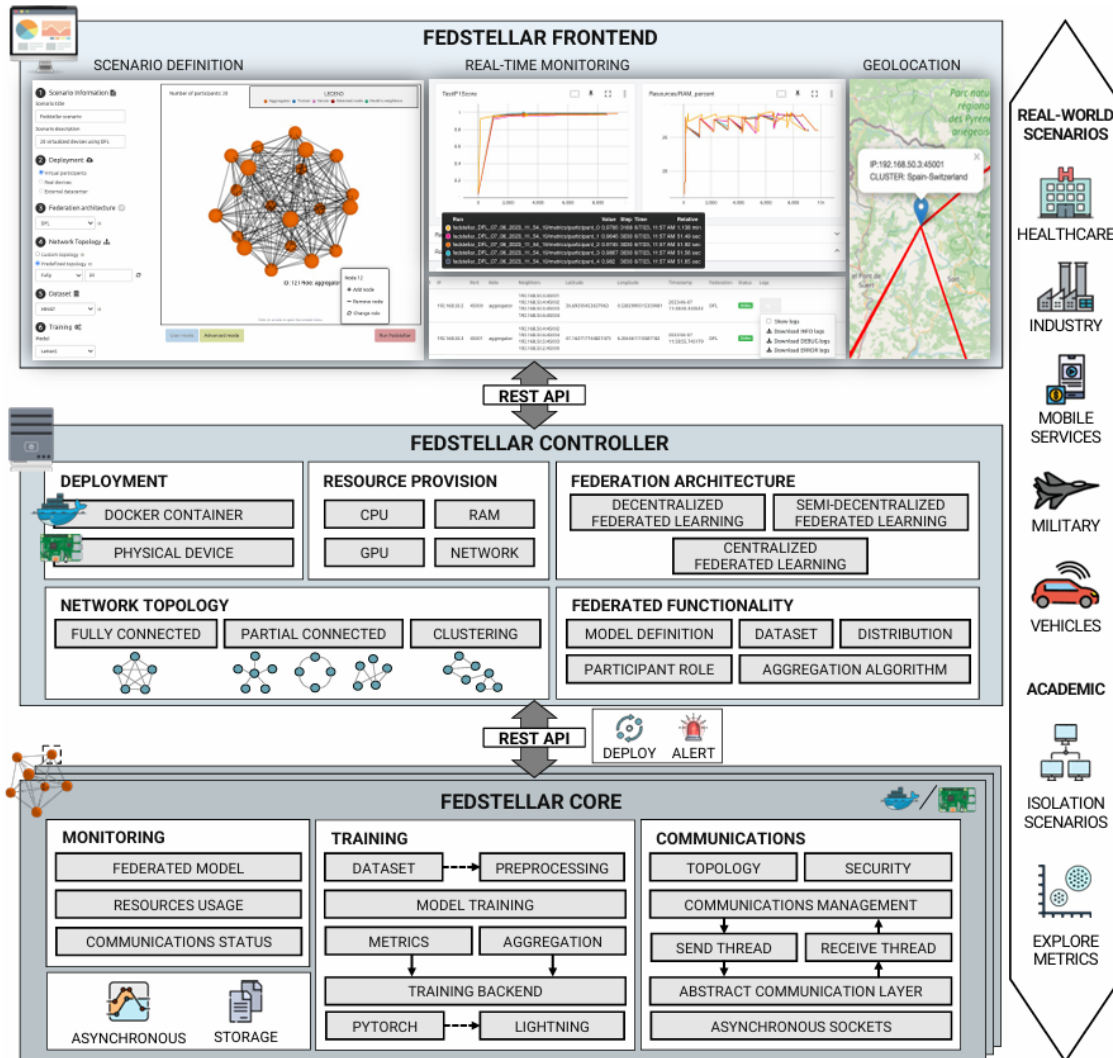
Chương 6

Đánh giá thử nghiệm

Phần này mô tả thiết lập thử nghiệm được sử dụng để đánh giá hiệu năng và tính bảo mật của giải pháp đề xuất. Các thí nghiệm phân tích một cách có hệ thống các yếu tố khác nhau, bao gồm bộ dữ liệu, cấu trúc liên kết mạng, giải thuật tổng hợp, các cuộc tấn công.

6.1. Giới thiệu nền tảng Nebula (Fedstellar)

6.1 Giới thiệu nền tảng Nebula (Fedstellar)



Hình 6.1.1: Tổng quan về kiến trúc nền tảng Nebula (Fedstellar)^[7]

Nebula là nền tảng kế thừa và mở rộng từ Fedstellar, một hệ thống tiên tiến dành cho học liên kết phi tập trung (Federated Learning - FL). Nebula, được phát triển dựa trên thư viện p2pfl, đã đóng vai trò quan trọng trong việc quản lý và triển khai FL trên cả thiết bị vật lý và ảo hóa.

Nebula mang đến nhiều tính năng quan trọng như:

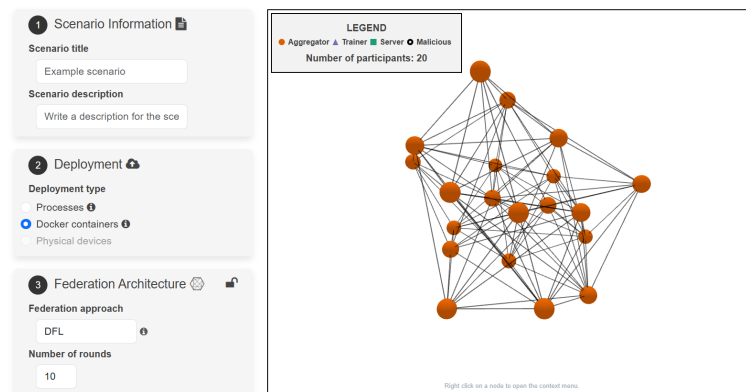
- Tạo và quản lý hệ thống Học liên kết, hỗ trợ đa dạng thiết bị với quy trình đơn giản hơn so với các giải pháp trước đây.
- Xây dựng và triển khai các mô hình liên kết phức tạp, bao gồm DFL, SDFL và CFL, đảm bảo khả năng giao tiếp giữa các nút mạng và tích hợp quản lý vai trò, tối ưu hóa quy trình.

6.1. Giới thiệu nền tảng Nebula (Fedstellar)

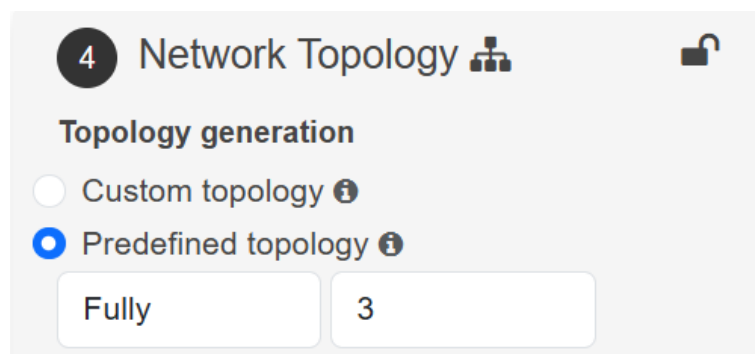
- Hỗ trợ nhiều mô hình ML/DL và tập dữ liệu khác nhau, giúp giải quyết các bài toán FL phức tạp.
- Giao diện thân thiện, hỗ trợ tùy chỉnh linh hoạt và giám sát quá trình đào tạo hiệu quả.
- Theo dõi KPI toàn diện, từ tài nguyên tính toán, hiệu suất truyền thông đến chất lượng mô hình FL.

6.1.1 Triển khai thí nghiệm FL

Dưới đây là giao diện và các tính năng chính Nebula hỗ trợ cho việc khởi tạo và triển khai một thí nghiệm Học liên kết.

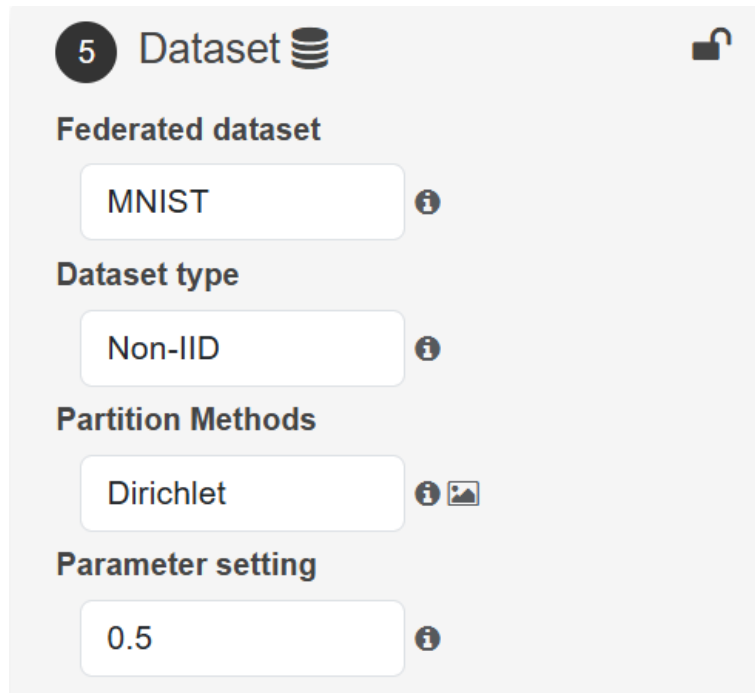


Người dùng có thể đặt tên và mô tả cho mỗi thí nghiệm. Nebula hỗ trợ hai kiểu triển khai: chạy trên process hoặc chạy trên docker. Đối với kiến trúc liên kết, Nebula cho phép lựa chọn giữa ba mô hình bao gồm DFL, SDFL và CFL.



Người dùng có thể lựa chọn thiết lập thủ công cấu trúc mạng của hệ thống hoặc lựa chọn các cấu trúc mạng đặc trưng ví dụ như cấu trúc mạng đầy đủ, cấu trúc sao, cấu trúc vòng,... với số node tùy ý.

6.1. Giới thiệu nền tảng Nebula (Fedstellar)



5 Dataset

Federated dataset

MNIST

Dataset type

Non-IID

Partition Methods

Dirichlet

Parameter setting

0.5

Người dùng có thể sử dụng các tập dữ liệu có sẵn phổ biến trong học máy như MNIST, FashionMNIST, EMNIST, CIFAR10 và CIFAR100. Ngoài ra, Nebula còn hỗ trợ quá trình xử lý và phân vùng dữ liệu. Một số lựa chọn phân vùng dữ liệu bao gồm:

1. Phân vùng IID:

- `balanced_iid_partition`: Tạo các phân vùng với phân phối lớp bằng nhau.
- `unbalanced_iid_partition`: Tạo các phân vùng với kích thước mẫu khác nhau nhưng phân phối lớp tương tự.
- `homo_partition`: Phân vùng đồng nhất đơn giản, chia ngẫu nhiên dữ liệu thành các phân vùng có kích thước bằng nhau.

2. Phân vùng Non-IID:

- `dirichlet_partition`: Tạo các phân vùng sử dụng phân phối Dirichlet, kiểm soát sự mất cân bằng lớp với tham số alpha.
- `percentage_partition`: Tạo các phân vùng dựa trên tỷ lệ phần trăm lớp được chỉ định cho mỗi phân vùng.
- `postprocess_partition`: Đảm bảo mỗi phân vùng có số lượng mẫu tối thiểu cho mỗi lớp.

The screenshot shows two configuration panels from the Nebula interface. Panel 6, titled 'Training' with a gear icon, has a 'Model' dropdown menu set to 'MLP'. Panel 7, titled 'Aggregation' with a calculator icon, has an 'Aggregation algorithm' dropdown menu set to 'FedAvg'. Both panels have an unlock icon in the top right corner.

Nebula hỗ trợ hai loại mô hình huấn luyện là MLP và CNN. Ngoài ra, người dùng có thể sử dụng các giải thuật tổng hợp cơ bản như FedAvg, Krum, Trim-mean,...

The screenshot shows the 'Security' configuration panel, which is titled 'Security' with a cloud and lock icon. Under 'Security type', there are two options: 'Encryption' (unchecked) and 'MTD' (checked with a blue checkmark). Below this, there is an 'Acceptance Random Probability' input field with the value '0.5'.

Ở hệ thống của chúng tôi, các lựa chọn về cấu hình bảo mật như mã hoá và MTD được hiện thực riêng và thêm vào.

6.2 Thiết lập thử nghiệm

6.2.1 Tập dữ liệu và mô hình

Chúng tôi lựa chọn sử dụng bộ dữ liệu **MNIST**^[32] để tiến hành thí nghiệm. Đây là một tiêu chuẩn được sử dụng rộng rãi để phân loại chữ số viết tay trong nghiên cứu về Học liên kết (FL) và Thị giác máy tính (CV). Bộ dữ liệu bao gồm 10 lớp, trong đó mỗi mẫu là một hình ảnh thang độ xám 28×28. Bộ dữ liệu này chứa 60.000 mẫu huấn luyện và 10.000 mẫu kiểm tra.

6.2. Thiết lập thử nghiệm

Tập dữ liệu được phân vùng theo kiểu Non-IID sử dụng phân phối Dirichlet với tham số $\alpha = 0.5$. Đây là một phân phối phổ biến trong các nghiên cứu Học liên kết với các đặc điểm sau:

1. **Mức độ không đồng nhất vừa phải:** Tạo ra sự không cân bằng trong phân phối lớp giữa các nút tham gia, nhưng không quá cực đoan.
2. **Mô phỏng thực tế:** Phản ánh khá tốt tình huống thực tế khi dữ liệu phân tán tự nhiên không tuân theo phân phối đồng nhất.
3. **Thách thức học tập:** Tạo ra thách thức đủ lớn cho mô hình học liên kết:
 - Mỗi nút sẽ có phân phối lớp khác nhau
 - Một số nút có thể có rất ít hoặc không có mẫu của một số lớp nhất định
 - Một số nút có thể có phân bố lớp bị lệch đáng kể

6.2.2 Cấu trúc mạng

Cấu trúc mạng định nghĩa sự giao tiếp giữa các nút trong một hệ thống DFL và ảnh hưởng đến việc tổng hợp các mô hình. Các mức độ kết nối khác nhau tác động đến hiệu quả của việc cập nhật mô hình và mức tiêu thụ năng lượng tổng thể của hệ thống. Để phân tích tác động của nó, các cấu trúc mạng sau đây được xem xét:

- **Kết nối hoàn toàn (Dense):** Mỗi nút được kết nối với tất cả các nút khác, cung cấp đồng bộ hóa giao tiếp cao nhất.
- **Đồ thị ngẫu nhiên Erdős-Rényi (ER) ($p = 0.5$):** Mạng lưới được kết nối ngẫu nhiên với xác suất $p = 0.5$, đại diện cho một mạng có mật độ vừa phải.
- **Cấu trúc vòng (Sparse):** Các nút được sắp xếp theo hình tròn, với mỗi nút chỉ được kết nối với các nút lân cận ngay bên cạnh.

6.2.3 Cơ chế tổng hợp

Chúng tôi đánh giá hiệu quả của phương pháp **BALANCE** được đề xuất bằng cách so sánh nó với các giải thuật tổng hợp mô hình: **FedAvg**, **Krum**, **Trimmed Mean** và **Median**. Nội dung chi tiết các giải thuật đã được trình bày ở mục 2.4.

6.2.4 Các cuộc tấn công

Các cuộc tấn công trong thí nghiệm sẽ sử dụng tỉ lệ malicious là 20%, riêng với tấn công Trim-Mean, tỉ lệ 60% sẽ được áp dụng.

Trong hiện thực của chúng tôi, Malicious Node tấn công hệ thống bằng cách phá hoại kết quả tổng hợp (aggregation) của mô hình, thay vì gửi model xấu ngay từ đầu.

Quy trình chi tiết:

- **Khởi động round:** Ngay đầu mỗi vòng học (round), Malicious Node cài đặt tấn công bằng cách override hàm `run_aggregation` của aggregator, chuẩn bị cho việc can thiệp kết quả aggregation sau này.
- **Train và gửi model sạch:** Malicious Node thực hiện test/train mô hình local bình thường. Sau đó, nó gửi mô hình local (chưa bị poison) cho các neighbor thông qua quá trình propagate.
- **Nhận models từ neighbors:** Malicious Node thu thập các mô hình mà neighbor gửi về, lưu vào bộ đệm aggregation.
- **Tấn công tại bước aggregation:** Khi đủ mô hình, hệ thống thực hiện aggregation. Ngay tại bước này, Malicious Node can thiệp và chỉnh sửa kết quả aggregation bằng hàm `model_attack()`, tạo ra mô hình poisoned.
- **Cập nhật mô hình poisoned:** Sau khi sửa, mô hình bị poison được cập nhật vào node và trở thành global model cho các vòng học tiếp theo.

Một số model attack:

Algorithm 8 Tấn công Noise Injection

Require: *received_weights* (tập trọng số mô hình đã tổng hợp), *strength* (độ mạnh của nhiễu)

Ensure: Trả về trọng số mô hình đã bị thêm nhiễu

```

1: for mỗi lớp  $k$  trong received_weights do
2:    $original\_weights \leftarrow received\_weights[k]$ 
3:    $noise \leftarrow$  tensor ngẫu nhiên từ phân phối  $\mathcal{N}(0,1)$  với cùng kích thước với original_weights
4:    $scaled\_noise \leftarrow noise \times strength$ 
5:    $received\_weights[k] \leftarrow original\_weights + scaled\_noise$ 
6: end for
7: return received_weights

```

6.2. Thiết lập thử nghiệm

Algorithm 9 Tần công Swapping Weights

Require: *received_weights* (tập trọng số mô hình đã tổng hợp), *layer_idx* (chỉ số lớp cần tần công)

Ensure: Trả về trọng số mô hình đã bị hoán đổi

```
1: Lấy trọng số của lớp thứ layer_idx:  $wm \leftarrow received\_weights[layer\_idx]$ 
2: Khởi tạo ma trận độ tương đồng sm kích thước  $(n, n)$  với  $n$  là số dòng của wm
3: for mỗi dòng  $j$  trong wm do
4:   Tính  $sm[j]$  là độ tương đồng cosine giữa  $wm[j]$  và toàn bộ wm
5: end for
6: Khởi tạo danh sách hoán vị nsort với giá trị ban đầu là  $-1$ 
7: Khởi tạo danh sách rows  $\leftarrow \emptyset$ 
8: for mỗi dòng  $j$  trong sm do
9:    $k \leftarrow$  chỉ số độ tương đồng nhỏ nhất trong  $sm[j]$ 
10:  if  $k$  là chỉ số độ tương đồng nhỏ nhất trong  $sm[:, k]$  then
11:     $nsort[j] \leftarrow k$ 
12:    Thêm  $j$  vào rows
13:  end if
14: end for
15: Xác định các dòng chưa được ghép:  $not\_rows \leftarrow$  các chỉ số chưa nằm trong rows
16: Sinh hoán vị ngẫu nhiên nrs từ not_rows
17: while tồn tại chỉ số trùng giữa nrs và not_rows do
18:   Sinh lại hoán vị ngẫu nhiên nrs
19: end while
20: Gán nrs vào  $nsort[not\_rows]$ 
21: Áp dụng hoán vị nsort cho các trọng số:
22:   Hoán đổi hàng của  $received\_weights[layer\_idx]$ 
23:   Hoán đổi hàng của  $received\_weights[layer\_idx + 1]$ 
24: if layer tiếp theo tồn tại then
25:   Hoán đổi cột của  $received\_weights[layer\_idx + 2]$  theo nsort
26: end if
27: return received_weights
```

Algorithm 10 Tấn công Model Poison (Gaussian Noise)

Require: *received_weights* (tập trọng số mô hình đã tổng hợp), *poisoned_ratio* (tỉ lệ nhiễu)**Ensure:** Trả về trọng số mô hình đã được thêm nhiễu Gaussian

```
1: Khởi tạo poisoned_model  $\leftarrow$  một OrderedDict rỗng
2: for mỗi lớp layer trong received_weights do
3:   t  $\leftarrow$  bản sao của trọng số layer
4:   if t là tensor 0 chiều then
5:     Biến đổi t thành tensor 1 chiều
6:   end if
7:   Sinh nhiễu Gaussian với trung bình 0 và phương sai poisoned_ratio
8:   poisoned  $\leftarrow t +$  nhiễu Gaussian
9:   if ban đầu là tensor 0 chiều then
10:    Chuyển poisoned về dạng scalar
11:   end if
12:   Thêm poisoned vào poisoned_model
13: end for
14: return poisoned_model
```

6.2. Thiết lập thử nghiệm

Algorithm 11 Tấn công Trimmed Mean

Require: *local_weights* (trọng số local), *benign_list* (các mô hình benign), *history* (lịch sử mô hình toàn cục)

Ensure: Trả về trọng số đã bị tấn công

```
1: if không có benign_list hoặc lịch sử history ít hơn 2 bước then
2:   return local_weights
3: end if
4: Lấy hai mô hình global trước đó: prev_g, curr_g
5: Tính direction cho mỗi tham số:  $direction[k] = sign(curr\_g[k] - prev\_g[k])$ 
6: for mỗi tham số name trong local_weights do
7:   if không có direction[name] then
8:     Giữ nguyên param
9:   else
10:    Stack tất cả bw[name] từ benign_list thành tensor stacked
11:    Tính  $wmax = \max(stack)$ ,  $wmin = \min(stack)$ 
12:    Xác định khoảng mẫu:
    • Nếu hướng là giảm ( $-1$ ): lấy ngẫu nhiên trong  $[wmax, b \times wmax]$ 
    • Nếu hướng là tăng ( $+1$ ): lấy ngẫu nhiên trong  $[wmin/b, wmin]$ 
    • Nếu không xác định hướng: giữ nguyên
13:    Sinh ngẫu nhiên trọng số atk trong khoảng đã xác định
14:    Gán attacked[name] = atk
15:   end if
16: end for
17: return attacked
```

Algorithm 12 Tấn công Krum Poisoning Attack

Require: *received_weights* (trọng số mô hình nhận được)

Ensure: Trả về trọng số mô hình đã bị tấn công hoặc giữ nguyên

```
1: Tăng biến đếm vòng hiện tại:  $current\_round \leftarrow current\_round + 1$ 
2: if current_round không nằm trong khoảng  $[round\_start, round\_stop]$  then
3:   Gán last_model  $\leftarrow received\_weights$ 
4:   return received_weights
5: end if
6: Lấy danh sách trọng số từ các node benign: benign_list  $\leftarrow get\_benign\_weights()$ 
7: if benign_list rỗng then
8:   Gán last_model  $\leftarrow received\_weights$ 
9:   return received_weights
10: end if
11: Tính toán mô hình tấn công dựa trên benign_list (chi tiết phần attack cụ thể sẽ được bổ sung sau)
12: return attacked
```

6.2.5 Thang đo đánh giá

Hiệu suất mô hình FL

- **Time:** Đánh giá thời gian hoàn tất một đợt huấn luyện. Các đợt huấn luyện trong thí nghiệm đều sẽ trải qua 100 vòng.
- **F1 Score:** Đánh giá độ chính xác tổng thể của mô hình FL, đặc biệt trong các bài toán mất cân bằng dữ liệu. Giá trị này được xác định dựa trên F1 score thấp nhất trong các node lành tính. Tập dữ liệu được sử dụng để kiểm tra là một tập dữ liệu toàn cục (global dataset) nhằm đánh giá trực quan hiệu quả của quá trình Học liên kết.

Hiệu suất tài nguyên hệ thống

- **Tỷ lệ sử dụng CPU và RAM:** Theo dõi mức tiêu thụ tài nguyên trên từng thiết bị tham gia, giúp tối ưu hóa việc phân bổ tài nguyên trong hệ thống. Các giá trị này sẽ ghi nhận mức sử dụng tối đa trong suốt quá trình huấn luyện.

Hiệu suất truyền thông

- **Lưu lượng mạng (Network Traffic) tính bằng MB hoặc số packets:** Đo lường lượng dữ liệu được trao đổi giữa các nút trong quá trình huấn luyện.

6.2.6 Triển khai hệ thống mô phỏng trên AWS EC2

Để đảm bảo việc thử nghiệm hệ thống trong môi trường có cấu hình phần cứng mạnh mẽ, đồng thời hỗ trợ khả năng mô phỏng số lượng lớn các nút trong mạng, chúng tôi sử dụng một máy chủ ảo Amazon EC2 (Elastic Compute Cloud).

Hệ thống được xây dựng trên nền tảng **Nebula**, trong đó toàn bộ các thành phần dịch vụ và node được mô phỏng dưới dạng các container Docker. Điều này giúp quá trình triển khai, vận hành và quản lý hệ thống trở nên đơn giản và đồng nhất. Các lý do chính cho việc lựa chọn EC2 bao gồm:

- **Cấu hình phần cứng cao:** Instance EC2 cung cấp tài nguyên tính toán lớn (CPU, RAM) đáp ứng yêu cầu của hệ thống khi chạy nhiều container đồng thời.
- **Hỗ trợ triển khai nhanh:** Nền tảng Nebula kết hợp với Docker cho phép dễ dàng khởi tạo và quản lý các service mô phỏng.
- **Đơn giản hoá hạ tầng:** Việc gom tất cả node mô phỏng trên một máy chủ duy nhất giúp tiết kiệm chi phí, giảm thiểu độ phức tạp trong việc thiết lập mạng.

6.3. Kết quả thử nghiệm

Cấu hình triển khai cụ thể:

- **Số lượng instance:** 1 máy ảo EC2.
- **Loại máy:** c6a.8xlarge.
- **Hệ điều hành:** Ubuntu 22.04 LTS.
- **Mạng:** Các container giao tiếp với nhau thông qua mạng ảo nội bộ do Docker cung cấp.

Quy trình thiết lập:

1. Khởi tạo máy EC2 với cấu hình phù hợp.
2. Cài đặt Nebula cùng các công cụ hỗ trợ (bao gồm Docker và Docker Compose).
3. Thiết lập nền tảng Nebula theo hướng dẫn.
4. Khởi động hệ thống và tiến hành thực nghiệm mô phỏng.

6.3 Kết quả thử nghiệm

6.3.1 Đánh giá hiệu quả giải thuật tổng hợp BALANCE

Đầu tiên, chúng tôi chứng minh tính hiệu quả của giải thuật tổng hợp BALANCE bằng cách so sánh nó với các giải thuật tổng hợp thường dùng dưới các cuộc tấn công khác nhau. Mỗi hàng ứng với mỗi giải thuật tổng hợp, cột ứng với các kiểu tấn công. Cột "No" thể hiện trường hợp hệ thống không bị tấn công (tất cả các client đều tốt).

Method	No	Gauss	Noise Injection	Swapping	Trim-Mean (*)
FedAvg	0.9588	0.4327	0.0214	0.6238	0.0462
Krum	0.8389	0.7905	0.5309	0.8867	0.7714
Trim-Mean	0.9598	0.3006	0.0210	0.6668	0.0824
Median	0.9597	0.9548	0.9572	0.9644	0.0583
Balance	0.9315	0.9474	0.9156	0.9292	0.8591

Bảng 6.3.1: So sánh **F1 Score** giữa các phương pháp tổng hợp

Khi không có tấn công (cột No), hầu hết các luật tổng hợp bao gồm FedAvg (0.9588), Trim-Mean (0.9598), Median (0.9597), và Balance (0.9315) đều đạt F1 score rất cao, vượt qua ngưỡng 0.93. Điều này cho thấy trong môi trường lý tưởng, các phương pháp này có khả năng học tốt và ổn định. Tuy nhiên, Krum chỉ đạt 0.8389 – thấp nhất trong số các phương pháp – phản

6.3. Kết quả thử nghiệm

ánh rằng thuật toán này hy sinh hiệu suất trong điều kiện sạch để đổi lấy tính chống chịu trong môi trường độc hại. Nhìn chung, luật tổng hợp Balance giữ được độ chính xác tương đương với FedAvg trong môi trường không có nhiễu – điều này rất quan trọng vì nó cho thấy tính "bền vững trong điều kiện lý tưởng" của giải thuật.

Dưới tấn công Gauss, FedAvg và Trim-Mean sụt giảm mạnh F1 score xuống lần lượt 0.4327 và 0.3006, cho thấy khả năng dễ tổn thương dưới kiểu tấn công này. Krum cũng tương tự cho thấy hiệu suất kém hơn so với điều kiện lý tưởng. Trong khi đó, Median (0.9548) và Balance (0.9474) giữ được hiệu suất cực kỳ cao – gần như không bị ảnh hưởng.

Dưới tấn công Noise Injection, FedAvg (0.0214) và Trim-Mean (0.0210) hầu như hoàn toàn mất khả năng học – F1 gần bằng 0. Điều này chỉ ra sự cực kỳ nhạy cảm của các phương pháp này với loại tấn công nhiễu trực tiếp. Krum (0.5309) giữ được mức trung bình, trong khi Median (0.9572) và Balance (0.9156) tiếp tục thể hiện khả năng chống chịu rất cao.

Dưới tấn công Swapping-Weight, Median (0.9644) tiếp tục dẫn đầu với hiệu suất rất cao, gần như không suy giảm. Krum (0.8867) và Balance (0.9292) cũng duy trì hiệu suất tốt. Trong khi đó, FedAvg (0.6238) và Trim-Mean (0.6668) lại tiếp tục giảm mạnh – cho thấy đây cũng là một điểm yếu của các phương pháp tổng hợp thuần túy theo trung bình.

(*) Riêng đối với tấn công Trim-Mean, để đảm bảo độ hiệu quả của giải thuật tấn công trong môi trường DFL và thể hiện rõ hơn khả năng bảo vệ của các luật tổng hợp, tỉ lệ khách hàng độc hại được điều chỉnh thành 60% cho các thí nghiệm này. Lúc đó, Median không còn đảm bảo được khả năng bảo vệ mô hình như các trường hợp trước khi F1 score chỉ đạt 0.0583 - gần như thất bại hoàn toàn. FedAvg và Trim-Mean cũng cho thấy kết quả tương tự. Krum giữ được hiệu suất trung bình (0.7714). Đối với luật tổng hợp BALANCE, mặc dù F1 score giảm đi so với hai trường hợp trước (0.8591), đây vẫn là một mức hiệu suất chấp nhận được và thể hiện khả năng bảo vệ tốt nhất trong tất cả luật tổng hợp được so sánh dưới tấn công phức tạp như Trim-Mean.

Method	No	Gauss	Swapping	Noise Injection	Trim-Mean
FedAvg	17.82	17.76	17.68	17.77	17.49
Krum	17.47	17.33	17.23	17.20	17.08
Trim-Mean	16.96	17.61	17.53	17.26	17.15
Median	17.07	17.29	17.43	17.04	17.42
Balance	17.49	17.35	17.80	17.59	17.50

Bảng 6.3.2: Bảng so sánh **thời gian huấn luyện (phút)** giữa các phương pháp tổng hợp

Thời gian huấn luyện trung bình của hệ thống được chọn rơi vào khoảng 17-18 phút và không có chênh lệch đáng kể khi so sánh giữa các luật tổng hợp khác nhau.

6.3. Kết quả thử nghiệm

Phương pháp	No	Gauss	Swapping	Noise Injection	Trim-Mean
FedAvg	77.4	77	73.2	77.3	77.2
Krum	75.6	78.7	78.3	77.2	76.1
Trim-Mean	70.7	75	76.7	76.4	76.5
Median	78.7	78.7	77.3	76.7	77
Balance	75.8	77.6	70	76.8	78.8

Bảng 6.3.3: Bảng so sánh **mức sử dụng CPU tối đa (%)** giữa các phương pháp tổng hợp

Mức sử dụng CPU tối đa của hệ thống đạt khoảng 70-80% trong tất cả trường hợp.

Phương pháp	No	Gauss	Swapping	Noise Injection	Trim-Mean
FedAvg	37.5	38.7	44.7	42.4	38.7
Krum	38.7	38.1	45.6	41.9	38.1
Trim-Mean	38.7	38.7	43.2	42.9	38.7
Median	39.2	39.6	45.0	43.0	39.6
Balance	38.6	37.7	45.4	42.4	37.7

Bảng 6.3.4: Bảng so sánh **mức sử dụng RAM tối đa (%)** giữa các phương pháp tổng hợp

Mức sử dụng RAM tối đa đạt khoảng 37-39% dưới điều kiện không có tấn công, tấn công Gauss, và Trim-Mean. Đối với tấn công Swapping và Noise Injection, các tác vụ thêm nhiễu của node độc hại vô hình chung ảnh hưởng tới mức sử dụng RAM của hệ thống, tăng giá trị trung bình lên khoảng 41-45%.

Phương pháp	No	Gauss	Swapping	Noise Injection	Trim-Mean
FedAvg	1,263.679	1,067.184	1,239.915	1,342.546	1,280.784
Krum	1,247.524	1,066.274	1,346.147	1,082.092	1,357.285
Trim-Mean	1,086.104	1,310.227	1,253.572	1,077.399	1,354.465
Median	1,168.3521	1,235.168	1,336.083	1,167.274	1,269.333
Balance	1,165.3101	1,061.338	1,167.607	1,167.947	1,361.63

Bảng 6.3.5: Bảng so sánh **số lượng bytes gửi/nhận (MB)** giữa các phương pháp tổng hợp

6.3. Kết quả thử nghiệm

Phương pháp	No	Gauss	Swapping	Noise Injection	Trim-Mean
FedAvg	764,708	678,972	653,489	776,847	723,569
Krum	636,005	701,418	745,851	665,461	753,405
Trim-Mean	712,213	604,680	682,423	688,747	679,605
Median	675,869	708,636	672,648	676,127	722,153
Balance	671,706	656,630	659,033	649,620	776,008

Bảng 6.3.6: Bảng so sánh **số lượng gói tin gửi/nhận (MB)** giữa các phương pháp tổng hợp

Các thông số liên quan đến lưu lượng mạng cho thấy BALANCE đảm bảo chi phí truyền thông ổn định, thậm chí là tiết kiệm hơn so với một số luật tổng hợp cơ bản như FedAvg. Điều này đến từ khả năng lọc và loại bỏ trao đổi mô hình với các khách hàng độc hại, giảm bớt chi phí giao tiếp không cần thiết.

Tổng kết

Các thí nghiệm trên cho thấy BALANCE là một luật tổng hợp có tính linh hoạt cao, không chỉ duy trì hiệu suất trong môi trường lý tưởng mà còn có khả năng chống chịu tốt trước các cuộc tấn công phức tạp. Đây là một lựa chọn đáng cân nhắc cho các hệ thống học liên kết phi tập trung, nơi tính bảo mật và độ chính xác đóng vai trò quan trọng.

6.3.2 Đánh giá hiệu suất mô hình, chi phí tài nguyên khi kết hợp Mã hoá và MTD

Phần này chúng tôi đánh giá ảnh hưởng của các cấu hình bảo mật lên hiệu suất hệ thống, tập trung vào độ chính xác mô hình (F1 score), thời gian hội tụ, chi phí tài nguyên như mức sử dụng RAM, CPU hay lưu lượng mạng.

FedAvg + No

Configuration	F1 score	Time (min)	RAM (%)	CPU (%)	Bytes	Packets
Baseline	0.9588	17.82	37.5	77.4	1,263.679	764,708
Encrypt	0.9588	18.13	41.4	78.3	1,192.876	746,932
MTD	0.9386	20.34	40.7	77	563.628	342,051
Encrypt+MTD	0.9187	20.45	41	73.6	566.933	341,209

Bảng 6.3.7: Hiệu suất hệ thống dưới các cấu hình bảo mật khác nhau (luật tổng hợp FedAvg và không chịu tấn công)

6.3. Kết quả thử nghiệm

Balance + No

Configuration	F1 score	Time (min)	RAM (%)	CPU (%)	Bytes	Packets
Baseline	0.9315	17.49	38.6	75.8	1,165.310	671,706
Encrypt	0.9308	17.78	41.1	76.1	1,200.434	770,145
MTD	0.911	20.27	40.7	77.8	633.53	360,617
Encrypt+MTD	0.9176	20.32	41.9	78.5	640.568	369,824

Bảng 6.3.8: Hiệu suất hệ thống dưới các cấu hình bảo mật khác nhau (luật tổng hợp Balance và không chịu tấn công)

FedAvg + Gauss

Configuration	F1 score	Time (min)	RAM (%)	CPU (%)	Bytes	Packets
Baseline	0.4327	17.76	38.7	77.0	1,067.184	678,972
Encrypt	0.4325	17.99	41.1	78.7	1,174.004	722,591
MTD	0.1529	19.79	41.2	78.5	588.326	335,856
Encrypt+MTD	0.1531	20.04	41.1	77.9	706.057	402,870

Bảng 6.3.9: Hiệu suất hệ thống dưới các cấu hình bảo mật khác nhau (luật tổng hợp FedAvg và tấn công Gauss)

Balance + Gauss

Configuration	F1 score	Time (min)	RAM (%)	CPU (%)	Bytes	Packets
Baseline	0.9474	17.35	37.7	77.6	1,061.338	656,630
Encrypt	0.936	18.02	41.7	77.3	1,248.179	737,313
MTD	0.8991	19.85	41.9	77.5	594.122	301,665
Encrypt+MTD	0.9183	20.02	42.8	77.5	606.99	341,798

Bảng 6.3.10: Hiệu suất hệ thống dưới các cấu hình bảo mật khác nhau (luật tổng hợp Balance và tấn công Gauss)

Ảnh hưởng của cơ chế bảo mật và MTD đến hiệu suất mô hình

Trong trường hợp không có tấn công, cấu hình Baseline đạt được độ chính xác mô hình cao nhất (0.9588 đối với luật tổng hợp FedAvg và 0.9315 cho luật tổng hợp BALANCE). Điều này thể hiện tiềm năng đạt hiệu suất cao khi không phải chịu chi phí từ các cơ chế bảo mật. Tuy nhiên, khi xem xét kết quả từ các thiết lập bảo mật khác, có thể thấy rằng hiệu suất mô hình giảm đi không đáng kể. Cụ thể, F1 score đều đạt được ≈ 0.91 cho cả hai luật tổng hợp trong

cấu hình kết hợp cả mã hoá và MTD, giá trị này chỉ giảm khoảng 0.03 so với cấu hình tốt nhất Baseline. Trên thực tế, sự suy giảm độ chính xác mô hình này đến từ việc sử dụng cơ chế phòng thủ MTD (do các node thực hiện trao đổi mô hình với số lượng hàng xóm ít hơn cấu hình gốc)

Đặc biệt khi chịu tấn công, các cơ chế bảo mật còn thể hiện khả năng kết hợp tốt với giải thuật tổng hợp BALANCE. Đối với luật tổng hợp cơ bản như FedAvg, chỉ số F1 score chỉ đạt được khoảng 0.15 trong các cấu hình có sử dụng MTD. Trong khi đó giá trị này lên tới 0.9183 cho cấu hình sử dụng cả mã hoá và MTD khi sử dụng luật tổng hợp BALANCE.

Ảnh hưởng của cơ chế bảo mật và MTD đến thời gian huấn luyện

Đối với các thí nghiệm sử dụng cấu hình cơ bản, thời gian hoàn thành 100 vòng huấn luyện là gần 18 phút. Khi kết hợp thêm cơ chế mã hoá, sẽ tốn thêm một khoảng thời gian dành cho việc mã hoá và giải mã dữ liệu, tuy nhiên theo kết quả thí nghiệm, khoảng thời gian này là rất nhỏ (tổng chưa tới 1 phút). Đối với cơ chế MTD, thời gian tăng thêm đến từ quá trình trao đổi message và đồng thuận để tạo ra tập con hàng xóm mới trong mỗi vòng. So sánh giữa các cấu hình có và không kết hợp MTD, thời gian huấn luyện tăng thêm khoảng 2 phút. Mặc dù có sự gia tăng về thời gian, nhưng mức độ tăng này không lớn và có thể coi là chấp nhận được trong bối cảnh các biện pháp bảo vệ được triển khai nhằm tăng cường an ninh cho hệ thống.

Ảnh hưởng của cơ chế bảo mật và MTD đến chi phí tài nguyên của hệ thống

Cấu hình có kết hợp mã hoá sẽ yêu cầu xử lý các tác vụ tính toán bổ sung như mã hoá và giải mã dữ liệu, trong khi cơ chế phòng thủ MTD yêu cầu thêm tài nguyên để hỗ trợ quá trình giao tiếp và đồng thuận tại mỗi vòng. Cụ thể, lấy ví dụ trường hợp ở bảng 6.3.8, mức sử dụng RAM và CPU tối đa ở cấu hình Baseline lần lượt là 38.6% và 75.8%. Tại cấu hình Encrypt và MTD, các giá trị này tăng khoảng 1-2%. Ở cấu hình đầy đủ nhất và sử dụng luật tổng hợp BALANCE, mức sử dụng RAM và CPU tối đa đạt 41.9% và 78.5%, giá trị này tăng so với cấu hình cơ bản 3-4%.

Nhìn chung, những giá trị này cho thấy việc sử dụng luật tổng hợp BALANCE cùng với các cơ chế bảo mật như mã hoá và MTD không tạo ra gánh nặng đáng kể đến hiệu năng tài nguyên của hệ thống, vẫn đảm bảo tính khả dụng và hiệu quả khi triển khai.

Ảnh hưởng của cơ chế bảo mật và MTD đến hiệu suất truyền thông

Kết hợp cơ chế mã hoá không làm tăng đột biến số lượng bytes và packets gửi/nhận, tức là hiệu suất truyền thông không bị ảnh hưởng tiêu cực. Đáng chú ý, khi kết hợp với MTD, hệ thống còn cho thấy xu hướng giảm số lượng bytes và packets trao đổi. Điều này được lý giải bởi MTD

6.3. Kết quả thử nghiệm

góp phần giảm số lượng hàng xóm tham gia trao đổi mô hình trong mỗi vòng huấn luyện, do đó số lượng bản cập nhật (và lưu lượng mạng) giảm xuống.

Tóm lại, các cơ chế bảo mật trên vẫn đảm bảo hiệu suất truyền thông ổn định cho hệ thống.

6.3.3 Ảnh hưởng của các tham số khác

6.3.3.1 Cấu trúc mạng

Topology	F1 score	Time (min)	RAM (%)	CPU (%)	Bytes	Packets
Full	0.9448	19.54	46.9	79.3	1,669.086	718,575
Partial	0.9474	17.35	37.7	77.6	1,061.338	656,630
Ring	0.6459	18.71	41.1	76.8	106.294	84,848

Bảng 6.3.11: Ảnh hưởng của cấu trúc mạng đối với luật tổng hợp BALANCE

Thí nghiệm trên được thực hiện dưới tấn công Gauss với tỉ lệ khách hàng độc hại là 20%. Nhìn chung, với cấu trúc mạng đầy đủ, độ chính xác mô hình không có nhiều thay đổi so với các thí nghiệm được thực hiện ở phần trên (đồ thị ngẫu nhiên Erdős-Rényi $p = 0.5$). Riêng đối với cấu trúc mạng vòng, độ chính xác mô hình đặc biệt bị ảnh hưởng do giới hạn về số hàng xóm trao đổi. Giá trị 0.6459 được ghi nhận là F1 score thấp nhất do có kết nối với một node độc hại.

6.3.3.2 Tỉ lệ khách hàng độc hại

Malicious	F1 score	Time (min)	RAM (%)	CPU (%)	Bytes	Packets
20%	0.9474	17.35	37.7	77.6	1,061.338	656,630
40%	0.9329	17.85	43.5	79.4	1,109.522	632,235
60%	0.9297	17.67	43.5	77	1,033.317	647,916
80%	0.9309	17.4	43.3	75.2	1,165.438	613,122

Bảng 6.3.12: Ảnh hưởng của tỉ lệ khách hàng độc hại đối với luật tổng hợp BALANCE

Tấn công Gauss được sử dụng cho các thí nghiệm này. Bảng trên cho thấy giá trị F1 score ổn định của mô hình sử dụng luật tổng hợp BALANCE dưới những điều kiện tấn công khác nhau. Đáng chú ý, ở môi trường lên tới 80% tỉ lệ khách hàng độc hại, những node tốt vẫn đạt được độ chính xác mô hình ≈ 0.9309 .

6.3.3.3 Tỷ lệ chấp nhận hàng xóm trong MTD

	F1 score	Time (min)	RAM (%)	CPU (%)	Bytes	Packets
no-0.3	0.887	19.64	44.5	77.9	364.765	248,678
no-0.5	0.9176	20.32	41.9	78.5	640.568	369,824
no-0.7	0.9224	20.38	44.4	78.7	857.779	418,130
no-0.9	0.9126	21.13	45.1	79.8	1,072.863	473,776
gauss-0.3	0.8892	19.57	43.4	77	394.117	264,869
gauss-0.5	0.9183	20.02	42.8	77.5	606.99	341,798
gauss-0.7	0.9238	20.32	42.7	78.9	794.761	383,453
gauss-0.9	0.9353	20.53	42.9	79	964.251	447,367

Bảng 6.3.13: Ảnh hưởng của tỷ lệ chấp nhận hàng xóm trong MTD

Trong kĩ thuật bảo vệ MTD, tại mỗi vòng huấn luyện, các node sẽ chọn ra một tập con từ tập các node hàng xóm để trao đổi mô hình. Tham số tỷ lệ chấp nhận hàng xóm sẽ quyết định số lượng node trong tập con đó và ảnh hưởng trực tiếp đến hiệu suất hệ thống.

Bảng trên so sánh hiệu suất hệ thống khi áp dụng cấu hình bảo mật tối đa (kết hợp cả Mã hoá và MTD) với tham số chấp nhận hàng xóm thay đổi từ 0.3-0.9, dưới điều kiện lý tưởng và điều kiện bị tấn công bởi Gauss (tỷ lệ Malicious là 20%).

Đối với độ chính xác mô hình, F1 score giảm xuống dưới 0.9 khi tỷ lệ chấp nhận hàng xóm chỉ là 0.3. Đây là kết quả khá rõ ràng khi số mô hình trao đổi thực tế quá ít so với quy mô chung của mạng. Khi tỷ lệ chấp nhận hàng xóm trên 0.5, độ chính xác mô hình cho thấy tính ổn định hơn đáng kể. Các thông số liên quan tới lưu lượng mạng cho thấy thay đổi tỷ lệ ứng với tham số tỷ lệ chấp nhận hàng xóm từ MTD.

Như vậy, có thể tùy theo nhu cầu và đánh đổi giữa chi phí truyền thông và độ chính xác mô hình cũng như cấu trúc mạng để lựa chọn mức chấp nhận phù hợp, tuy nhiên giá trị này vẫn nên lớn hơn hoặc bằng 0.5 để đảm bảo độ chính xác mô hình tổng thể của hệ thống.

6.3.3.4 Các tham số trong BALANCE

Các thí nghiệm trong bài báo cáo đều sử dụng thiết lập tham số như sau:

- $\gamma = 1.5$
- $\kappa = 1.0$
- $\alpha = 0.5$
- $\lambda(t) = \frac{t}{T}$

6.3. Kết quả thử nghiệm

Chúng tôi lựa chọn các tham số này vì nhận thấy giá trị ngưỡng chấp nhận *threshold* vừa đủ tương thích với khoảng cách giữa các node lân cận trong ngữ cảnh hệ thống và tập dữ liệu được sử dụng.

Khi áp dụng phương pháp tổng hợp BALANCE cho các môi trường, tập dữ liệu hay mô hình huấn luyện khác nhau, các tham số trên cần được cân nhắc và thay đổi linh hoạt để đạt được hiệu suất tốt nhất.

Phần hiện thực BALANCE của chúng tôi sử dụng khoảng cách Euclidean(L2) để xác định chênh lệch giữa hai mô hình. Hàm khoảng cách này cũng có thể thay đổi linh hoạt tùy theo cấu trúc của mô hình huấn luyện. Trong trường hợp giá trị khoảng cách giữa hai mô hình lân cận giảm đi, người dùng có thể giảm giá trị γ xuống cho phù hợp.

Tham số α thể hiện mức độ kết hợp giữa mô hình cục bộ và trung bình các mô hình hàng xóm tương đồng. Giá trị trên có thể linh hoạt tùy theo trạng thái an toàn của hệ thống. Ví dụ, với các hệ thống nhận thấy ít bị tấn công, có thể giảm tham số α xuống, chấp nhận các mô hình hàng xóm nhiều hơn để đẩy nhanh quá trình hội tụ. Ngược lại, với các hệ thống nhận thấy thường xuyên bị tấn công, có thể tăng tham số α lên để hạn chế tác động của các mô hình xung quanh.

Chương 7

Kết luận

7.1 Kết luận

Trong đề án này, nhóm chúng tôi đã thực hiện các nội dung chính sau:

1. Nghiên cứu cơ sở lý thuyết về học liên kết phi tập trung

- Khái quát về hệ thống học liên kết phi tập trung Decentralized Federated Learning và các bài toán liên quan.

2. Thiết kế và triển khai giải thuật BALANCE kết hợp TLS và MTD

- Hiện thực giải thuật BALANCE để tổng hợp mô hình công bằng và ngăn chặn thao túng trọng số.
- Ứng dụng TLS 1.3 đảm bảo tính toàn vẹn và xác thực kênh truyền.
- Kết hợp Moving Target Defense (MTD) dựa trên giao thức coin-flipping để thay đổi cấu hình mạng liên tục.

3. Thử nghiệm, so sánh và đánh giá hiệu năng

- Thiết lập thí nghiệm với Gauss, Noise Injection, Weight Swapping, Krum attack. . .
- So sánh chất lượng mô hình và tốc độ hội tụ giữa BALANCE và các phương pháp khác.
- Đo lường chi phí tài nguyên (CPU, băng thông, thời gian mã hóa) khi áp dụng TLS và MTD.

7.2 Hướng phát triển cho tương lai

1. Tối ưu hóa tham số BALANCE

- Hiện thực các giải thuật giúp tự động tối ưu giá trị tham số α , γ , κ , và $\lambda(t)$ cho mỗi ngữ cảnh hệ thống khác nhau.
- Sử dụng các phương pháp tối ưu hóa như Grid Search, Random Search, và Bayesian Optimization.
- Kỳ vọng cải thiện độ chính xác, khả năng kháng tấn công, và tốc độ hội tụ.

2. Áp dụng kỹ thuật Differential Privacy nhằm nâng cao bảo vệ riêng tư dữ liệu

- Nghiên cứu tích hợp cơ chế Differential Privacy (DP) vào quá trình tổng hợp mô hình, ví dụ: thêm Gaussian noise hoặc Laplace noise vào gradient/model updates.
- Xác định và cân chỉnh mức độ bảo vệ riêng tư (tham số ϵ , δ) để cân bằng giữa độ chính xác mô hình và mức độ riêng tư.
- Đánh giá ảnh hưởng của DP tới tốc độ hội tụ và chất lượng mô hình dưới các kịch bản tấn công khác nhau, từ đó đề xuất chiến lược phối hợp BALANCE + DP hiệu quả.

3. Triển khai hệ thống trên các thiết bị cứng biên (edge devices)

- Thử nghiệm triển khai trên các nền tảng phần cứng phổ biến như Raspberry Pi, Jetson Nano, điện thoại thông minh để đánh giá khả năng tính toán và tiêu thụ năng lượng.
- Tối ưu hóa các thành phần TLS 1.3 và coin-flipping MTD để giảm thiểu độ trễ và chi phí mã hóa/truyền thông trên thiết bị tài nguyên hạn chế.
- Xây dựng bộ công cụ quản lý và cập nhật cấu hình MTD (firmware, mạng ảo) cho phép triển khai, giám sát và cấu hình động trực tiếp trên phần cứng phân tán.

Tài liệu tham khảo

- [1] Ehsan Hallaji, Roozbeh Razavi-Far, Mehrdad Saif, Boyu Wang, and Qiang Yang. Decentralized federated learning: A survey on security and privacy. *arXiv preprint arXiv:2401.17319v1*, 2024. Supported by the Natural Sciences and Engineering Research Council of Canada (NSERC).
- [2] Enrique Tomás Martínez Beltrán, Pedro Miguel Sánchez Sánchez, Sergio López Bernal, G r me Bovet, Manuel Gil P rez, Gregorio Mart nez P rez, and Alberto Huertas Celdr n. Mitigating communications threats in decentralized federated learning through moving target defense. *arXiv preprint arXiv:2307.11730v2*, 2023.
- [3] Enrique Tomás Mart nez Beltr n, Mario Quiles P rez, Pedro Miguel S nchez S nchez, Sergio L pez Bernal, G r me Bovet, Manuel Gil P rez, Gregorio Mart nez P rez, and Alberto Huertas Celdr n. Decentralized Federated Learning: Fundamentals, State of the Art, Frameworks, Trends, and Challenges. *IEEE Communications Surveys & Tutorials*, 25(4):2860–2896, 2023.
- [4] Minghong Fang, Zifan Zhang, Prashant Khanduri, Jia Liu, Songtao Lu, Yuchen Liu, Neil Gong, et al. Byzantine-robust decentralized federated learning. *arXiv preprint arXiv:2406.10416*, 2024.
- [5] CyberHoot. Transport layer security (tls), 2025. Accessed: 2025-03-12.
- [6] CyberSafeNV. Changes in ssl and tls support in 2024, 2024. Accessed: 2025-03-12.
- [7] Enrique Tom s Mart nez Beltr n,  ngel Luis Perales G mez, Chao Feng, Pedro Miguel S nchez S nchez, Sergio L pez Bernal, G r me Bovet, Manuel Gil P rez, Gregorio Mart nez P rez, and Alberto Huertas Celdr n. Fedstellar: A platform for decentralized federated learning. *arXiv preprint arXiv:2306.09750 [cs.DC]*, Jun 2023.
- [8] Coban. Gradient descent. Guide to Gradient Descent: [Read here](#) (accessed: 10.12.2024), 2017.

- [9] Quy Nguyen. Gradient descent. Article on Gradient Descent: [Read here](#) (accessed: 03.04.2016), 2016.
- [10] GeeksforGeeks. Stochastic gradient descent (sgd). Tutorial on Stochastic Gradient Descent: [Read here](#) (accessed: 10.12.2024), 2023.
- [11] Nerd for Tech. Linear regression from scratch pt3: Stochastic gradient descent. Post on Stochastic Gradient Descent: [Read here](#) (accessed: 10.12.2024), 2020.
- [12] Muhammad Asad, Saima Shaukat, Dou Hu, Zekun Wang, Ehsan Javanmardi, Jin Nakazato, and Manabu Tsukada. Limitations and future aspects of communication costs in federated learning: A survey. *Sensors*, 23(17), 2023.
- [13] Miguel Fernández Llamas, Jérôme Bovet, Gregorio Martínez Pérez, Alberto Huertas Celdrán, Pedro Miguel Sánchez Sánchez, Enrique Tomás Martínez Beltrán. Profe: Communication-efficient decentralized federated learning via distillation and prototypes. *arXiv preprint arXiv:2412.11207v1*, 2024.
- [14] Shuyu Zhang, Haoyu Jin, Wen Xia, Binxing Fang, Donglei Wu, Xiangyu Zou. Smartidx: Reducing communication cost in federated learning by exploiting the cnns structures. In *Proceedings of the Thirty-Sixth AAAI Conference on Artificial Intelligence (AAAI-22)*, Harbin Institute of Technology, Shenzhen, China, 2022. Association for the Advancement of Artificial Intelligence (AAAI).
- [15] Authors from SCIRP. Advances, challenges & recent developments in federated learning. *Scientific Research Publishing*, Oct 2024.
- [16] Ligeng Zhu, Zhijian Liu, and Song Han. Deep leakage from gradients. *arXiv preprint arXiv:1906.08935*, 2019.
- [17] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. *arXiv preprint arXiv:1610.05820v2*, 2017.
- [18] Shangwei Guo, Tianwei Zhang, Han Yu, Xiaofei Xie, Lei Ma, Tao Xiang, and Yang Liu. Byzantine-resilient decentralized stochastic gradient descent. *IEEE Transactions on Circuits and Systems for Video Technology*, 32(6):4096–4106, 2021.
- [19] El Mahdi El-Mhamdi, Sadegh Farhadkhani, Rachid Guerraoui, Arsany Guirguis, Lê-Nguyên Hoàng, and Sébastien Rouault. Collaborative learning in the jungle (decentralized, byzantine, heterogeneous, asynchronous and nonconvex learning). *NeurIPS*, 2021.

- [20] Lie He, Sai Praneeth Karimireddy, and Martin Jaggi. Byzantine-robust decentralized learning via self-centered clipping. *arXiv preprint arXiv:2202.01545*, 2023.
- [21] Wikipedia contributors. Elliptic-curve Diffie-Hellman, 2025. Accessed: 2025-03-12.
- [22] River Financial. What are schnorr signatures?, 2025. Accessed: 2025-03-12.
- [23] Wikipedia contributors. Galois/Counter Mode, 2025. Accessed: 2025-03-12.
- [24] National Cybersecurity Center of Excellence (NCCoE). Addressing visibility challenges in tls 1.3, 2025. Accessed: 2025-03-12.
- [25] Internet Engineering Task Force (IETF). Tls 1.3: The new standard, 2018. Accessed: 2025-03-12.
- [26] Stack Exchange User. How to fairly select a random number for a game without trusting a third party?, 2011. Accessed: 2025-03-12.
- [27] Stack Exchange User. What are the pros and cons of Pedersen commitments vs. hash-based commitments?, 2018. Accessed: 2025-03-12.
- [28] Stack Exchange User. Computational requirements for breaking sha-256, 2018. Accessed: 2025-03-12.
- [29] Stack Exchange User. Why haven't any sha-256 collisions been found yet?, 2017. Accessed: 2025-03-12.
- [30] Shiqiang Wang and Mingyue Ji. A unified analysis of federated learning with arbitrary client participation. *arXiv preprint*, 2024. Accessed: 2025-03-12.
- [31] Python Software Foundation. *secrets — Generate secure random numbers for managing secrets*, 2025. Accessed: 2025-03-12.
- [32] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [33] Ligeng Zhu, Zhijian Liu, and Song Han. Deep leakage from gradients. In *Advances in Neural Information Processing Systems*, 2019.
- [34] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. *NeurIPS*, 2017.

- [35] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. *AISTATS*, 2017.
- [36] Dong Yin, Yudong Chen, Kannan Ramchandran, and Peter Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. *ICML*, 2018.
- [37] Xiaoyu Cao, Minghong Fang, Jia Liu, and Neil Zhenqiang Gong. Fltrust: Byzantine-robust federated learning via trust bootstrapping. *NDSS*, 2021.
- [38] Gregory Cohen, Saeed Afshar, Jonathan Tapson, and Andre Van Schaik. Emnist: Extending mnist to handwritten letters. In *2017 international joint conference on neural networks (IJCNN)*, pages 2921–2926. IEEE, 2017.
- [39] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.
- [40] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- [41] Jonas Geiping, Liam Fowl, W. Ronny Huang, Wojciech Czaja, Gavin Taylor, Michael Moeller, and Tom Goldstein. Witches’ brew: Industrial scale data poisoning via gradient matching. *arXiv preprint arXiv:2009.02276*, 2021.